



**Politechnika
Śląska**

PRACA MAGISTERSKA

Ziarnisty algorytm aglomeracyjnego grupowania hierarchicznego

Zofia LORENC

Nr albumu: 300306

Kierunek: Informatyka

Specjalność: Oprogramowanie Systemowe

PROWADZĄCY PRACĘ

dr hab. inż. Krzysztof Simiński

KATEDRA ALGORYTMIKI I OPROGRAMOWANIA

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

Tytuł pracy

Ziarnisty algorytm aglomeracyjnego grupowania hierarchicznego

Streszczenie

Aglomeracyjne grupowanie hierarchiczne polega na iteracyjnym scalaniu najbliższych sobie grup danych aż do uzyskania z góry ustalonej ich liczby. Wadą tego algorytmu jest duża złożoność czasowa w zależności od liczby danych wejściowych. W pracy zaproponowany został rozmyty algorytm grupowania hierarchicznego, który rozszerza algorytm standardowy o wykorzystanie ziaren informacyjnych mających postać wielowymiarowych liczb rozmytych. Ziarna te wyznaczone są na podstawie wyników algorytmu rozmytych c -średnich o niskiej złożoności czasowej. Modyfikacja algorytmu ma na celu zmniejszenie czasu wykonywania grupowania. W pracy badany jest również wpływ wprowadzenia rozmycia danych na jakość grupowania.

Słowa kluczowe

grupowanie hierarchiczne, fuzzy C-means, obliczenia ziarniste, liczby rozmyte, Python

Thesis title

Granular agglomerative hierarchical clustering algorithm

Abstract

In agglomerative hierarchical clustering, groups of data that are the closest to each other are iteratively merged until a predetermined number of clusters is reached. A drawback of this algorithm is its high time complexity based on the amount of input data. This paper proposes a fuzzy hierarchical clustering algorithm that extends the standard algorithm with the usage of information granules, represented by multidimensional fuzzy numbers. These granules are calculated based on fuzzy c -means algorithm's results, which has a low time complexity. The modification of the algorithm aims to decrease execution time of the clustering. The paper also examines the impact of introducing data fuzziness on clustering quality.

Key words

hierarchical clustering, fuzzy C-means, granular computing, fuzzy numbers, Python

Spis treści

1	Wstęp	1
2	Grupowanie danych	3
2.1	Algorytmy grupowania hierarchicznego	3
2.1.1	Single Linkage	5
2.1.2	Complete Linkage	8
2.2	Algorytm rozmytych c średnich	9
3	Obliczenia ziarniste	13
3.1	Ziarna informacyjne	13
3.2	Zbiory rozmyte	13
3.2.1	Operacje na zbiorach rozmytych	14
3.3	Rozszerzalne liczby rozmyte	15
3.3.1	Kształty liczb rozmytych	16
3.3.2	Operacje arytmetyczne	16
3.3.3	Operacje logiczne	17
3.4	Rozmyty algorytm aglomeracyjnego grupowania hierarchicznego	18
3.4.1	Granulacja danych oryginalnych	19
3.4.2	Przekształcenie granul	19
3.4.3	Rozmyte grupowanie hierarchiczne	19
3.5	Badania	20
3.5.1	Zbiory danych	20
3.5.2	Metodyka badań	23
3.5.3	Wyniki	33
3.6	Podsumowanie	69
	Bibliografia	75
	Dokumentacja techniczna	81
	Spis skrótów i symboli	87

Spis rysunków	91
Spis tabel	94

Rozdział 1

Wstęp

Grupowanie danych, nazywane też klasteryzacją, jest powszechnie wykorzystywane do analizy danych. Uogólniając, polega ono na znalezieniu takich klas (grup, klastrów) elementów, żeby elementy należące do tej samej klasy były do siebie jak najbardziej podobne według pewnej wybranej metryki, a należące do różnych klas – niepodobne. Istnieje wiele rodzajów algorytmów klasteryzacyjnych, między innymi hierarchiczne, gęstościowe, grafowe. Wybór algorytmu zależy od charakteru danych, które mają zostać zgrupowane – na przykład czy dane zawierają zaszumienie albo czy tworzą wyraźnie oddzielone grupy.

W pracy rozważany będzie algorytm aglomeracyjnego grupowania hierarchicznego. Polega on na iteracyjnym łączeniu najbliższych sobie mniejszych klastrów (gdzie najmniejszym klastrem jest pojedynczy punkt danych) w większe, aż do uzyskania z góry określonej ich liczby. Złączanie odbywa się na podstawie wybranych metryk odległości klastrów. Problemem tej rodziny algorytmów jest ich wysoka złożoność czasowa w zależności od ilości danych wejściowych – $O(n^3)$ w implementacji naiwnej, choć istnieją specjalizowane implementacje o lepszych złożonościach.

Do ulepszenia takiego algorytmu można wykorzystać obliczenia granularne. Jest to szeroka dziedzina pozwalająca na analizę danych na sposób zbliżony do ludzkiego pojmowania rzeczywistości. Bezpośrednio związanym z nią pojęciem jest ziarno informacyjne – encja o pewnym znaczeniu, grupa podobnych obiektów. Ziarna mogą być reprezentowane jako na przykład zbiory rozmyte, czyli takie zbiory, do których dane mogą należeć w pewnym stopniu i każda dana może należeć do wielu zbiorów jednocześnie. Szczególnym przypadkiem zbioru rozmytego jest liczba rozmyta, która modeluje daną liczbę ostrą w pewnym przybliżeniu. Takie zbiory mogą być uzyskane na przykład jako wynik działania algorytmu rozmytych c -średnich, który jest rodzajem algorytmu klasteryzacyjnego. Otrzymane w ten sposób rozmyte klastry mogą być ostatecznym wynikiem grupowania, jednak można je też wykorzystać jako krok pośredni – dalej grupować je innymi algorytmami. Wymaga to jednak modyfikacji tych algorytmów, aby mogły działać zgodnie z logiką rozmytą, jako że zarówno operacje arytmetyczne, jak i logiczne, muszą uwzględnić rozmycie klastrów.

Celem niniejszej pracy będzie opracowanie i przebadanie własności granularnego algorytmu aglomeracyjnego grupowania hierarchicznego, który operować będzie na ziarnach informacyjnych uzyskanych w wyniku grupowania danych algorytmem rozmytych c -średnich.

Algorytm ten ma niższą złożoność czasową – liniową w zależności od liczby danych – powinien więc przyspieszyć proces grupowania. Jako że wprowadzone zostanie rozmycie danych, dopuszczalne jest obniżenie jakości grupowania.

W pracy wykorzystano następującą notację: kursywą i małą literą oznacza się skalar bądź element zbioru (x), wielką literą kreśloną – zbiór, w którym się znajduje ($\mathbb{X}$), kursywą i wielką literą – liczebność zbioru (X), pogrubieniem i małą literą – wektory ($\mathbf{x}$), a pogrubieniem i wielką literą – macierze ($\mathbf{X}$).

W rozdziałach 2 i 3 przedstawiono teoretyczną analizę zagadnień związanych z tematem. Rozdział 2 przedstawia algorytmy grupowania danych, w szczególności algorytm hierarchiczny oraz algorytm rozmytych c -średnich. W rozdziale 3 opisano istotne dla rozwiązania elementy logiki rozmytej, w tym rozszerzalne liczby rozmyte. Rozdział 3.4 zawiera omówienie własnej implementacji rozmytego algorytmu aglomeracyjnego grupowania hierarchicznego i konieczne kroki przygotowujące dane. W rozdziale 3.5 analizowane jest działanie tego algorytmu. Przedstawione są wykorzystane dane oraz metodyki badania czasu i jakości grupowania, a następnie w oparciu o nie omawiane są wyniki grupowania.

Rozdział 2

Grupowanie danych

Grupowanie jest wykorzystywane do analizy danych. Umożliwia znalezienie podobieństw między rozpatrywanymi obiektami i sklasyfikowanie ich. Grupowanie (klasteryzacja) danych polega na przypisaniu elementów do klas o pewnym znaczeniu, gdzie elementy tej samej klasy są podobne do siebie nawzajem [10, 27], a równocześnie niepodobne do obiektów z innych grup. Algorytmy grupowania są algorytmami nienadzorowanymi – mają jedynie informacje o podobieństwie między poszczególnymi elementami.

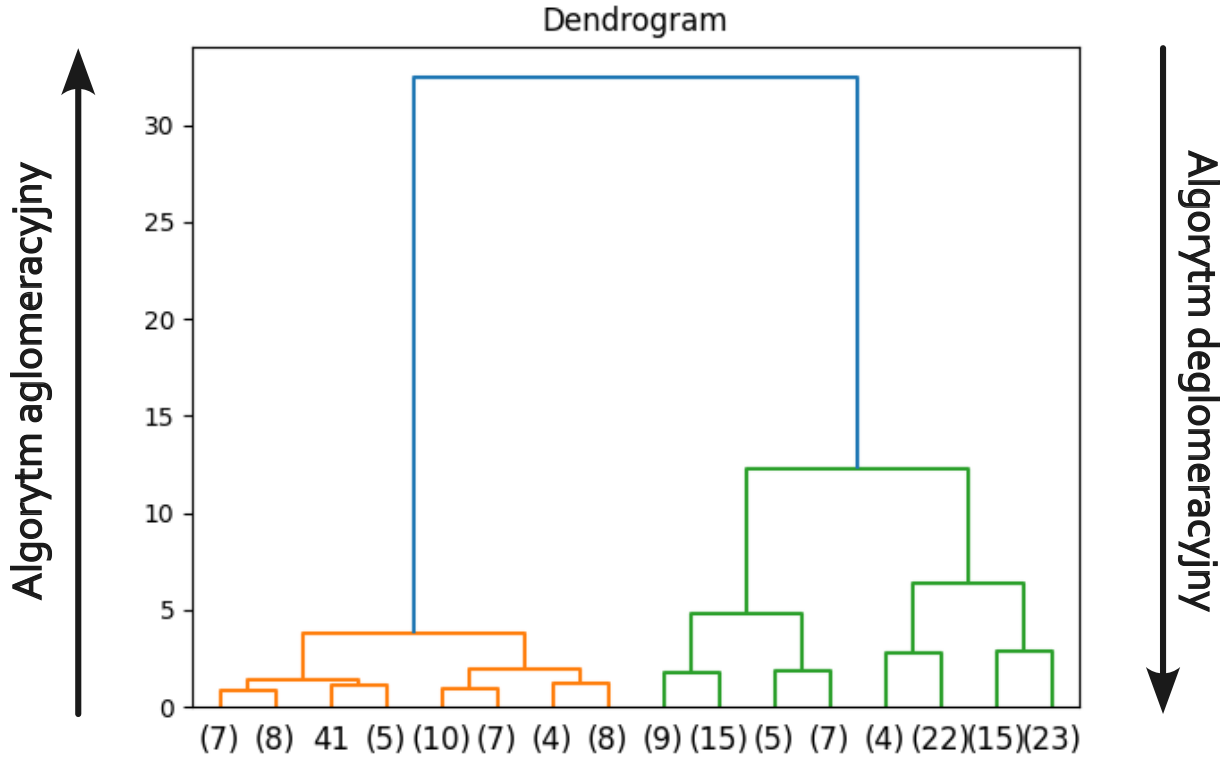
Do głównych rodzin algorytmów grupowania zaliczane są [27]:

- Algorytmy hierarchiczne: grupowanie polega na utworzeniu sekwencji zagnieżdżonych podziałów danych [28],
- Algorytmy oparte na minimalizacji funkcji kryterialnej: związane z iteracyjnym wyznaczaniem środków klastrów do momentu spełnienia kryterium zatrzymania (na przykład: k -means, fuzzy c -means),
- Algorytmy gęstościowe: obszary o dużej gęstości danych tworzą klastry, pozostałe uznawane są za szumy (na przykład: DBSCAN [8], OPTICS [3]),
- Algorytmy grafowe: dane uznawane są za wierzchołki grafów, a podobieństwa między nimi – za krawędzie (na przykład algorytmy oparte na minimalnym drzewie rozpinającym),
- Inne algorytmy (na przykład: DBCLASD [29], CLIQUE [1]).

2.1 Algorytmy grupowania hierarchicznego

Grupowanie hierarchiczne polega na utworzeniu sekwencji zagnieżdżonych podziałów danych [28].

Algorytm aglomeracyjnego grupowania hierarchicznego iteracyjnie scala mniejsze klastry danych w większe, zaczynając od klastrów jednoelementowych (każdy punkt danych



Rysunek 2.1: Dendrogram otrzymany na podstawie grupowania hierarchicznego o łączności SINGLELINKAGE przykładowego zbioru danych ‘iris’.

jest klastrem), kończąc na jednym wspólnym klastrze. Kolejne dwa klastry, które mają zostać scalone w danej iteracji, wybierane są na podstawie metryki odległości między nimi oraz łączności (ang. *linkage*). Przykładami tego typu algorytmów są SLINK [22] i BIRCH [30].

Odwrotnością algorytmu aglomeracyjnego jest algorytm deglomeracyjny (ang. *divisive*), w którym na początku wszystkie dane znajdują się w jednym klastrze, który jest iteracyjnie dzielony na mniejsze, aż do uzyskania klastrów wyłącznie jednoelementowych. Przykładem takiego algorytmu jest DIANA [17].

Proces hierarchicznego scalania (lub rozdzielania) klastrów można zwizualizować jako dendrogram (rys. 2.1). Wykres dendrogramu zazwyczaj przycina się, aby pokazywał tylko kilka ostatnich scaleń. Oś pozioma reprezentuje scalane klastry: jeżeli klastrem jest pojedynczy element, to reprezentuje go jego indeks, jeżeli zawiera więcej elementów – jego liczebność podana w nawiasach. Oś pionowa odpowiada odległości między scalanymi klastrami.

Często wybierana euklidesowa metryka odległości [19] zdefiniowana jest jako

$$d_e(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{a=0}^A (x_{ia} - x_{ja})^2}, \quad (2.1)$$

gdzie A to liczba atrybutów punktu danych $\mathbf{x}$, a x_{ia} oznacza a -ty atrybut i -tego punktu.

Metryka ta jest uszczegółowieniem metryki Minkowskiego

$$d_m(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{a=0}^A |x_{ia} - x_{ja}|^p \right)^{1/p}, \quad (2.2)$$

dla $p = 2$. Inną popularną metryką jest odległość Manhattan, również należąca do rodziny metryk Minkowskiego, dla $p = 1$.

Łączność określa, jak na podstawie odległości pomiędzy poszczególnymi punktami należącymi do klastrów wyznaczyć odległość pomiędzy tymi klastrami. Do łączności zaimplementowanych w popularnych bibliotekach należą *single linkage* (najbliższe sąsiedztwo), *complete linkage* (najdalsze sąsiedztwo), *average linkage* (średnie sąsiedztwo) oraz metoda Warda (minimalna wariancja) [12]. Implementacje aglomeracyjnego algorytmu hierarchicznego przedstawione są w [18]. Wykorzystywane one są przez bibliotekę *SciPy* języka Python [16].

Algorytm naiwny (PRIMITIVECLUSTERING) może być wykorzystany dla każdej łączności, jednak jego wadą jest złożoność czasowa wynosząca $O(n^3)$. Jego parametrami wejściowymi są lista etykiet danych oraz macierz podobieństwa między danymi. W algorytmie naiwnym, iteracyjnie wykonywane są trzy kroki:

1. Znalezienie w macierzy podobieństwa minimalnej wartości – znalezienie najbliższej sobie pary klastrów,
2. Scalenie odnalezionych klastrów,
3. Aktualizacja wartości podobieństw w macierzy według wybranej łączności.

Pseudokod działania algorytmu przedstawia alg. 1.

Algorytm ogólny `GENERAL_LINKAGE()` zawiera dodatkową optymalizację w stosunku do algorytmu naiwnego: wykorzystywana jest kolejka priorytetowa zawierająca kandydatów na najbliższych sąsiadów klastrów, co pozwala ograniczyć czas poszukiwania minimum macierzy w kroku 1. Optymistyczna złożoność czasowa tego algorytmu wynosi $\Omega(n^2)$, pesymistyczna – nadal $O(n^3)$.

2.1.1 Single Linkage

W algorytmie `SINGLELINKAGE` [28] odległość między klastrami jest równa minimalnej odległości między dwoma punktami należącymi do tych klastrów. Oznacza to, że w momencie złączenia klastrów A_1 i A_2 w jeden klaster A , odległość tego klastra do innego klastra B zdefiniowana jest jako:

$$d(A, B) = \min(d(A_1, B), d(A_2, B)). \quad (2.3)$$

Algorytm 1 Naiwny algorytm aglomeracyjnego grupowania hierarchicznego PRIMITIVECLUSTERING [18]

Require: $\mathbf{e}$: etykiety elementów

Require: $\mathbf{D}$: macierz podobieństwa między każdą parą elementów

```

1: procedure PRIMITIVECLUSTERING( $S, \mathbf{D}$ )
2:    $N \leftarrow$  liczba elementów
3:    $L \leftarrow []$   $\triangleright$  Lista wynikowa zawierająca trójki: para klastrów łączonych w danej
   iteracji i odległość między nimi
4:    $rozmiar \leftarrow []$   $\triangleright$  rozmiar klastrów o danych indeksach
5:   for  $x$  in  $\mathbf{e}$  do
6:      $rozmiar[x] \leftarrow 1$   $\triangleright$  na początku wszystkie klastry są jednoelementowe
7:   end for
8:   for  $i = 0$  to  $N - 2$  do
9:      $(a, b) \leftarrow \arg \min_{(\mathbf{e} \times \mathbf{e}) \setminus \Delta} \mathbf{D}$   $\triangleright$  Operacja  $\arg \min$  na macierzy  $\mathbf{D}$ , uwzględniająca
     elementy ze zbioru  $\mathbf{e}$  z wyłączeniem przekątnej głównej
10:     $\mathbf{e} \leftarrow \mathbf{e} \setminus \{a, b\}$   $\triangleright$  Klastry  $a$  i  $b$  usuwane są ze zbioru rozpatrywanych etykiet
11:    nowy element  $n \notin \mathbf{e}$   $\triangleright$  Do zbioru etykiet dodawany jest nowy element,
    odpowiadający klastrowi powstałemu po złączeniu  $a$  i  $b$ 
12:    for  $x$  in  $S$  do
13:       $\mathbf{D}[n, x] \leftarrow \mathbf{D}[x, n] \leftarrow \text{UPDATEFORMULA}(\dots)$   $\triangleright$  wzór dla odpowiedniej
      łączności, np. alg. 2, alg. 3
14:    end for
15:     $rozmiar[n] \leftarrow rozmiar[a] + rozmiar[b]$ 
16:     $S \leftarrow S \cup n$ 
17:  end for
18:  return  $L$   $\triangleright$  Zwracana lista zawiera zapis kolejnych scalań klastrów.
19: end procedure

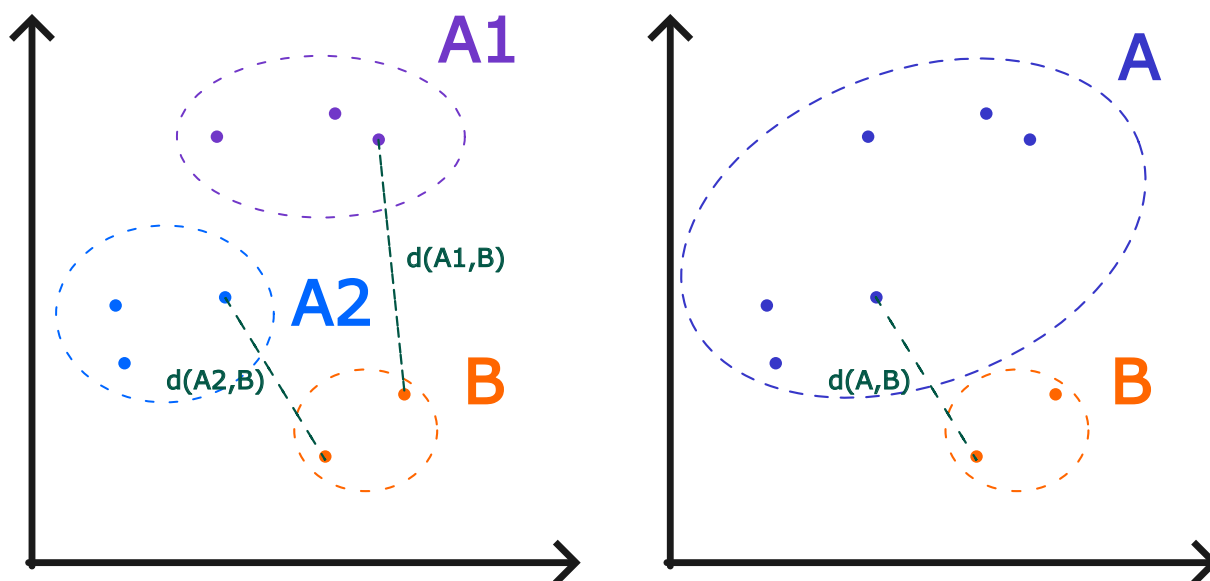
```

Wzór ten można przedstawić również w postaci pseudokodu 2, którym uzupełnić można algorytm 1.

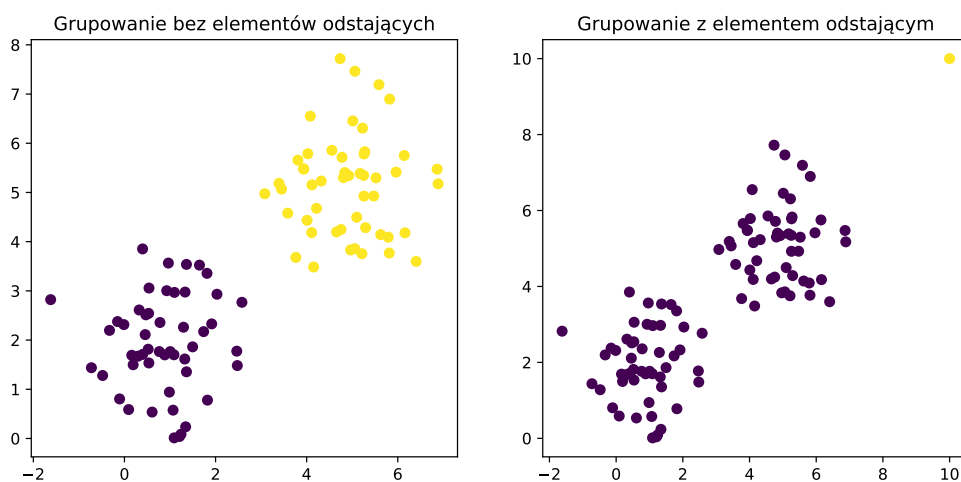
Przykład takiego przypisania ilustruje rys. 2.2. Odległość klastra B od klastra A_1 jest znana i wynosi $d(A_1, B)$, czyli jest ona najmniejszą odległością między dowolnymi dwoma elementami klastrów B i A_1 . Odległość klastra B od klastra A_2 również jest znana i wynosi $d(A_2, B)$. Klastry A_1 i A_2 scalane są w jeden klaster A , należy więc obliczyć odległość nowopowstałego klastra od pozostałych. Zamiast porównywać wszystkie możliwe kombinacje elementów klastrów A i B , wystarczy znaleźć minimum z dwóch wartości: $d(A_1, B)$ i $d(A_2, B)$. W tym przypadku jest to odległość $d(A_2, B)$, czyli $d(A, B) = d(A_2, B)$

Algorytm ten jest wrażliwy na dane odstające, szumy. Pojedynczy punkt znajdujący się w dużej odległości od reszty danych zostanie uznany za osobny klaster. Dodatkowym efektem tego błędnego przypisania będzie złączenie dwóch innych klastrów.

Zachowanie to ilustruje rys. 2.3. Gdy dane nie zawierają znacznie odstających danych, algorytm wyznacza dwa klastry zgodnie z oczekiwaniami. Po dodaniu do nich pojedynczej danej o współrzędnej (10, 10), czyli w dużej odległości od reszty danych, dana ta klasyfikowana jest jako osobny klaster.



Rysunek 2.2: Ilustracja działania algorytmu SINGLELINKAGE. Odległość klastra B od klastrów A_1 i A_2 jest znana. Po złączeniu A_1 i A_2 w klaster A , nowa odległość od klastra B jest równa minimalnej z poprzednich dwóch – w tym przypadku jest to odległość $d(A_2, B)$.

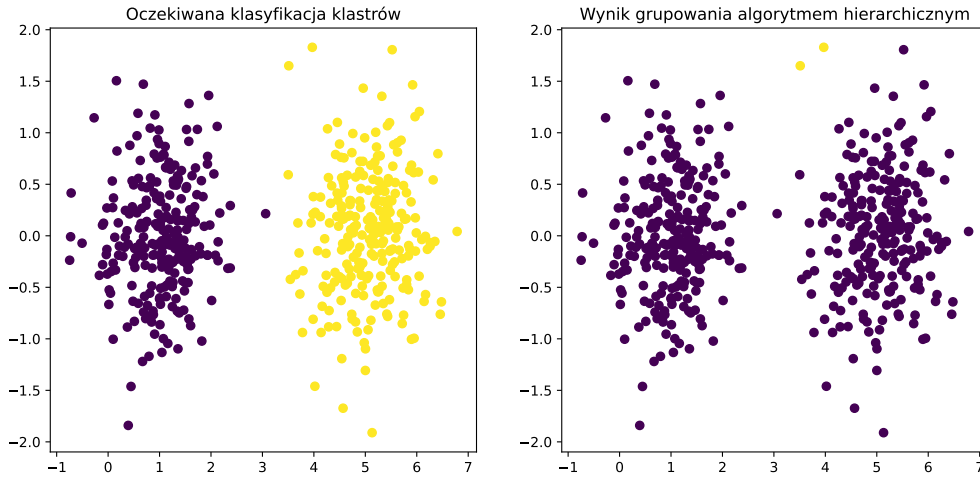


Rysunek 2.3: Ilustracja wpływu danych odstających na działanie algorytmu SINGLELINKAGE.

Algorytm 2 Funkcja aktualizacji odległości między klastrami dla łączności SINGLELINKAGE

Require: $d_{a,x}, d_{b,x}$: wartości odległości między klastrem x a łączonymi w jeden klastrami a i b

- 1: **procedure** UPDATEFORMULA($d_{a,x}, d_{b,x}$)
 - 2: **return** $\min(d_{a,x}, d_{b,x})$
 - 3: **end procedure**
-



Rysunek 2.4: Ilustracja działania algorytmu SINGLELINKAGE, gdy klastry są zbyt zbliżone.

Algorytm działa dobrze, gdy klastry są wyraźnie oddzielone od siebie. Jeżeli będą zbyt blisko siebie, możliwe jest, że zostaną połączone, jak pokazuje przykład na rys. 2.4.

Dla SINGLELINKAGE wykorzystać można szybsze, dedykowane algorytmy, na przykład algorytm minimalnego drzewa rozpinającego (MST_LINKAGE()) o złożoności $O(n^2)$ [18].

2.1.2 Complete Linkage

Algorytm COMPLETELINKAGE jest odwrotnością SINGLELINKAGE: odległość między klastrami jest równa maksymalnej odległości między dwoma punktami należącymi do tych klastrów. W momencie złączenia dwóch klastrów w jeden, jego odległość od pozostałych klastrów określa się więc jako:

$$d(A, B) = \max(d(A_1, B), d(A_2, B)). \quad (2.4)$$

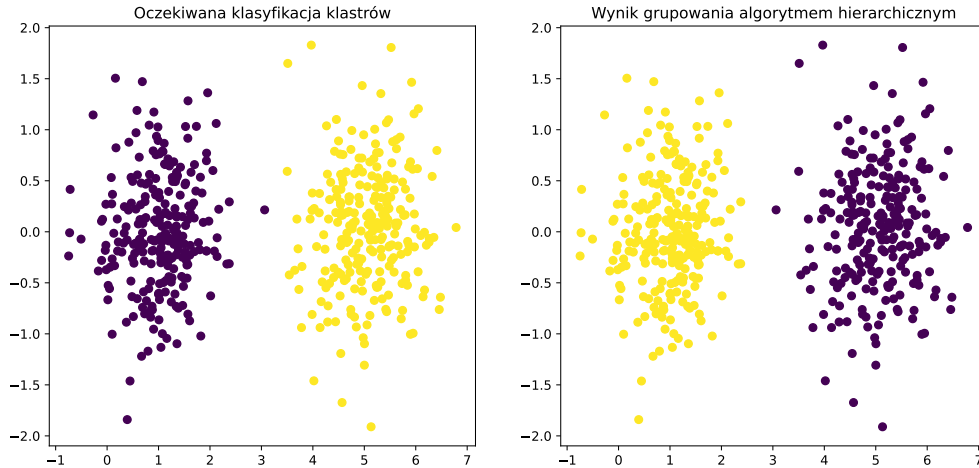
Wzór również można przedstawić jako pseudokod 3, uzupełniający algorytm 1.

Algorytm dobrze działa dla kulistych, gęstych klastrów. Pokazuje to rys. 2.5, gdzie algorytm COMPLETELINKAGE wywołano dla tych samych danych, co SINGLELINKAGE na rys. 2.4. W przeciwieństwie do niego, COMPLETELINKAGE odnajduje oczekiwane klastry

Algorytm 3 Funkcja aktualizacji odległości między klastrami dla łączności COMPLETE-LINKAGE

Require: $d_{a,x}, d_{b,x}$: wartości odległości między klastrem x a łączonymi w jeden klastrami a i b

- 1: **procedure** UPDATEFORMULA($d_{a,x}, d_{b,x}$)
 - 2: **return** $\max(d_{a,x}, d_{b,x})$
 - 3: **end procedure**
-



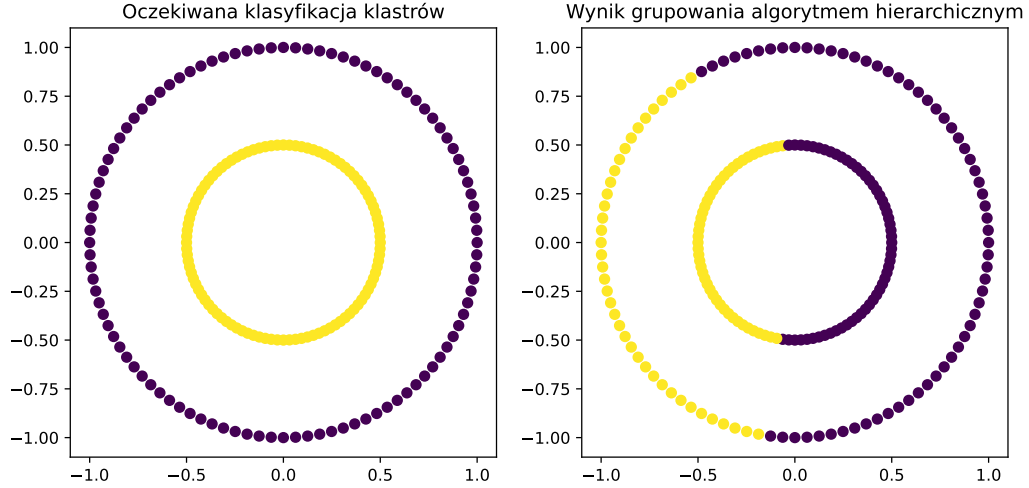
Rysunek 2.5: Ilustracja działania algorytmu COMPLETELINKAGE, dla zbliżonych, gęstych klastrów.

niemal idealnie.

W porównaniu z SINGLELINKAGE, algorytm COMPLETELINKAGE zwraca jednak gorsze wyniki dla danych o innych kształtach [28, 5]. Ilustruje to przykład na rys. 2.6. W przykładzie oczekiwane jest, że każdy z dwóch wyraźnie oddzielonych od siebie okręgów zaklasyfikowany zostanie jako osobny klaster. Algorytm COMPLETELINKAGE nie wyznacza tych klastrów – granica między klastrami wydaje się być wyznaczona bez względu na faktyczny kształt danych.

2.2 Algorytm rozmytych c średnich

Algorytm rozmytych c średnich (ang. *fuzzy c-means*, FCM) należy do rodziny algorytmów opartych na minimalizacji funkcji kryterialnej. Algorytm wyznacza klastry, dla których przynależność każdego punktu danych jest liczbą z przedziału $[0, 1]$. Wynikiem klasteryzacji FCM jest macierz środków klastrów $\mathbf{V}$ oraz macierz przynależności $\boldsymbol{\mu}$. Złożoność czasowa ze względu na liczbę danych tego algorytmu wynosi $O(n)$ [9].



Rysunek 2.6: Ilustracja działania algorytmu COMPLETELINKAGE, dla klastrów w kształcie dwóch okręgów.

Funkcja kryterialna algorytmu fuzzy c-means ma postać:

$$J(\mathbf{V}, \boldsymbol{\mu}) = \sum_{i=1}^N \sum_{c=1}^C \mu_{ic}^m \|\mathbf{x}_i - \mathbf{v}_c\|^m, \quad (2.5)$$

gdzie $\mathbf{x}_i$ – współrzędne i -tego punktu danych, $\mathbf{v}_c$ – współrzędne środka c -tego klastra danych, $\mu_{ic} \in [0, 1]$ – wartość przynależności i -tego punktu do c -tego klastra. Parametr $m \in (1, +\infty]$ jest współczynnikiem rozmycia algorytmu, najczęściej wykorzystuje się wartość 2 [23, 2].

Dla każdego punktu, suma jego wartości przynależności do wszystkich klastrów wynosi 1:

$$\forall i \in [1, N] \sum_{c=1}^C \mu_{ic} = 1 \quad (2.6)$$

Pseudokod algorytmu rozmytych c -średnich przedstawia alg. 4.

W każdej iteracji algorytmu obliczane są najpierw nowe współrzędne środków klastrów:

$$\mathbf{v}_c = \frac{\sum_{i=1}^N \mu_{ic}^m \mathbf{x}_i}{\sum_{i=1}^N \mu_{ic}^m}, \quad (2.7)$$

a następnie wartości elementów macierzy przynależności:

$$\mu_{ic} = \frac{1}{\sum_{j=1}^C \left(\frac{\|\mathbf{x}_i - \mathbf{v}_c\|}{\|\mathbf{x}_i - \mathbf{v}_j\|} \right)^{\frac{2}{m-1}}}. \quad (2.8)$$

Algorytm 4 Algorytm rozmytych c średnich

Require: $\mathbb{X}$: dane do klasteryzacji**Require:** C : liczba klastrów docelowych**Require:** m : współczynnik rozmycia**Require:** I : maksymalna liczba iteracji**Require:** ε : tolerancja

```

1: procedure FUZZYCMETHODS( $\mathbb{X}, C, m, I, \varepsilon$ )
2:    $X \leftarrow$  liczba próbek
3:    $A \leftarrow$  liczba atrybutów  $\triangleright X$  i  $A$ : wymiary  $\mathbb{X}$ 
4:    $\mathbf{v} \leftarrow$  losowe wartości początkowe środków klastrów
5:    $\boldsymbol{\mu} \leftarrow$  losowe wartości początkowe przynależności punktów do klastrów, spełniające
   warunek ze wzoru 2.6
6:    $i \leftarrow 1$ 
7:   koniec  $\leftarrow$  false
8:   while  $i < I \wedge \neg$  koniec do
9:     aktualizacja wartości  $\mathbf{v}$  na podstawie wzoru (2.7)
10:    aktualizacja wartości  $\boldsymbol{\mu}$  na podstawie wzoru (2.8)
11:    if spełniony warunek 2.9 then
12:      koniec  $\leftarrow$  true
13:    end if
14:  end while
15:  return  $\mathbf{v}, \boldsymbol{\mu}$ 
16: end procedure
```

Kroki powtarzane są do momentu spełnienia warunku zatrzymania, na przykład:

$$\|\boldsymbol{\mu}^{(t)} - \boldsymbol{\mu}^{(t-1)}\| < \epsilon, \quad (2.9)$$

gdzie t – numer iteracji, ϵ – parametr tolerancji. Dodatkowym warunkiem zatrzymania może też być z góry określona maksymalna liczba iteracji [2, 21].

Algorytm rozmytych c -średnich stanowi podstawę dla wielu innych algorytmów. Należą do nich między innymi algorytm *possibilistic c-means* (PCM) [13], w którym warunek (2.6) nie musi być spełniony, oraz *fuzzy c-ordered-means* (FCOM) [15], gdzie dane sortowane są według odległości od środków klastrów.

Rozdział 3

Obliczenia ziarniste

Obliczenia ziarniste (granularne) pozwalają na analizę danych w bardziej zbliżony do ludzkiego sposób zrozumienia.

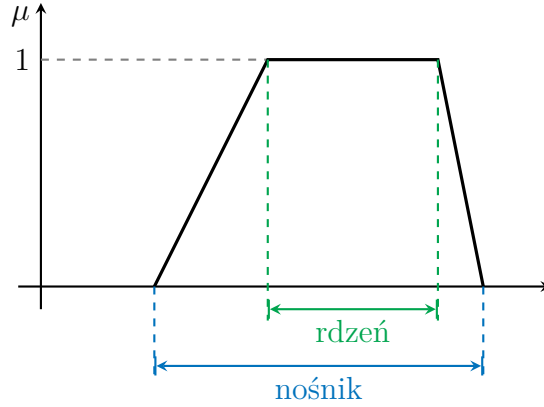
Przykładem ilustrującym ideę obliczeń granularnych w jednym wymiarze może być subiektywna ocena temperatur jako „zimne” lub „ciepłe”. Temperaturę 10°C trudno jednoznacznie zaklasyfikować do jednej z tych dwóch kategorii, dla niektórych osób będzie ona „zimna”, dla niektórych „ciepła”. Temperatura 20°C dużo bardziej pasuje do kategorii „ciepła” niż „zimna”, jednak w mniejszym stopniu, niż 25°C . Choć w uzyskaniu bardziej szczegółowych informacji może pomóc dodanie większej liczby kategorii, na przykład „umiarkowana” i „gorąca”, to nie odzwierciedli ono subiektywnej oceny poszczególnych wartości.

3.1 Ziarna informacyjne

Jednym z podstawowych pojęć związanych z obliczeniami ziarnistymi są ziarna informacyjne. Są one encjami o pewnej semantyce, kolekcjami obiektów o określonym w pewien sposób podobieństwie [20, 26]. Semantyką taką może być na przykład „ciepła” lub „zimna” temperatura. W zależności od znaczenia i zastosowania ziaren, mogą one być reprezentowane na różne sposoby: jako zbiory klasyczne, przedziały liczb, zbiory rozmyte, zbiory cieniowane, zbiory przybliżone, czy funkcje prawdopodobieństwa [20]. Ziarna informacyjne utworzyć można poprzez grupowanie danych. W przypadku ziaren reprezentowanych przez zbiory rozmyte, typowym algorytmem grupowania jest algorytm rozmytych c -średnich, opisanym w sekcji 2.2.

3.2 Zbiory rozmyte

Zbiory klasyczne są pojęciem pierwotnym: obiekt x (liczba, punkt, dana itp.) należy do zbioru $\mathbb{A}$ ($x \in \mathbb{A}$) lub do niego nie należy ($x \notin \mathbb{A}$).



Rysunek 3.1: Wykres trapezowego zbioru rozmytego, ilustrujący jego rdzeń i nośnik.

Tabela 3.1: Warunki t-norm i s-norm

	t-norma	s-norma
warunki graniczne	$x \odot 1 = x, x \odot 0 = 0$	$x \oplus 1 = 1, x \oplus 0 = x$
przemienność	$x \odot y = y \odot x$	$x \oplus y = y \oplus x$
monotoniczność	jeżeli $x \leq u$ to $x \odot y \leq u \odot y$, jeżeli $y \leq r$ to $x \odot y \leq x \odot r$	jeżeli $x \leq u$ to $x \oplus y \leq u \oplus y$, jeżeli $y \leq r$ to $x \oplus y \leq x \oplus r$
łączność	$x \odot (y \odot z) = (x \odot y) \odot z$	$x \oplus (y \oplus z) = (x \oplus y) \oplus z$

W przypadku zbiorów rozmytych, obiekt x należy do zbioru $\mathbb{A}$ z pewnym stopniem przynależności z przedziału $[0, 1]$, gdzie 0 oznacza całkowity brak przynależności, a 1 – całkowitą przynależność [11]. Zbiór rozmyty $\mathbb{A}$ opisać można poprzez funkcję przynależności $\mu_{\mathbb{A}} : \mathbb{X} \rightarrow [0, 1]$.

Dla zbioru rozmytego można wyróżnić jego rdzeń oraz nośnik:

Definicja 1. [11] *Rdzeń zbioru rozmytego $\mathbb{A}$ to taki jego podzbiór $\mathbb{A}_C$, dla którego $\mu_{\mathbb{A}_C} = 1$.*

Definicja 2. [11] *Nośnik zbioru rozmytego $\mathbb{A}$ to taki jego podzbiór $\mathbb{A}_S$, dla którego $\mu_{\mathbb{A}_S} > 0$.*

Graficzną interpretację tych charakterystyk przedstawiono na rys. 3.1. Zbiór rozmyty może nie posiadać rdzenia ($\forall_{x \in \mathbb{A}} \mu(x)_{\mathbb{A}} < 1$), może mieć nieskończony nośnik ($\forall_{x \in \mathbb{A}} \mu(x)_{\mathbb{A}} > 0$).

3.2.1 Operacje na zbiorach rozmytych

Podstawowymi operacjami na zbiorach klasycznych są iloczyn i suma. Ich odpowiednikami dla zbiorów rozmytych są odpowiednio t-norma ($\odot$) i s-norma ($\oplus$), które spełniają warunki określone w tabeli 3.1.

Istnieje wiele t-norm i s-norm spełniających te warunki. W ramach tej pracy, najistotniejsze są:

- t-norma i s-norma algebraiczna:

$$x \odot y = xy \quad (3.1)$$

$$x \oplus y = x + y - xy \quad (3.2)$$

- t-norma i s-norma Łukasiewicza:

$$x \odot y = \max(x + y - 1, 0) \quad (3.3)$$

$$x \oplus y = \min(x + y, 1) \quad (3.4)$$

Inną typową operacją na zbiorach rozmytych jest negacja. Musi ona spełniać warunki $\bar{0} = 1, \bar{1} = 0, x \leq y \Rightarrow \bar{x} \geq \bar{y}$ [4]. Najczęściej wykorzystywaną negacją jest negacja standardowa:

$$\bar{x} = 1 - x \quad (3.5)$$

Jest to negacja silna, czyli spełnia ona również warunek $\bar{\bar{x}} = x$.

3.3 Rozszerzalne liczby rozmyte

Rozszerzalne liczby rozmyte (ang. *extensional fuzzy numbers*) są uogólnieniem liczb rzeczywistych. Są to zbiory rozmyte, które modelują semantykę taką, jak „około 5”, „prawie 10”, „ponad 20”, czyli odpowiadają rzeczywistym liczbom ostrym z pewną tolerancją [25, 4]. Definiowana jest dla nich rozmyta relacja *-podobieństwa S .

Definicja 3. [25] *Rozmyta relacja *-podobieństwa $S(x, y)$, gdzie $*$ jest lewostronnie ciągłą t-normą, spełnia warunki:*

$$S(x, x) = 1 \quad (3.6)$$

$$S(x, y) = S(y, x) \quad (3.7)$$

$$S(x, y) * S(y, z) \leq S(x, z) \quad (3.8)$$

Rozszerzalne liczby rozmyte to liczby ostre rozszerzone o relację *-podobieństwa. Dla dowolnego $y \in \mathbb{R}$ liczbę rozmytą można przedstawić jako [25]:

$$x_S(y) = S(x, y) \quad (3.9)$$

Relacja S zawiera zazwyczaj parametr p , który określa stopień rozmycia liczby. Im większa wartość tego parametru, tym większe rozmycie.

3.3.1 Kształty liczb rozmytych

Kształt rozszerzalnej liczby rozmytej określony jest przez relację $*$ -podobieństwa. Może on być dowolny, o ile spełnia warunki określone w definicji 3. W pracy wykorzystywane są trzy kształty, opisane w [24]:

Liczby trójkątne

Dla liczb trójkątnych relacja $*$ -podobieństwa S_t definiowana jest jako:

$$S_t(x, y) = \max \left\{ 1 - \frac{|x - y|}{p}, 0 \right\}, \quad (3.10)$$

gdzie $p > 0$, a $*$ to t-norma Łukasiewicza.

Liczby wykładnicze

Dla liczb wykładniczych relacja $*$ -podobieństwa S_e definiowana jest jako:

$$S_e(x, y) = \exp \left(-\frac{|x - y|}{p} \right), \quad (3.11)$$

gdzie $p > 0$, a $*$ to t-norma iloczynowa.

Liczby gaussowskie

Dla liczb gaussowskich relacja $*$ -podobieństwa S_g definiowana jest jako:

$$S_g(x, y) = \exp \left(-\frac{(x - y)^2}{2p^2} \right), \quad (3.12)$$

gdzie $p > 0$, a $*$ to t-norma iloczynowa.

3.3.2 Operacje arytmetyczne

Operacje arytmetyczne na rozszerzalnych liczbach rozmytych są proste obliczeniowo i nie zwiększają rozmycia. Wynikiem operacji arytmetycznej na rozszerzalnej liczbie lub liczbach rozmytych jest rozszerzalna liczba rozmyta.

Operacje arytmetyczne jednoargumentowe można zapisać w ogólny sposób jako:

$$\mathbf{op}(x_S) = \mathbf{op}(x)_S, \quad (3.13)$$

na przykład:

$$(x_S)^2 = (x^2)_S. \quad (3.14)$$

Operacje arytmetyczne dwuargumentowe można zapisać jako:

$$x_{S_x} \mathbf{op} y_{S_y} = (x \mathbf{op} y)_{cl(S_x \cup S_y)}, \quad (3.15)$$

gdzie $cl(S_x \cup S_y)$ jest domknięciem relacji S_x i S_y . Dla zdefiniowanych w podsekcji 3.3.1 kształtów ($S \in \{S_t, S_e, S_g\}$) przyjąć można, że

$$cl(S(p_x) \cup S(p_y)) = \max\{p_x, p_y\}. \quad (3.16)$$

Wzór na odległość euklidesową (2.1) można rozszerzyć, aby wykorzystywał rozszerzalne liczby rozmyte, w następujący sposób:

$$d_{fe}(\mathbf{g}_i, \mathbf{g}_j) = \left(\sqrt{\sum_{a=0}^A (g_{ia} - g_{ja})^2} \right)_{S(\max\{g_{i0}, g_{i1}, \dots, g_{iA}, g_{j0}, g_{j1}, \dots, g_{jA}\})}, \quad (3.17)$$

Przykładowo, odległość euklidesową między dwoma punktami, będącymi dwuwymiarowymi trójkątnymi liczbami rozmytymi, zapisać można jako:

$$\begin{aligned} d((5_{S_t(2,5)}, -2_{S_t(0,5)}), (8_{S_t(1,25)}, 2_{S_t(4)})) &= \\ &= \sqrt{(5_{S_t(2,5)} - 8_{S_t(1,25)})^2 + (-2_{S_t(0,5)} - 2_{S_t(4)})^2} = 5_{S_t(4)}. \end{aligned} \quad (3.18)$$

3.3.3 Operacje logiczne

Operacje porównania na rozszerzalnych liczbach rozmytych oparte są na operatorze równości rozmytej [24]:

Definicja 4. Operator równości rozmytej $=_S: \mathbb{E} \times \mathbb{E} \rightarrow [0, 1]$ zdefiniowany jest przy pomocy rozmytej relacji $*$ -podobieństwa S :

$$(x_{S(p_x)} =_S y_{S(p_y)}) = S_{\max\{p_x, p_y\}}(x, y) \quad (3.19)$$

Przykładowo, dla liczb z relacją trójkątną:

$$(x_{S_t(p_x)} =_S y_{S_t(p_y)}) = \max \left\{ 1 - \frac{|x - y|}{\max\{p_x, p_y\}}, 0 \right\} \quad (3.20)$$

Operatory nierówności wyznaczane są w następujący sposób:

$$(x_{S(p_x)} <_S y_{S(p_y)}) = \begin{cases} 1 - (x_{S(p_x)} =_S y_{S(p_y)}) & : x \leq y \\ 0 & : x > y \end{cases} \quad (3.21)$$

$$(x_{S(p_x)} \leq_S y_{S(p_y)}) = \begin{cases} 1 & : x \leq y \\ (x_{S(p_x)} =_S y_{S(p_y)}) & : x > y \end{cases} \quad (3.22)$$

$$(x_{S(p_x)} >_S y_{S(p_y)}) = \begin{cases} 1 - (x_{S(p_x)} =_S y_{S(p_y)}) & : x \geq y \\ 0 & : x < y \end{cases} \quad (3.23)$$

$$(x_{S(p_x)} \geq_S y_{S(p_y)}) = \begin{cases} 1 & : x \geq y \\ (x_{S(p_x)} =_S y_{S(p_y)}) & : x < y \end{cases} \quad (3.24)$$

3.4 Rozmyty algorytm aglomeracyjnego grupowania hierarchicznego

Problemem algorytmów grupowania hierarchicznego jest ich duża złożoność czasowa – w przypadku algorytmu ogólnego aż do $O(X^3)$, gdzie X to liczba danych. Zmniejszenie liczby danych poprzez zastosowanie wstępnej granulacji algorytmem rozmytych c -średnich może znacznie przyspieszyć jego działanie, jako że algorytm ten ma stałą złożoność czasową w zależności od liczby danych ($O(X)$).

Zaproponowano rozmyty algorytm aglomeracyjnego grupowania hierarchicznego: GRAG-GLO. Składa się on z trzech głównych kroków:

1. granulacja oryginalnych danych przy pomocy algorytmu rozmytych c -średnich (roz. 3.4.1),
2. przekształcenie otrzymanych wyników w granule o potrzebnych parametrach (roz. 3.4.2),
3. aglomeracyjne grupowanie hierarchiczne granul z uwzględnieniem ich rozmycia: FUZZYAGGLO (roz. 3.4.3).

Złożoność czasowa tego algorytmu może zostać określona jako $O(X + XG + G^3)$, przy czym $G \ll X$ odpowiada liczbie granul.

Implementacja algorytmu, skrypty pomocnicze pozwalające na analizę jego działania, badane dane syntetyczne oraz wyniki uzyskane na ich podstawie zamieszczono w repozytorium pod adresem <https://github.com/ZofiaLor/magister-granule>.

3.4.1 Granulacja danych oryginalnych

W pierwszym kroku oryginalne dane są granulowane przez algorytm rozmytych c -średnich. Dane mają postać macierzy liczb rzeczywistych o wymiarach $X \times A$, gdzie X – liczba danych, A – liczba atrybutów. Wartość parametru rozmycia m algorytmu określona jest na stałe jako 2. Liczba docelowych klastrow C jest jednym z parametrów badawczych i jest znacznie mniejsza od X .

Wyjściem tego kroku algorytmu będą struktury wyznaczone przez algorytm FCM, w szczególności wektor środków klastrow $\mathbf{v}$ oraz macierz przynależności danych do klastrow μ .

3.4.2 Przekształcenie granul

Granule wykorzystywane do grupowania hierarchicznego mają postać wielowymiarowych rozszerzalnych liczb rozmytych. Dla każdego wymiaru musi zostać określona wartość ostra liczby x oraz jej rozmycie p .

Wartości ostre są tożsame ze środkami klastrow będących wynikiem algorytmu rozmytych c -średnich, czyli z wektorem $\mathbf{v}$ (wzór 2.7).

Wartościom rozmycia nie odpowiada bezpośrednio żadna ze struktur zwracanych przez FCM. Dla każdego klastra c i wymiaru (atrybutu) a , rozmycie oblicza się jako następujące odchylenie standardowe:

$$p_{ca} = \sqrt{\frac{\sum_{i=1}^X \mu_{ci}^m (x_{ia} - v_{ca})^2}{\sum_{i=1}^X \mu_{ci}^m}}. \quad (3.25)$$

Wyjściem tego kroku jest zbiór granul $\mathbb{G}$ – obiektów posiadających listę a -elementową liczb rozmytych, które z kolei zawierają informacje o wartościach x i p . Granule nie mają na stałe określonego kształtu rozszerzalnych liczb rozmytych, które składają się na ich wymiary. Kształt jest więc parametrem dla operacji logicznych na nich wykonywanych. Złożoność czasowa tego kroku wynosi $O(XG)$.

Liczby rozmyte stanowią rozszerzenie liczb ostrych, więc każda arytmetyczna i logiczna operacja rozmyta zwraca poprawny wynik również wtedy, gdy ich parametrami są liczby, dla których $p = 0$. Implementacja operacji $=_s$ w przypadku, gdy obydwie liczby są ostre, wykonuje zwykłą, ostrą operację logiczną $=$, w przeciwnym razie wystąpiłoby dzielenie przez 0. Operacja zwraca 1, jeżeli liczby są równe, 0, jeżeli nie są zamiast wartości logicznych odpowiednio prawda lub fałsz.

3.4.3 Rozmyte grupowanie hierarchiczne

Zbiór granul $\mathbb{G}$ oraz ich kształt są parametrami wejściowymi algorytmu rozmytego aglomeracyjnego grupowania hierarchicznego FUZZYAGGLO. Na początku wyznaczane

Algorytm 5 Minimum rozmyte

Require: a, b : porównywane rozszerzalne liczby rozmyte**Require:** r : typ relacji (trójkątna, wykładnicza, gaussowska)**Require:** ξ : wartość progowa

```

1: procedure FUZZYMINIMUM( $a, b, r, \xi$ )
2:   if ( $a <_{s_r} b$ )  $> \xi$  then
3:     return  $a$ 
4:   else
5:     return  $b$ 
6:   end if
7: end procedure

```

są rozmyte odległości euklidesowe (wzór (3.17)) między każdą parą granul, przy pomocy operacji arytmetycznych określonych w sekcji 3.3 (alg. 8). Wynikiem obliczeń jest symetryczna macierz odległości $\mathbf{D}$ o wymiarach $X \times X$, gdzie $d_{ij} = d_{ji} = d(\mathbf{x}_i, \mathbf{x}_j)$. Odległość danego elementu od samego siebie (d_{ii}) nie jest wykorzystywana w algorytmie, jej wartość nie jest więc istotna.

Grupowanie hierarchiczne FUZZYAGGLO ma postać zmodyfikowanego algorytmu naiwnego PRIMITIVECLUSTERING, o złożoności czasowej $O(G^3)$. Zmieniony musi zostać sposób wyznaczania kolejnych minimalnych odległości w macierzy $\mathbf{D}$ oraz, zależne od łączności, aktualizacje odległości do nowo scalonego klastra.

Minimum i maksimum wyznacza się przy pomocy operacji porównania. W przypadku rozszerzalnych liczb rozmytych, wynikami takich operacji są liczby z przedziału $[0, 1]$. W celu określenia, która z liczb rozmytych jest większa lub mniejsza, wprowadzony zostaje parametr progowy ξ , będący parametrem badawczym. Algorytmy rozmytego minimum i maksimum przedstawiają odpowiednio pseudokody 5 i 6. Na podstawie alg. 5 można też utworzyć rozmytą funkcję arg min (alg. 7).

Dla łączności SINGLELINKAGE, algorytm FUZZYAGGLO ma postać opisaną pseudokodem 9. Ten sam algorytm dla łączności COMPLETELINKAGE jest niemal identyczny: należy jedynie podmienić wywołanie algorytmu rozmytego minimum (alg. 5) w linii 10 na algorytm rozmytego maksimum (alg. 6).

3.5 Badania

Działanie algorytmu GRAGGLO musi zostać zbadane pod kątem czasu wykonania oraz jakości grupowania, również w porównaniu z jego ostrym odpowiednikiem.

3.5.1 Zbiory danych

Pierwszym zbiorem danych są dane syntetyczne, które znajdują się w plikach `.data` zlokalizowanych w folderze `dane`, pogrupowanych w podfoldery według kształtów. Dla

Algorytm 6 Maksimum rozmyte**Require:** a, b : porównywane rozszerzalne liczby rozmyte**Require:** r : typ relacji (trójkątna, wykładnicza, gaussowska)**Require:** ξ : wartość progowa

```

1: procedure FUZZYMAXIMUM( $a, b, r, \xi$ )
2:   if ( $a >_{s_r} b$ )  $> \xi$  then
3:     return  $a$ 
4:   else
5:     return  $b$ 
6:   end if
7: end procedure

```

Tabela 3.2: Charakterystyka zbiorów danych dołączonych do projektu.

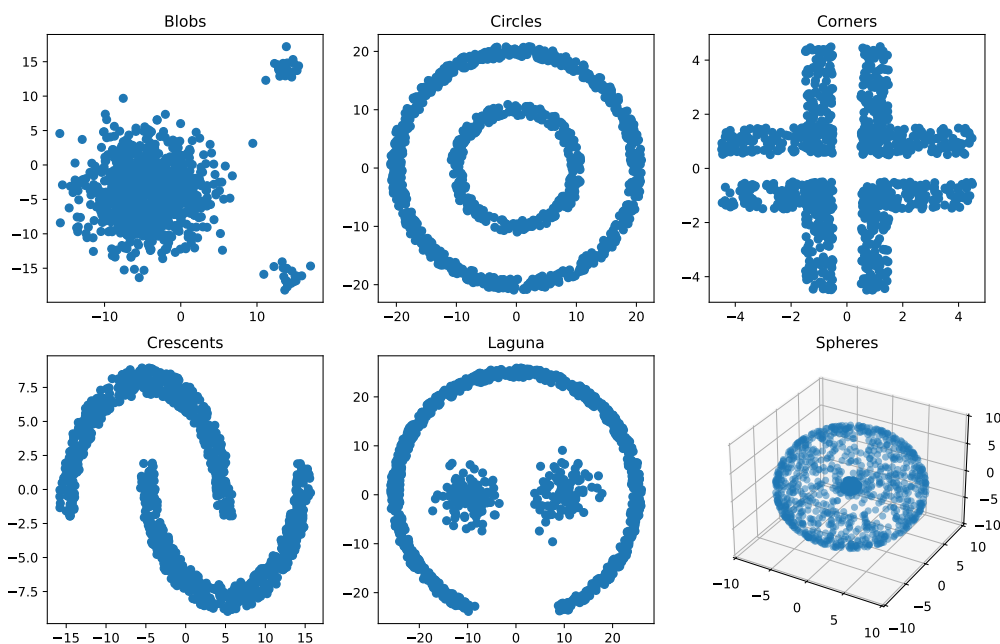
	‘blobs’	‘circles’	‘corners’	‘crescents’	‘laguna’	‘spheres’
liczba klastrów	3	2	4	2	3	2
liczba wymiarów	2	2	2	2	2	3

każdego kształtu zbiory mają liczebność 1000, 2000, 5000, 10 000, 20 000, 30 000, 40 000 oraz 50 000 wierszy.

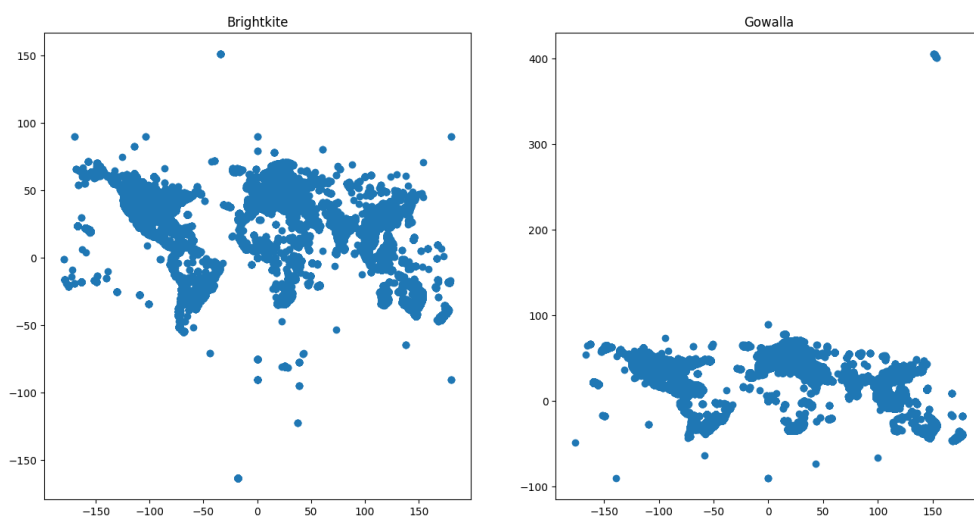
Dane mają postać współrzędnych punktów w dwóch lub trzech wymiarach – każdy wiersz w pliku reprezentuje współrzędne jednego punktu, współrzędne oddzielone są znakami tabulacji. Taka liczba wymiarów pozwala na wizualizację całości danych w postaci wykresu. W zależności od kształtu, dla danego zbioru można na podstawie wykresu wyróżnić 2, 3 lub 4 wyraźnie odseparowane klastry. Wykresy danych przedstawiono na rys. 3.2, a w tabeli 3.2 – dodatkowo ich charakterystykę (liczbę wymiarów i klastrów).

Dane te nie posiadają etykiet. W sposób opisany w sekcji na stronie 25 każdemu punktowi przypisano etykietę w postaci liczby całkowitej $[0, C - 1]$ i zapisano w folderze `dane_labelled` w analogicznym formacie.

Wykorzystano również zbiory danych ‘brightkite’ [6] (4 491 143 wierszy) oraz ‘gowalla’ [7] (6 442 890 wierszy). Dane przedstawiają informacje o logowaniach się użytkowników platform społecznościowych. Dane przycięto, aby zawierały jedynie kolumny „latitude” oraz „longitude” – dwuwymiarowe współrzędne lokalizacji logowań. Dane nie są etykietowane i nie mają przypisanych klas. Analiza wykresów (rys. 3.3) pozwala zauważyć kształty kontynentów – można uznać je za docelowe klastry. Dane zawierają błędne współrzędne. W danych ‘brightkite’, kolumna „latitude” (szerokość geograficzna) zawiera wartości z przedziału $[-163,193\,333, 151,198]$, podczas gdy dopuszczalnym przedziałem jest $[-90, 90]$. Dla danych ‘gowalla’ przedział ten wynosi $[-90, 405,658\,498]$. Błędne współrzędne pozostawiono w zbiorze danych.



Rysunek 3.2: Wykresy danych o różnych kształtach z folderu **dane** o rozmiarze 1000 punktów.



Rysunek 3.3: Wykresy danych ‘brightkite’i ‘gowalla’.

Algorytm 7 Rozmyta funkcja arg min

Require: $\mathbf{X}$: macierz liczb rozmytych, $\mathbf{p}$: wektor indeksów, dla których znajdowane jest minimum, r : typ relacji (trójkątna, wykładnicza, gaussowska), ξ : parametr porównania

```

1: procedure FUZZYARGMIN( $\mathbf{X}, r, \xi$ )
2:   result  $\leftarrow \infty_{S(0)}$  ▷ liczba rozmyta, dla której  $x = \infty, p = 0$ 
3:    $(w, k) \leftarrow (0, 0)$  ▷ Współrzędne (wiersz, kolumna) wartości minimalnej w
   przeszukiwanej części macierzy  $\mathbf{X}$ 
4:   for  $i = 1$  to  $|\mathbf{p}| - 1$  do
5:     for  $j = i + 1$  to  $|\mathbf{p}|$  do
6:        $p_w \leftarrow \mathbf{p}[i]$ 
7:        $p_k \leftarrow \mathbf{p}[j]$ 
8:       if  $(\mathbf{X}[p_w][p_k] <_{S_r} \text{result}) > \xi$  then
9:         result  $\leftarrow \mathbf{X}[p_w][p_k]$ 
10:         $(w, k) \leftarrow (p_w, p_k)$ 
11:      end if
12:    end for
13:  end for
14:  return  $(w, k)$  ▷ Współrzędne (wiersz, kolumna) wartości minimalnej w
  przeszukiwanej części macierzy  $\mathbf{X}$ 
15: end procedure

```

3.5.2 Metodyka badań

Algorytm GRAGGLO (3.4) zbadano pod kątem szybkości wykonania oraz jakości grupowania poprzez porównanie jego efektów z efektami ostrego algorytmu aglomeracyjnego grupowania hierarchicznego SINGLELINKAGE i COMPLETELINKAGE.

Czas grupowania

Aby znaleźć charakterystykę zależności czasu wykonania algorytmów od liczby danych, przygotowano dane o rozmiarach 500-10 000 z krokiem 500. W tym celu brane były początkowe podzbiory ze zbiorów danych o rozmiarze 10 000. Jako że nieistotne były wyniki grupowania, nie było konieczne losowe próbkowanie.

Przyjęto też stałą maksymalną liczbę 80 iteracji algorytmu rozmytych c -średnich. Sekcja 3.5.3 pokazuje, że jest to wystarczająca wartość. Złożoność algorytmu jest liniowa w zależności od liczby iteracji [9]. Algorytm wykonano też dla różnych liczości granul: 50, 100 oraz 200.

Czas pojedynczego wykonania algorytmu ostrego zmierzono poprzez różnicę czasu rozpoczęcia i zakończenia grupowania hierarchicznego. Wykorzystany został algorytm `AgglomerativeClustering` biblioteki `scikit-learn` (łączność SINGLELINKAGE) oraz własna implementacja ostrego algorytmu grupowania, analogiczna do wersji rozmytej. Pozwoliło to na lepszy pogląd na wpływ granulacji na szybkość, jako że sama logika grupowania we własnej implementacji pozostała ta sama, co w algorytmie rozmytym. Algorytm biblio-

Algorytm 8 Rozmyte odległości euklidesowe**Require:** $\mathbf{X}$: macierz liczb rozmytych

```

1: procedure FUZZYDISTANCE( $\mathbf{X}$ )
2:   result  $\leftarrow \infty_{S(0)}$  ▷ liczba rozmyta, dla której  $x = \infty, p = 0$ 
3:    $X \leftarrow |\mathbb{X}|$  ▷ liczba danych
4:    $\mathbf{D} \leftarrow$  macierz zerowa  $X \times X$ 
5:   for  $i = 1$  to  $X - 1$  do
6:     for  $j = i + 1$  to  $X$  do
7:        $d_{ij} \leftarrow 0_0$  ▷ liczba rozmyta, dla której  $x = 0, p = 0$ 
8:       for  $k$  in  $|\mathbf{X}[i]|$  do ▷ dla każdego wymiaru liczby rozmytej
9:          $d_{ij} \leftarrow d_{ij} + (\mathbf{X}[i][k] - \mathbf{X}[j][k])^2$  ▷ Operacje arytmetyczne na liczbach
10:        rozmytych (wzór (3.15))
11:        $d_{ij} \leftarrow \sqrt{d_{ij}}$  ▷ Operacja arytmetyczna na liczbie rozmytej (wzór (3.13))
12:        $\mathbf{D}[i][j] \leftarrow \mathbf{D}[j][i] \leftarrow d_{ij}$  ▷ macierz jest symetryczna względem przekątnej
13:     end for
14:   end for
15:   return  $\mathbf{D}$  ▷ macierz rozmytych odległości między parami liczb rozmytych
16: end procedure

```

teczny ma z kolei najniższą złożoność czasową dla łączności SINGLELINKAGE (wykorzystuje algorytm minimalnego drzewa rozpinającego), można więc porównać czas wykonania GRAGGLO z najlepszym możliwym czasem grupowania tych samych danych algorytmem ostrym. Do tego porównania wykorzystano oryginalne dane wejściowe, we wszystkich rozmiarach (1000-50 000). W czasie pojedynczego wykonania algorytmu rozmytego zawarto czas wykonania algorytmu rozmytych c -średnich, utworzenia na jego podstawie listy granul oraz rozmytego grupowania hierarchicznego.

Poszczególne pomiary czasu działania algorytmu dla dokładnie takich samych rozmiarów danych mogą różnić się o kilka milisekund. Wynika to między innymi z działania w tle innych procesów komputera, na którym przeprowadzane są badania. Typowym rozwiązaniem tego problemu jest kilkukrotne powtórzenie pomiarów dla tych samych danych i obliczenie średniej arytmetycznej wyników. Podczas badań pomiary powtórzono czterokrotnie, co wystarczyło do wyeliminowania odstających wyników.

Wykonano również pomiary czasu wykonania algorytmu GRAGGLO dla dużych liczebności danych: 50 000, 100 000, 500 000, 1 000 000, 2 000 000, 3 000 000, 4 000 000, 5 000 000 i 6 000 000, dla liczebności granul 50, 100 i 200. Jako że dla tak dużych rozmiarów danych czas ten jest rzędu minut, nie mają znaczenia milisekundowe przekłamanie wyników. Z tego powodu wystarczyło wykonać jeden pomiar dla każdego rozmiaru danych.

Jakość grupowania

Jakość zmierzono poprzez porównanie wyników algorytmu GRAGGLO z oczekiwanymi przynależnościami punktów do klastrów, oznaczonymi etykietami. Jako miary jakości wy-

Algorytm 9 Algorytm rozmytego aglomeracyjnego grupowania hierarchicznego FUZZY-AGGLO (SINGLELINKAGE)

Require: $\mathbb{X}$: dane do klasteryzacji**Require:** C : docelowa liczba klastrów**Require:** r : typ relacji (trójkątna, wykładnicza, gaussowska)**Require:** ξ : wartość progowa

```

1: procedure FUZZYAGGLO( $\mathbb{X}, C, r, \xi$ )
2:    $X \leftarrow |\mathbb{X}|$  ▷ liczba danych
3:    $\mathbf{D} \leftarrow \text{FUZZYDISTANCE}(\mathbb{X})$  ▷ symetryczna macierz  $X \times X$  rozmytych odległości
   euklidesowych między każdą parą danych
4:    $\mathbf{p} \leftarrow [1, 2, \dots, X - 1, X]$  ▷ wektor indeksów, które będą sprawdzane
5:    $\mathbf{e} \leftarrow [1, 2, \dots, X - 1, X]$  ▷ etykiety
6:   for  $i = 1$  to  $X - C$  do
7:      $w, k \leftarrow \text{FUZZYARGMIN}(\mathbf{D}, \mathbf{p}, r, \xi)$  ▷ znalezienie indeksów dwóch najbliższych
     klastrów
8:      $\mathbf{p} \leftarrow \mathbf{p} \setminus k$  ▷ nieużywane w dalszych iteracjach
9:     for  $p \in \mathbf{p}$  do
10:       $\mathbf{D}[w, p] = \mathbf{D}[p, w] = \text{FUZZYMINIMUM}(\mathbf{D}[w, p], \mathbf{D}[k, p], r, \xi)$  ▷ przypisanie
      wynikające z łączności
11:    end for
12:     $\mathbf{e} \leftarrow \text{SCAL}(\mathbf{e}, w, k)$  ▷ zamiana wszystkich etykiet o wartości  $k$  na wartość  $w$ .
13:  end for
14:   $\mathbf{e} \leftarrow \text{PRZEMIANY}(\mathbf{e})$  ▷ przemianowanie etykiet  $\mathbf{e}$  na kolejne liczby od 0.
15:  return  $\mathbf{e}$ 
16: end procedure

```

korzystano dokładność, trafność oraz precyzję.

Pomiary wykonano dla wszystkich kształtów i rozmiarów danych. W przypadku algorytmu rozmytego, pomiary przeprowadzono również dla różnych łączności (SINGLELINKAGE, COMPLETELINKAGE), liczebności granul (50, 100, 200), typu relacji liczb rozmytych (trójkątne, wykładnicze, gaussowskie) oraz parametru $\xi \in \{0,05, 0,10, 0,15, \dots, 0,95\}$.

Podobnie jak w przypadku pomiarów czasu grupowania, przyjęto stałą maksymalną liczbę 80 iteracji. Sparametryzowanie tej wartości oznaczałoby badanie działania bibliotecznej funkcji FCM, zamiast własnego algorytmu.

Kolejność danych została przed pomiarem losowo zmieszana przy pomocy generatora liczb pseudolosowych o ustalonym ziarnie. Dzięki temu, przy każdym uruchomieniu kolejność ta jest identyczna, wystarczyły więc jednokrotne pomiary wyników działania algorytmu GRAGGLO dla każdej kombinacji parametrów.

Etykietowanie danych W celu przypisania etykiet do danych z folderu `dane`, przygotowano pomocniczy skrypt w pliku `label_assignment.py`. W zależności od tego, czy dane są dwu- czy trójwymiarowe, przypisanie etykiet odbywa się na inny sposób.

Dane dwuwymiarowe Oznaczanie etykiet dla danych dwuwymiarowych wykorzystuje widżet `LassoSelector` biblioteki `matplotlib`. Dla każdego pliku z danymi dwuwymiarowymi, wyświetlono wykres połączony z obiektem klasy `SelectFromCollection` z przykładu [14]. Klasa ta między innymi umożliwia wyznaczenie indeksów i współrzędnych punktów zaznaczonych myszą na wykresie.

Każdemu punktowi domyślnie przypisano etykietę 0. Po zaznaczeniu zbioru punktów na wykresie naciśnięcie klawisza cyfry na klawiaturze powoduje przypisanie tym punktom etykiety o danym numerze i wyświetlenie w konsoli informacji, ilu punktom ją przypisano. Proces można powtarzać dowolną liczbę razy, można go zakończyć poprzez naciśnięcie klawisza ENTER i zamknięcie wykresu.

W celu ostatecznej weryfikacji pojawia się drugi wykres, na którym każdej etykietce odpowiada inny kolor. Po zamknięciu tego wykresu, plik z etykietami jest zapisywany w folderze `dane_labelled`. Plik ma format zgodny z oryginalnym plikiem z danymi (każdy wiersz to oddzielone znakami tabulacji współrzędne jednego punktu), z dodaną dodatkową kolumną z etykietami punktów.

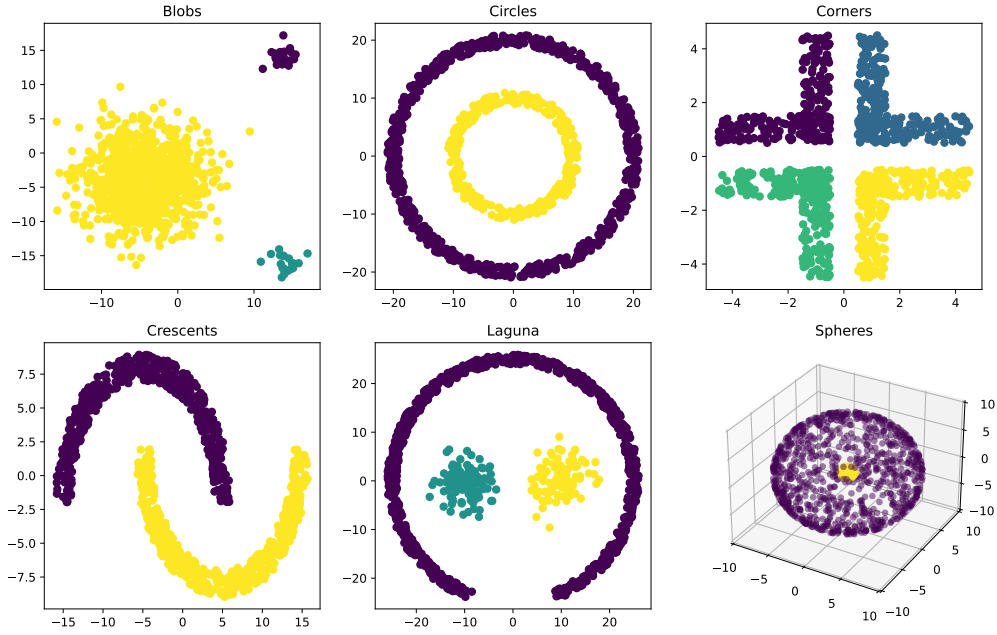
Dane trójwymiarowe W przypadku danych trójwymiarowych skorzystano z faktu, że jedyne dane wykorzystywane w pracy, dla których należy znaleźć etykiety, to dane ‘spheres’. Punkty otrzymane z tych danych tworzą dwie sfery: mniejsza, wewnętrzna sfera jest zamknięta w sześcianie o wymiarach $1 \times 1 \times 1$ jednostek, a punkty drugiej, większej sfery znajdują się poza tym sześcianem. Wszystkim punktom, których wszystkie współrzędne mają wartości mniejsze lub równe 1 przypisano etykietę 1, pozostałym – 0. Dane z etykietami zostały zapisane w folderze `dane_labelled`, w takim samym formacie, jak dane dwuwymiarowe.

Wyniki etykietowania Na wykresie 3.4 przedstawiono wyniki etykietowania danych. Punkty oznaczone tym samym kolorem mają te same etykiety.

W przypadku danych ‘laguna’ o liczebności 20 000 i większej, dwa środkowe klastry przestają być wyraźnie oddzielone od siebie (rys. 3.5). Liczebności tych klastrów po podziale ich narzędziem `LassoSelector` nie są równe, jak w przypadku mniejszych zbiorów o tym samym kształcie – różnice wynoszą nie więcej niż 18 punktów. Przy tak dużych zbiorach danych wpływ pojedynczych punktów jest marginalny, należy jednak uwzględnić tę niepewność w analizie wyników jakości grupowania.

Miara jakości Wybranymi miarami jakości grupowania są dokładność A , trafność R i precyzja P .

W przypadku dwuklasowym (klasy *positive* i *negative*), dokładność (ang. *accuracy*) jest definiowana jako:



Rysunek 3.4: Wykresy danych o różnych kształtach z folderu `dane_labelled` o rozmiarze 1000 punktów, o etykietach oznaczonych kolorami.

$$A = \frac{TP + TN}{TP + FP + TN + FN}, \quad (3.26)$$

gdzie:

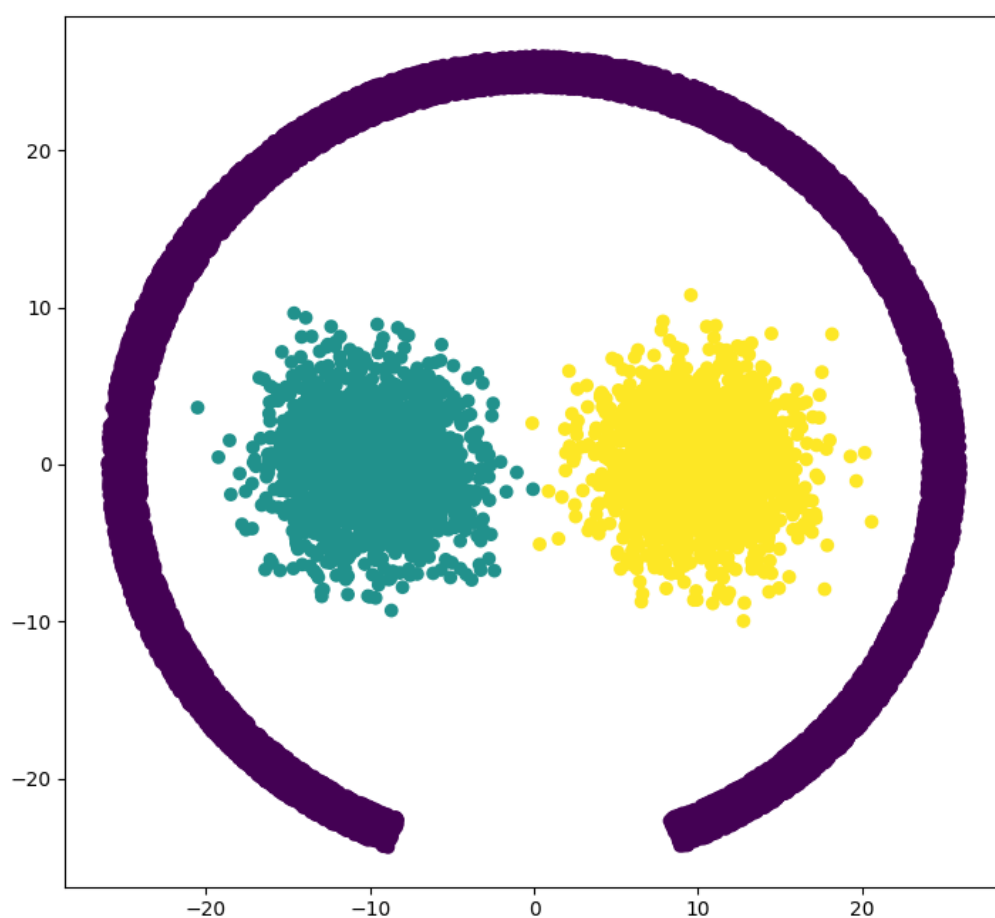
- TP – liczba danych poprawnie zaklasyfikowanych do klasy *positive*
- FP – liczba danych niepoprawnie zaklasyfikowanych do klasy *positive*
- TN – liczba danych poprawnie zaklasyfikowanych do klasy *negative*
- FN – liczba danych niepoprawnie zaklasyfikowanych do klasy *negative*

Dokładność jest łatwo rozszerzalna na więcej klas: sumę liczb danych poprawnie zaklasyfikowanych do każdej klasy należy podzielić przez liczbę wszystkich danych.

Trafność (czułość, ang. *recall*) definiowana jest jako:

$$R = \frac{TP}{TP + FN}, \quad (3.27)$$

czyli stosunek liczby danych poprawnie zaklasyfikowanych do klasy *positive* do wszystkich danych do niej należących.



Rysunek 3.5: Wykres danych ‘laguna’ z folderu `dane_labelled` o rozmiarze 20 000 punktów, o etykietach oznaczonych kolorami.

Precyzję (ang. *precision*) definiuje się jako:

$$P = \frac{TP}{TP + FP}, \quad (3.28)$$

czyli stosunek liczby danych poprawnie zaklasyfikowanych do klasy *positive* do wszystkich danych które zostały do niej zaklasyfikowane.

Trafność i precyzję można rozszerzyć na wiele klas na różne sposoby. W badaniach wybrano metodę, w której najpierw obliczana jest wartość metryk cząstkowych dla każdej klasy z osobna (jedna klasa odpowiada klasie *positive*, wszystkie pozostałe – *negative*), a następnie obliczana jest z nich średnia arytmetyczna. Oznacza to, że każda klasa ma jednakową wagę dla ostatecznego wyniku – jeżeli będą miały różne licznosci, to klasy zawierające małą liczbę danych będą mieć proporcjonalnie większy wpływ na wyniki trafności i precyzji, niż dokładności.

Przynależność punktów do granul Rozmyty algorytm grupowania jako wynik zwraca przynależność do klastrów granul, nie oryginalnych danych. Na potrzeby porównania konieczne jest określenie, do której granuli należy każdy punkt. Przynależność tą wyznaczano na dwa sposoby na podstawie (1) macierzy przynależności do klastra wyznaczonego przez algorytm rozmytych *c*-średnich μ i (2) wartości relacji *-podobieństwa S .

Pierwsze podejście dla każdego punktu i , znajduje granulę, dla której wartość odpowiedniego wiersza w macierzy przynależności μ_i jest maksymalna.

Drugie podejście przedstawia pseudokod 10. Dla każdego punktu i , przynależność do granuli j obliczana na podstawie wartości operacji $=_S$ (wzór 3.19) dla każdego wymiaru punktu i granuli. Operacja ta jest zależna od kształtu granul, dla liczb trójkątnych, wykładniczych i gaussowskich należy wykorzystać odpowiednio wzory 3.10, 3.11 lub 3.12 uwzględniając, że dla liczby ostrej (współrzędnej oryginalnego punktu) $p = 0$.

Wyniki dla każdej współrzędnej łączone są t-normą algebraiczną (wzór 3.1). Wreszcie określa się, że punkt należy do tej granuli, dla której wynik tego złączenia jest maksymalny.

Należy uwzględnić, że rozszerzalne liczby rozmyte trójkątne mają skończony nośnik. Oznacza to, że punkt może nie należeć do żadnej granuli – dla każdej z nich przynależność będzie zerowa. Punkt taki uznawany jest za szum i nie bierze udziału w wyznaczaniu wartości dokładności, trafności i precyzji, jednak odsetek punktów stanowiących szum jest w tym przypadku dodatkową miarą jakości.

Rysunek 3.6 przedstawia przykład takiego przypisania punktów do granul, dla czytelności wykresów wyznaczono jedynie 10 granul. Wykresy dla liczb wykładniczych i gaussowskich są bardzo podobne, ponadto wszystkie punkty są przypisane do granuli. W przypadku liczb trójkątnych, duża część punktów stanowi szum.

Algorytm 10 Wyznaczenie przynależności punktu do granuli na podstawie relacji *-podobieństwa S

Require: $\mathbb{X}$: dane oryginalne

Require: $\mathbb{G}$: granule wyznaczone przez algorytm GRAGGLO

Require: $\mathbf{e}_g$: wektor o długości G etykiet wyznaczonych dla wszystkich granul

Require: r : typ relacji (trójkątna, wykładnicza, gaussowska)

```

1: procedure CALCULATESIMILARITYMEMBERSHIP( $\mathbb{X}, \mathbb{G}, \mathbf{e}, r$ )
2:    $\mathbf{e}_x \leftarrow \emptyset$  ▷ etykiety punktów
3:    $szum \leftarrow 0$  ▷ liczba punktów nieprzynależących do żadnej granuli
4:   for  $x$  in  $X$  do
5:      $e \leftarrow 0$ 
6:      $\mu_{max} \leftarrow 0$ 
7:     for  $i = 1$  to  $G$  do
8:        $\mu \leftarrow (\mathbb{G}[i][1] =_S x[1])$ 
9:       for  $j = 2$  to liczba wymiarów  $x$  do
10:         $\mu \leftarrow \mu \odot (\mathbb{G}[i][j] =_S x[j])$  ▷  $\odot$ : t-norma
11:      end for
12:      if  $\mu > \mu_{max}$  then
13:         $\mu_{max} \leftarrow \mu$ 
14:         $e \leftarrow i$ 
15:      end if
16:    end for
17:    if  $\mu_{max} > 0$  then
18:      dodaj  $i$  do  $\mathbf{e}_x$ 
19:    else ▷ punkt nie przynależy do żadnej granuli – jest szumem
20:      dodaj  $-1$  do  $\mathbf{e}_x$ 
21:       $szum \leftarrow szum + 1$ 
22:    end if
23:  end for
24:  return  $\mathbf{e}_x, szum$ 
25: end procedure

```

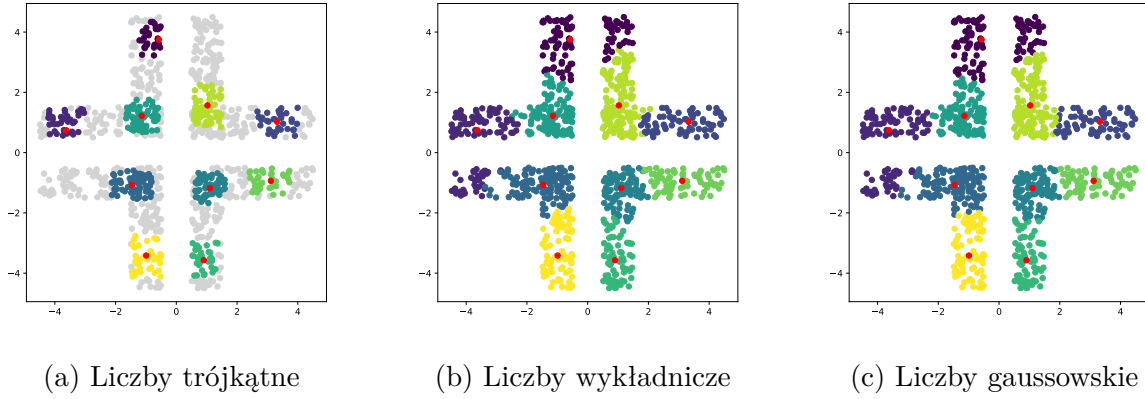
Macierz pomyłek i obliczanie miar jakości Wyznaczono macierz $\mathbf{L}$, w której wartość elementu l_{ij} jest równa liczbie punktów, dla których oczekiwana etykieta ma wartość i i przynależących do klastra j wyznaczonego przez algorytm GRAGGLO:

$$l_{ij} = \sum_{k=0}^N [x_k \in i \wedge x_k \in j]. \quad (3.29)$$

Sposób wyznaczenia tej macierzy przedstawiono również na pseudokodzie 11.

Numeracja klastrów wyznaczonych przez algorytm GRAGGLO może być inna od oczekiwanych etykiet ze względu na różną kolejność ich wyznaczenia. Wykorzystując algorytm przedstawiony na pseudokodzie 12, wyznaczono taką permutację wierszy i kolumn macierzy $\mathbf{L}$, która maksymalizuje wartości na przekątnej.

Z uzyskanej w ten sposób macierzy pomyłek, dokładność wyznaczana jest jako suma



Rysunek 3.6: Przynależność punktów do granul w zależności od kształtu liczb rozmytych. 10 środków granul oznaczono kolorem czerwonym, punkty należące do różnych granul oznaczone są różnymi kolorami. Kolorem szarym oznaczono punkty stanowiące szum.

Algorytm 11 Wyznaczenie macierzy $\mathbf{L}$

Require: μ : macierz $X \times X$ przynależności danych do klastrów

Require: $\mathbf{e}_r$: wektor o długości X etykiet wyznaczonych dla wszystkich punktów przez algorytm ostry

Require: $\mathbf{e}_g$: wektor o długości C etykiet wyznaczonych dla wszystkich granul przez algorytm GRAGGLO

```

1: procedure CALCULATELMATRIX( $\mu, \mathbf{e}_r, \mathbf{e}_g$ )
2:    $\mathbf{L} \leftarrow$  macierz zerowa  $C \times C$ 
3:   for  $i = 1$  to  $X$  do
4:      $\mu \leftarrow \text{ARGMAX}(\mu[i]) \triangleright$  Indeks identyfikujący granulę, dla której przynależność
       punktu  $i$ -tego jest największa
5:     inkrementacja  $\mathbf{L}[\mathbf{e}_r[i]][\mathbf{e}_g[\mu]]$ 
6:   end for
7:   return  $\mathbf{L}$ 
8: end procedure

```

wartości na przekątnej macierzy podzielona przez całkowitą liczbę danych:

$$A = \frac{\sum_{i=0}^C l_{ii}}{N} \quad (3.30)$$

lub

$$A = \frac{\sum_{i=0}^C l_{ii}}{\sum_{i=0}^C \sum_{j=0}^C l_{ij}}, \quad (3.31)$$

gdzie C to liczba klastrów.

Trafność (R) i precyzja (P) wyznaczane są najpierw osobno dla każdego klastra c :

$$R_c = \frac{l_{cc}}{\sum_{i=0}^C l_{ci}}, \quad (3.32)$$

Algorytm 12 Permutacja macierzy **L**

Require: **L**: macierz $C \times C$, wyznaczona przez algorytm 11

```

1: procedure PERMUTELMATRIX(L)
2:   for  $i = 1$  to  $C - 1$  do
3:      $(w, k) \leftarrow \arg \max(\mathbf{L}[i:][i:])$   $\triangleright$  Indeksy wiersza i kolumny, w których znajduje
       się maksymalna wartość podmacierzy  $\mathbf{L}[i : ][i : ]$ 
4:      $w \leftarrow w + i$ 
5:      $k \leftarrow k + i$ 
6:     zamiana kolejnością wiersza  $i$ -tego i  $w$ -tego macierzy L
7:     zamiana kolejnością kolumny  $i$ -tej i  $k$ -tej macierzy L
8:   end for
9:   return L
10: end procedure

```

$$P_c = \frac{l_{cc}}{\sum_{i=0}^C l_{ic}}. \quad (3.33)$$

Trafność i precyzja dla całości wyników otrzymywane są jako średnia arytmetyczna metryk częściowych:

$$R = \frac{\sum_{i=0}^C R_i}{C}, \quad (3.34)$$

$$P = \frac{\sum_{i=0}^C P_i}{C}. \quad (3.35)$$

Sposób obliczenia metryk jakości przedstawia również pseudokod 13.

Jeżeli dane zawierają szum, może on być interpretowany na dwa sposoby w kontekście wyznaczania metryk:

1. Szum nie jest brany pod uwagę, analizowane są jedynie dane niebędące szumem.
2. Szum uznawany jest za błędną klasyfikację. Aby skorygować wartość metryki o szum, należy pomnożyć ją przez odsetek punktów niebędących szumem w stosunku do całości danych. Na przykład, jeżeli początkowa wartość metryki wynosi 80%, a 40% danych stanowi szum, to ostateczna wartość tej metryki będzie równa $80\% * (100\% - 40\%) = 48\%$.

Oprócz niezależnych procentowych średnich wartości metryk dokładności, trafności i precyzji istotne jest, jakie są wyniki działania GRAGGLO w porównaniu z algorytmem ostrym. W tym celu wykorzystano algorytm **AgglomerativeClustering** biblioteki **scikit-learn** aby zgrupować dane oryginalne, a także środki klastrów (w liczebnościach 50, 100, 200) wyznaczonych przez algorytm rozmytych c -średnich, bez uwzględnienia rozmycia, czyli macierzy μ .

Algorytm 13 Wyznaczenie dokładności, trafności i precyzji**Require:** $\mathbf{L}$: macierz $C \times C$, otrzymana na podstawie algorytmu 12**Require:** X : liczba danych

```

1: procedure CALCULATEQUALITYMETRICS( $\mathbf{L}$ ,  $X$ )
2:    $diag \leftarrow 0$  ▷ Suma wartości na przekątnej macierzy  $\mathbf{L}$ 
3:    $R \leftarrow 0$ 
4:    $P \leftarrow 0$ 
5:    $suma\_R \leftarrow$  sumy wartości dla każdego wiersza macierzy  $\mathbf{L}$ 
6:    $suma\_P \leftarrow$  sumy wartości dla każdej kolumny macierzy  $\mathbf{L}$ 
7:   for  $i = 1$  to  $C$  do
8:      $diag \leftarrow diag + \mathbf{L}[i, i]$ 
9:     if  $suma\_R[i] > 0$  then
10:       $R \leftarrow R + \mathbf{L}[i, i] / suma\_R[i]$ 
11:     end if
12:     if  $suma\_P[i] > 0$  then
13:       $P \leftarrow P + \mathbf{L}[i, i] / suma\_P[i]$ 
14:     end if
15:   end for
16:    $A \leftarrow diag / X$ 
17:    $R \leftarrow R / C$ 
18:    $P \leftarrow P / C$ 
19:   return  $A, R, P$ 
20: end procedure

```

Do wyznaczenia metryk jakości dla tych badań, również wykorzystano algorytmy 11, 12 oraz 13. W przypadku oryginalnych danych, wejściem algorytmu 11 były etykiety wyznaczone przez algorytm **AgglomerativeClustering**. Dla środków klastrów algorytmu rozmytych c -średnich, najpierw wyznaczono przynależności punktów do tych klastrów na podstawie macierzy μ (tak samo jak dla GRAGGLO).

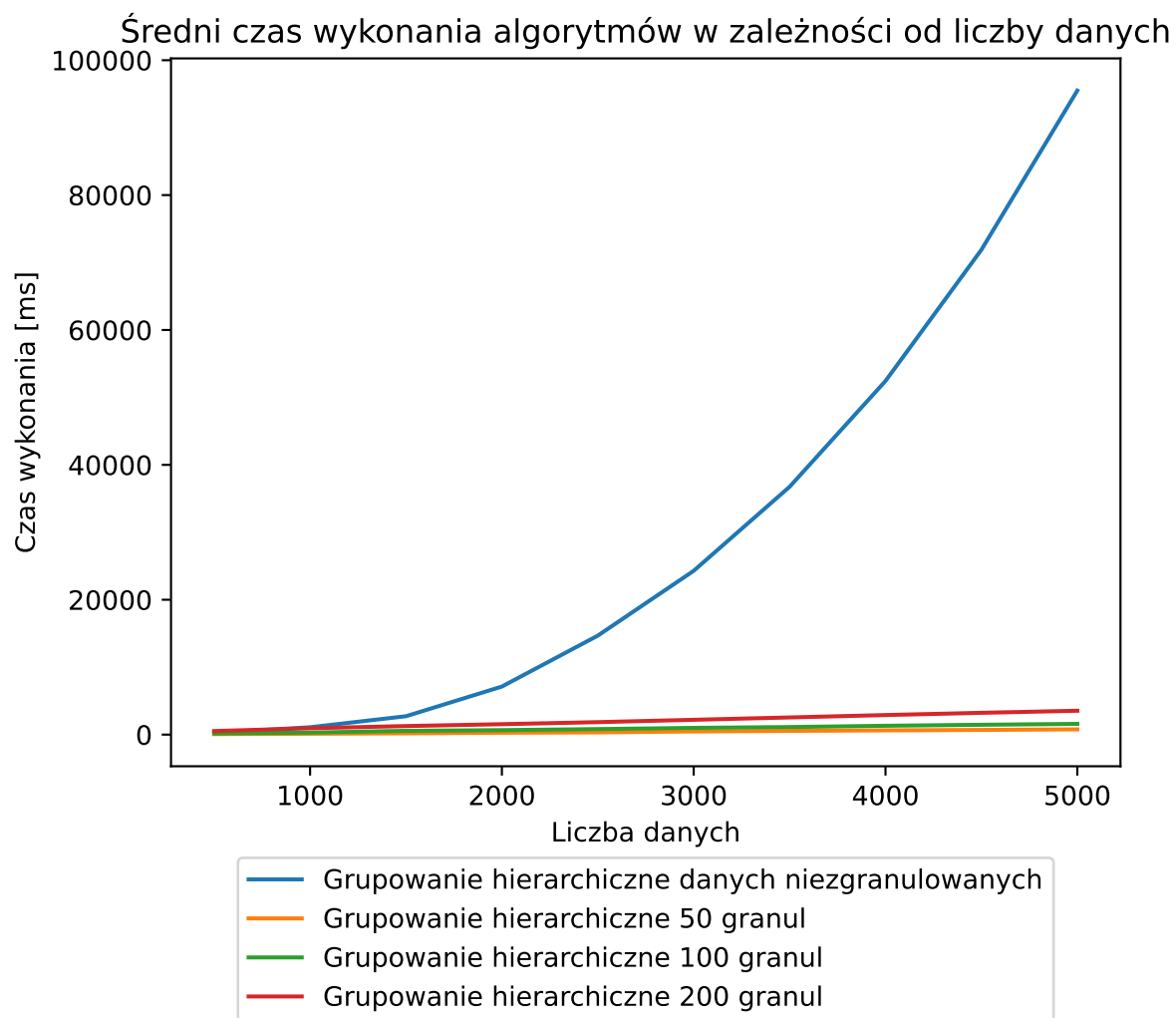
3.5.3 Wyniki

Czas grupowania

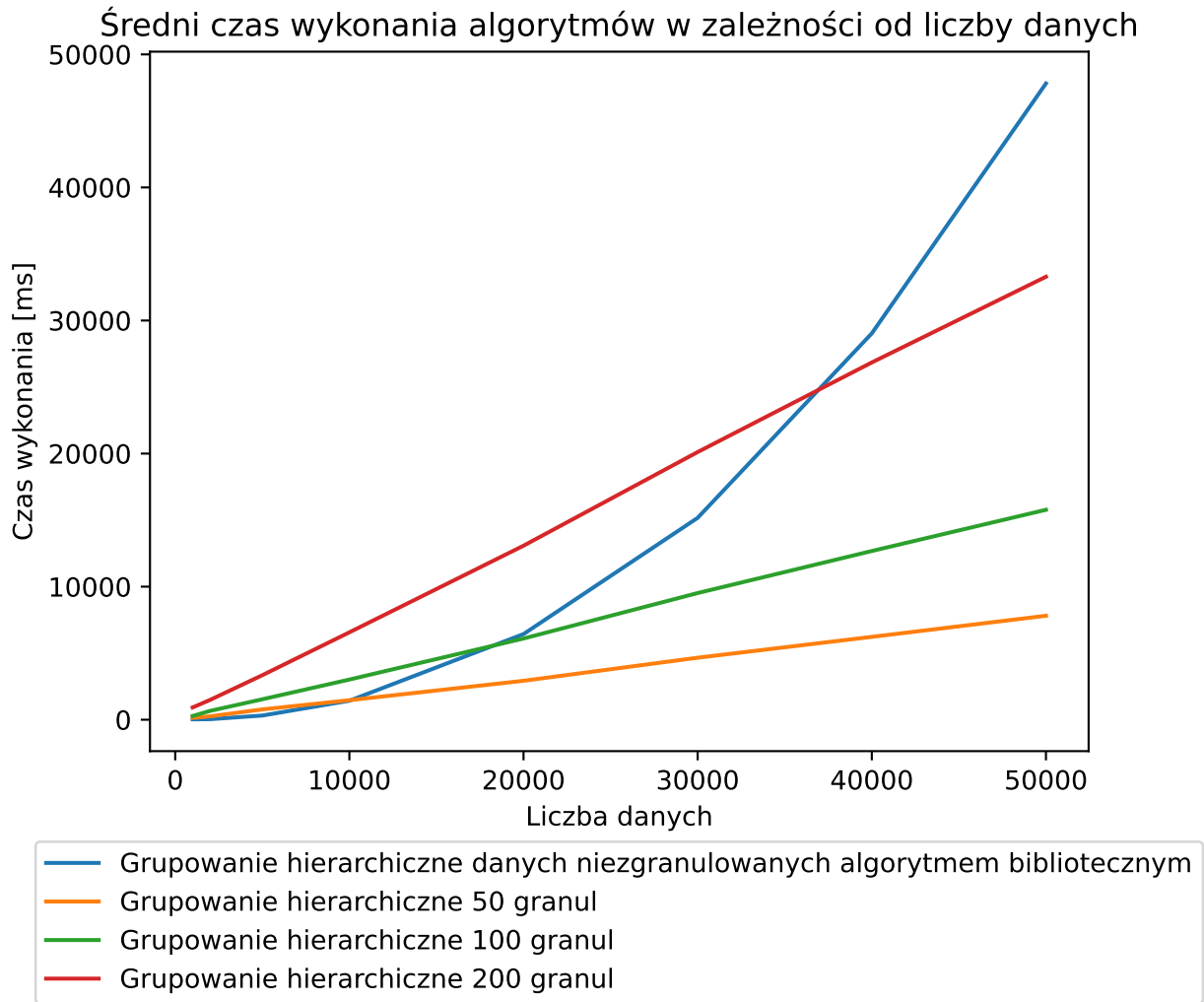
Algorytm GRAGGLO, który wykorzystuje wstępną granulację FCM, ma znacząco mniejszy czas wykonania w stosunku do własnej implementacji algorytmu ostrego. Charakterystykę czasu wykonania algorytmów w zależności od liczby danych przedstawiono na rys. 3.7.

Dla obydwu algorytmów, złożoność czasowa grupowania hierarchicznego jest sześcienna i wynosi dla algorytmu ostrego $O(X^3)$, dla rozmytego $O(G^3)$, gdzie liczba ziaren G jest stała i zdecydowanie mniejsza od liczby danych ($G \ll X$). Złożoność czasowa algorytmu rozmytych c -średnich wynosi $O(X)$, więc jest on mniej wrażliwy na zwiększanie rozmiaru danych wejściowych.

Porównano również czas wykonania algorytmu GRAGGLO z algorytmem **Agglome-**



Rysunek 3.7: Charakterystyka czasu wykonania własnych implementacji algorytmów ostrego i rozmytego dla różnej liczby granul od liczby danych wejściowych.

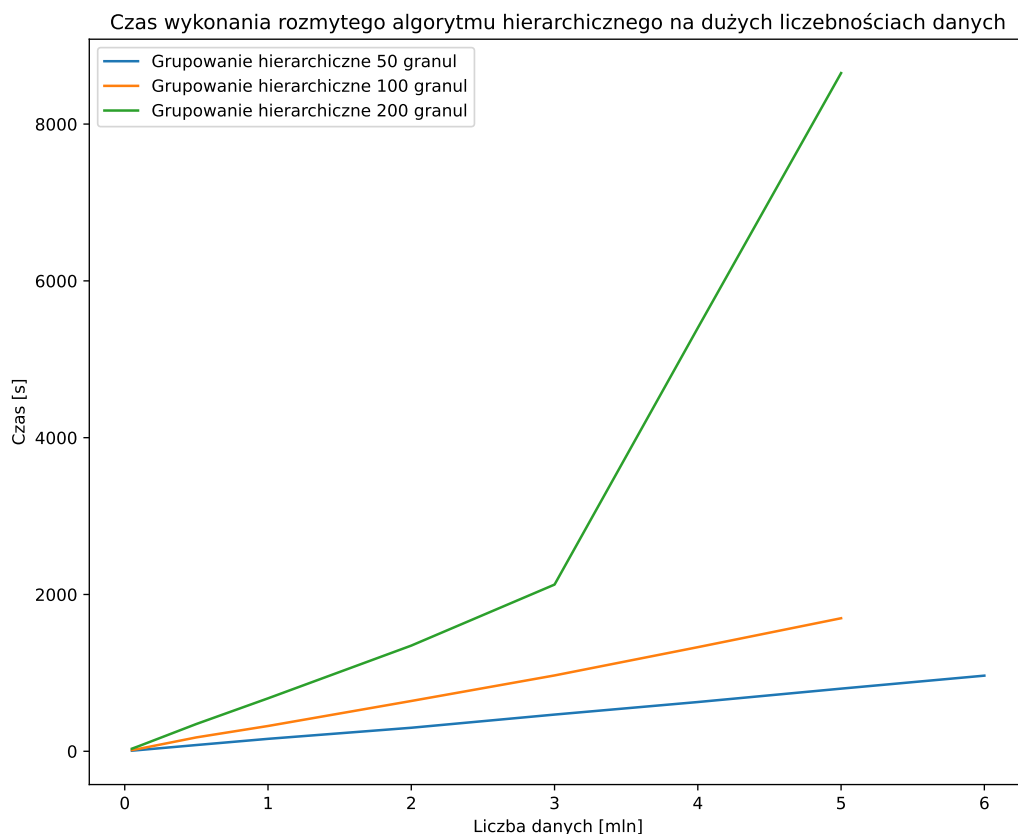


Rysunek 3.8: Charakterystyka czasu wykonania algorytmu ostrego `AgglomerativeClustering` biblioteki `scikit-learn` i własnej implementacji algorytmu rozmytego dla różnej liczby granul od liczby danych wejściowych.

`rativeClustering` biblioteki `scikit-learn`, dla złożoności `SINGLELINKAGE`. Wyniki przedstawiono na rys. 3.8.

Wykres pokazuje, że dla małych danych (do 10 000 punktów), najkrócej wykonuje się algorytm biblioteczny – klasteryzacja algorytmem rozmytych c -średnich trwa stosunkowo długo, mimo złożoności liniowej. Powyżej 10 000 danych algorytm GRAGGLO dla 50 granul wykonywany jest szybciej niż algorytm ostry. Od 40 000 danych, jest on szybszy dla wszystkich badanych liczebności granul. Oznacza to, że mimo gorszej złożoności samego algorytmu hierarchicznego FUZZYAGGLO, całość algorytmu rozmytego jest lepsza pod względem czasu wykonania od algorytmu `AgglomerativeClustering`, powyżej pewnej liczności danych.

Czasy wykonania algorytmu rozmytego GRAGGLO na dużych danych przedstawia rys. 3.9. W przypadku 200 granul zauważyć można zaburzenie liniowości wyników. Spowodowane jest zbyt dużym wykorzystaniem pamięci: sama macierz μ ma wymiary $X \times G$,



Rysunek 3.9: Charakterystyka czasu wykonania algorytmu rozmytego dla różnej liczby granul od liczby danych wejściowych dla dużych danych.

czyli dla 4 milionów danych wejściowych zawiera ona 800 milionów liczb zmiennoprzecinkowych. Z tego samego powodu możliwe było wykonanie algorytmu na 6 milionach danych tylko dla 50 granul – dla większej liczby nie wystarczyło pamięci.

Jakość grupowania

Na wykresach wyników grupowania hierarchicznego metodą rozmytą punkty szare odpowiadają współrzędnym oryginalnych danych. Wyjątkiem są wykresy danych ‘spheres’, gdzie oryginalne punkty pominięto, aby zachować czytelność wykresu. Punkty kolorowe odpowiadają środkom granul, gdzie środki o tym samym kolorze należą do tego samego klastra. Elipsy (elipsoidy dla ‘spheres’) w kolorze różowym wizualizują rozmycie granul, gdzie długość każdej półosi jest równa dwukrotności rozmycia w danym wymiarze. Mnożnik ten został wybrany ze względu na dobrą czytelność wynikowych wykresów.

Wszystkie wykresy zamieszczono w katalogu `img/granule_visualization`, pogrupowane w podkatalogach według kształtu danych (‘blobs’, ‘circles’, ‘corners’, ‘crescents’, ‘laguna’ i ‘spheres’). Nazwa każdego pliku ma format `[kształt][liczba_danych][re-`

Tabela 3.3: Dokładność grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE

kształt	dokładność			
	dane oryginalne	50 granul	100 granul	200 granul
‘blobs’	0,9899	0,9999	0,9939	0,9696
‘circles’	1,0000	1,0000	1,0000	1,0000
‘corners’	1,0000	1,0000	1,0000	1,0000
‘crescents’	1,0000	1,0000	1,0000	1,0000
‘laguna’	0,9125	0,9997	0,9998	0,9999
‘spheres’	1,0000	1,0000	1,0000	1,0000

Tabela 3.4: Trafność grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE

kształt	trafność			
	dane oryginalne	50 granul	100 granul	200 granul
‘blobs’	0,8333	1,0000	0,9571	0,7487
‘circles’	1,0000	1,0000	1,0000	1,0000
‘corners’	1,0000	1,0000	1,0000	1,0000
‘crescents’	1,0000	1,0000	1,0000	1,0000
‘laguna’	0,7084	0,9990	0,9995	0,9997
‘spheres’	1,0000	1,0000	1,0000	1,0000

lacja]_ [łączność].pdf, gdzie „relacja”: t (trójkątna), e (wykładnicza) lub g (gaussowska), a „łączność”: s (SINGLELINKAGE) lub c (COMPLETELINKAGE).

Dla niektórych wyników utworzono również wykresy na podstawie sumarycznej przynależności punktów do granul z danego klastra. Mają postać kilku podwykresów, gdzie punktami w kolorze niebieskim oznaczono dane oryginalne. Dla każdego podwykresu opisanego jako „Klaster c ”, dla każdego punktu x wyznaczana jest suma $\sum_{g=0}^G \mu_{xg} \cdot [g \in c]$, gdzie zapis $[P]$ wartościowany jest do 1, gdy warunek P jest spełniony, w przeciwnym razie do 0 (wartość μ_{xg} jest wliczana do sumy, gdy granula g należy do klastra c). Wyższe wartości tej sumy oznaczają większą przynależność punktu do tego klastra. Im wyższa wartość sumy, tym ciemniejszy kolor punktu na wykresie.

Dane syntetyczne Wyniki dokładności, trafności i precyzji otrzymane na podstawie ostrego algorytmu hierarchicznego **AgglomerativeClustering** grupującego dane oryginalne i środki klastrów wyznaczonych przez algorytm rozmytych c -średnich przedstawiają odpowiednio tabele 3.3, 3.4 i 3.5 dla łączności SINGLELINKAGE oraz 3.6, 3.7 i 3.8 dla łączności COMPLETELINKAGE.

Tabela 3.5: Precyzja grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE

kształt	precyzja			
	dane oryginalne	50 granul	100 granul	200 granul
‘blobs’	0,8295	0,9990	0,9392	0,7282
‘circles’	1,0000	1,0000	1,0000	1,0000
‘corners’	1,0000	1,0000	1,0000	1,0000
‘crescents’	1,0000	1,0000	1,0000	1,0000
‘laguna’	0,8375	0,9996	0,9998	0,9998
‘spheres’	1,0000	1,0000	1,0000	1,0000

Tabela 3.6: Dokładność grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE

kształt	dokładność			
	dane oryginalne	50 granul	100 granul	200 granul
‘blobs’	0,7261	0,9999	0,8956	0,7237
‘circles’	0,5620	0,5533	0,5330	0,5627
‘corners’	0,5265	0,4818	0,5053	0,5130
‘crescents’	0,7986	0,8012	0,8032	0,7822
‘laguna’	0,4508	0,4875	0,4653	0,4664
‘spheres’	0,5852	0,5712	0,5650	0,6615

Tabela 3.7: Trafność grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE

kształt	trafność			
	dane oryginalne	50 granul	100 granul	200 granul
‘blobs’	0,7825	1,0000	0,8822	0,9041
‘circles’	0,4900	0,4940	0,4876	0,5071
‘corners’	0,5265	0,4818	0,5053	0,5130
‘crescents’	0,7986	0,8012	0,8033	0,7822
‘laguna’	0,5447	0,6026	0,4856	0,5072
‘spheres’	0,5473	0,3729	0,4805	0,5897

Tabela 3.8: Precyzja grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE

kształt	precyzja			
	dane oryginalne	50 granul	100 granul	200 granul
‘blobs’	0,6054	0,9990	0,8753	0,6591
‘circles’	0,4881	0,4948	0,4880	0,5052
‘corners’	0,4956	0,4779	0,4631	0,4828
‘crescents’	0,8372	0,8072	0,8285	0,8093
‘laguna’	0,3984	0,4302	0,3561	0,3804
‘spheres’	0,5143	0,4480	0,4891	0,5324

Tabela 3.9: Dokładność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności

kształt	dokładność	
	SINGLELINKAGE	COMPLETELINKAGE
‘blobs’	0,9874	0,8758
‘circles’	0,9982	0,5738
‘corners’	0,9485	0,5167
‘crescents’	0,9943	0,7977
‘laguna’	0,9876	0,4685
‘spheres’	0,9876	0,7056

Kształt danych Średnią dokładność grupowania danych w zależności od ich kształtu dla łączności SINGLELINKAGE i COMPLETELINKAGE przedstawiono w tabeli 3.9, trafność – w tabeli 3.10, a precyzję – w tabeli 3.11.

W przypadku SINGLELINKAGE, najlepszą średnią dokładność, trafność i precyzję zaobserwowano dla danych ‘circles’. Dane ‘corners’ mają najgorszą dokładność, a dane ‘blobs’ – najgorszą trafność i precyzję.

Co istotne, dla większości plików wejściowych, za wyjątkiem niektórych danych ‘blobs’ i ‘laguna’, maksymalne wartości wszystkich metryk jakości wynosi 100%. Oznacza to, że istnieje przynajmniej jedna taka kombinacja parametrów (typu relacji, liczby granul, parametru ξ), dla których grupowanie przebiega w pełni zgodnie z oczekiwaniami.

‘blobs’ Wykresy wyników grupowania danych ‘blobs’ (rys. 3.10) pokazują, że wysoka dokładność uzyskana dla nich z macierzy pomyłek nie przekłada się na zgodność wyników z oczekiwaniami. Ze względu na dużą dysproporcję danych pomiędzy trzema klastrami (48 : 1 : 1), algorytm rozmytych c -średnich często traktuje dwa mniejsze z nich jako szum i nie tworzy dla nich granul. Zawierają one jednak jedynie 4% danych, dlatego dokładność grupowania pozostaje wysoka. Trafność i precyzja lepiej odzwierciedlają zgodność z oczekiwaniami – są znacznie niższe od dokładności. Mniejsze klastry mają tę samą

Tabela 3.10: Trafność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności

kształt	trafność	
	SINGLELINKAGE	COMPLETELINKAGE
‘blobs’	0,8846	0,8968
‘circles’	0,9971	0,4970
‘corners’	0,9485	0,5167
‘crescents’	0,9943	0,7977
‘laguna’	0,9625	0,5571
‘spheres’	0,9512	0,4086

Tabela 3.11: Precyzja grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności

kształt	precyzja	
	SINGLELINKAGE	COMPLETELINKAGE
‘blobs’	0,8727	0,8325
‘circles’	0,9984	0,4947
‘corners’	0,9523	0,4840
‘crescents’	0,9964	0,8207
‘laguna’	0,9812	0,4003
‘spheres’	0,9480	0,4442

Tabela 3.12: Maksymalne wartości metryk jakości dla danych ‘blobs’ przy łączności SINGLELINKAGE.

rozmiar danych	jakość		
	dokładność	trafność	precyzja
1000	1,0000	1,0000	1,0000
2000	1,0000	1,0000	1,0000
5000	0,9998	0,9999	0,9967
10 000	1,0000	1,0000	1,0000
20 000	1,0000	1,0000	1,0000
30 000	0,9999	1,0000	0,9989
40 000	1,0000	1,0000	1,0000
50 000	0,9998	1,0000	0,9997

wagę, co większy klaster, dlatego niepoprawne grupowania wyraźnie wpływają na wartość metryki. Jako że błędy wynikają z klasteryzacji FCM, wyniki uzyskane dla grupowania samych środków granul są bardzo zbliżone do uzyskanych przy pomocy GRAGGLO.

Jak pokazuje tabela 3.12, dla niektórych liczebności danych nie istnieje kombinacja parametrów, dla której wartości metryk jakości wynoszą 100%. W tych przypadkach granule wyznaczane są w poprawnych miejscach, natomiast pojedyncze punkty oryginalnych danych zostają przypisane do niepoprawnej granuli. Dla tych liczebności, wartości metryk otrzymane przez algorytm ostry również nie osiągają 100%.

‘circles’ Dane ‘circles’ (rys. 3.11) mają najwyższe wyniki dokładności, trafności i precyzji. Dla mniejszej liczby ($G = 50$) rozmyte odległości między granulami położonymi na zewnętrznym okręgu są większe niż między okręgiem zewnętrznym a wewnętrznym, jedyne błędy grupowania pojawiają się dla tej liczebności granul, dla wysokich wartości parametru ξ ($\xi \in \{0,9, 0,95\}$). W znacznej większości przypadków, dokładność grupowania wynosi 100%, tak jak dla grupowania ostrego. Okrąg zewnętrzny jest też bardziej liczny niż wewnętrzny (proporcja 7:3). Dysproporcja ta nie jest jednak tak duża, jak w przypadku ‘blobs’, algorytm rozmytych c -średnich tworzy granule w równomiernych odstępach.

‘corners’ Klastry dla danych ‘corners’ mają nierównomierną gęstość, przez co algorytm rozmytych c -średnich tworzy więcej granul bliżej środka układu współrzędnych (rys. 3.12). Odległości między granulami położonymi na „ramionach” i w „środkach” klastrów są na tyle duże, że rozmyte odległości „środków” różnych klastrów mogą być od nich mniejsze. Algorytm GRAGGLO osiąga więc średnio gorsze wyniki niż algorytm ostry, choć wiele kombinacji parametrów pozwala na uzyskanie 100-procentowej dokładności. Wszystkie klastry mają tę samą liczebność, wartości dokładności, trafności i precyzji są zbliżone do siebie.

‘crescents’ Dane ‘crescents’ tworzą dwa klastry o identycznej liczebności i gęstości, dzięki czemu dokładność, trafność i precyzja ich grupowania są prawie najlepsze (rys. 3.13). Ze względu na bliskie położenie końców łuków, błędy wciąż mogą się pojawić – koniec jednego łuku złączony zostaje z łukiem drugim. Dzieje się tak tylko dla bardzo wysokich wartości parametru ξ – 0,9 i 0,95.

‘laguna’ Dla danych ‘laguna’ błędy mogą pojawić się, gdy pojedyncze granule utworzone zostają w większej odległości od środkowego klastra, do którego oczekiwane jest, że powinny należeć. Punkt taki może wtedy zostać zakwalifikowany, jako osobny klaster, co na rysunku 3.14 pokazuje przykład dla 50 granul i $\xi = 0,9$. Podobny błąd przedstawia wykres dla 200 granul i $\xi = 0,9$ – pewna grupa punktów na otaczającym okręgu jest na tyle oddalona od pozostałych, że zostaje uznana za osobny klaster, jeden ze środkowych klastrów jest łączony z pozostałą częścią okręgu.

Powyżej liczebności 20 000 punktów włącznie, dokładność nigdy nie osiąga 100% (tab. 3.13), jednak najczęściej wynosi powyżej 99,9%. Jak zaznaczono w sekcji dotyczącej etykietowania danych (sekcja na stronie 26), środkowe klastry są na tyle zbliżone do siebie, że na podstawie wykresu nie można definitywnie określić, które punkty pomiędzy nimi należą do którego z nich. Pojedyncze punkty mogą więc być klasyfikowane niepoprawnie z punktu widzenia przypisanych do nich etykiet.

Ilustrują to wykresy 3.15 oraz 3.16. Na rys. 3.15 punktów jest jedynie 1000 i wszystkie klastry są wyraźnie oddzielone od siebie. Każdy punkt wyraźnie należy do granul jednego klastra. Na rys. 3.16 punktów jest 40 000. Klastry środkowe (o numerach 1 i 2) pokrywają się w pobliżu współrzędnych (0, 0). Punkty w bliskim otoczeniu tych współrzędnych przynależą więc w podobnym stopniu do klastra 1 i klastra 2. Ponadto punkty położone na zewnątrz klastrów 1 i 2 są położone blisko otaczającego je okręgu, oznaczonego jako klaster 0. Można zauważyć ich wysoki stopień przynależności do tego klastra.

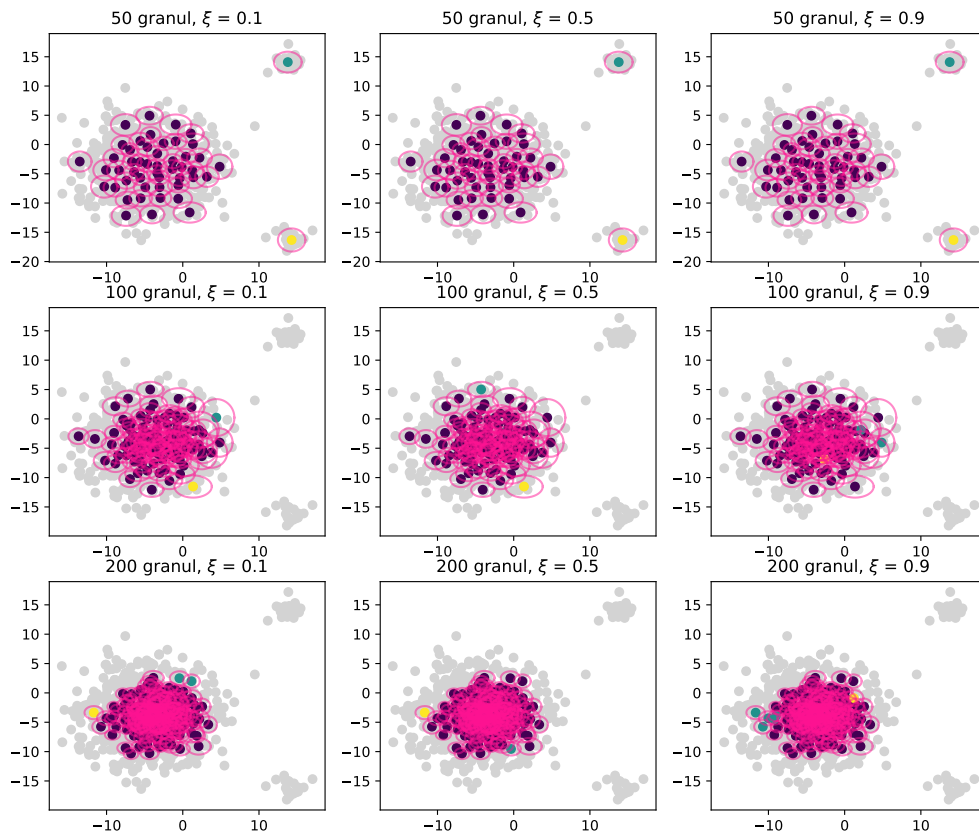
Dzięki temu nałożeniu się klastrów, jakość grupowania tych danych jest lepsza, gdy wykorzystany zostanie algorytm GRAGGLO, niż gdy zgrupowane zostaną dane oryginalne. Bez wstępnej klasteryzacji algorytmem rozmytych c -średnich, środkowe klastry są łączone ze sobą.

Proporcja danych wynosi 8:1:1 (otaczający okrąg ma większą liczebność), jednak jakość grupowania jest na tyle wysoka, że zarówno dokładność, jak i trafność i precyzja mają podobne wartości.

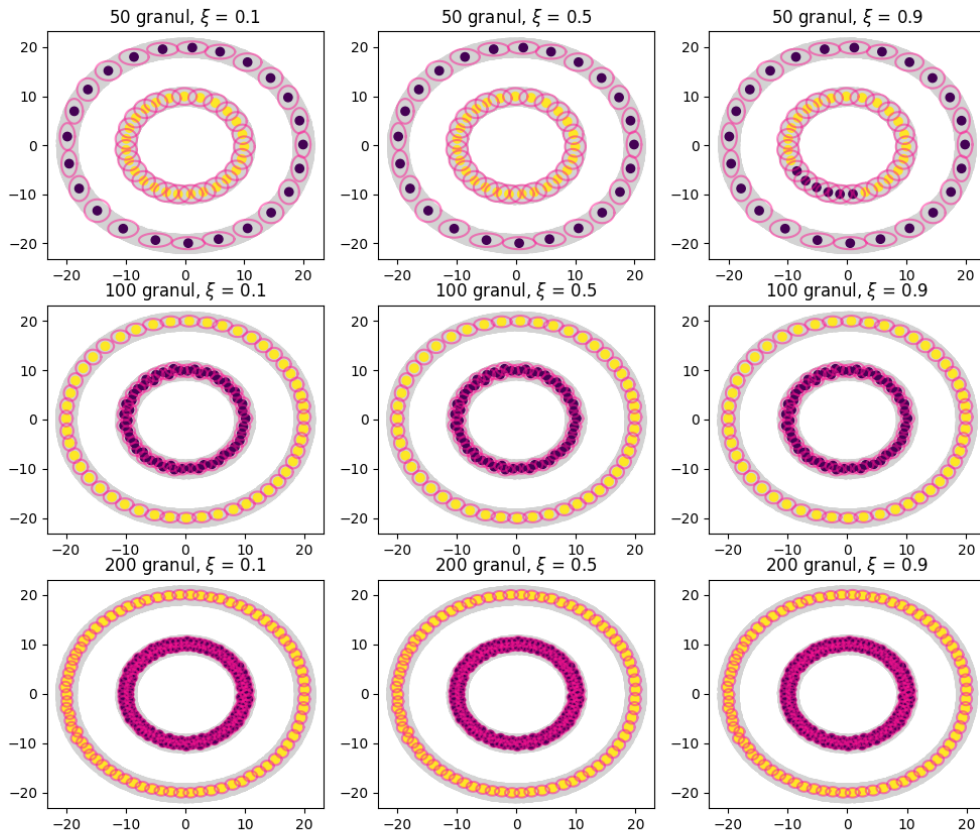
‘spheres’ W przypadku danych ‘spheres’ (rys. 3.17) można zauważyć, że wyniki poprawiają się wraz ze zwiększeniem liczby granul i zmniejszeniem wartości parametru ξ . Proporcja liczebności klastrów wynosi 9:1, trafność i precyzja są o około 4 punkty procentowe mniejsze od dokładności.

Tabela 3.13: Maksymalne wartości metryk jakości dla danych ‘laguna’ przy łączności SINGLELINKAGE.

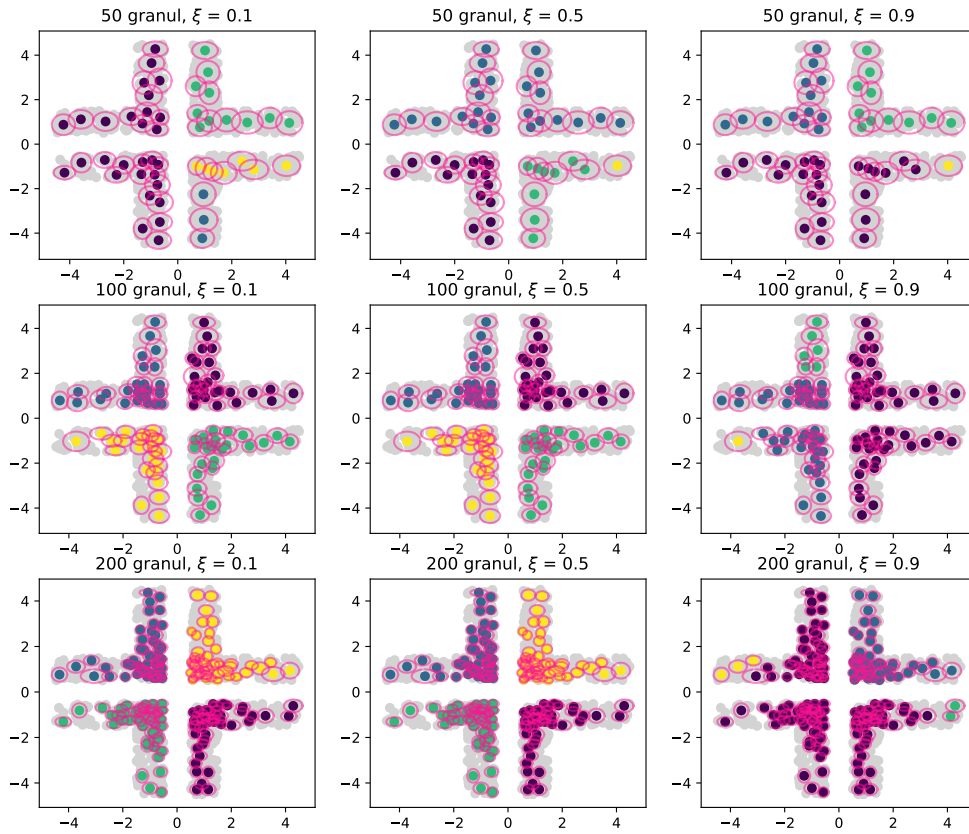
rozmiar danych	jakość		
	dokładność	trafność	precyzja
1000	1,0000	1,0000	1,0000
2000	1,0000	1,0000	1,0000
5000	1,0000	1,0000	1,0000
10 000	1,0000	1,0000	1,0000
20 000	0,9999	0,9997	0,9998
30 000	0,9999	0,9998	0,9999
40 000	0,9999	0,9998	0,9998
50 000	0,9997	0,9990	0,9997



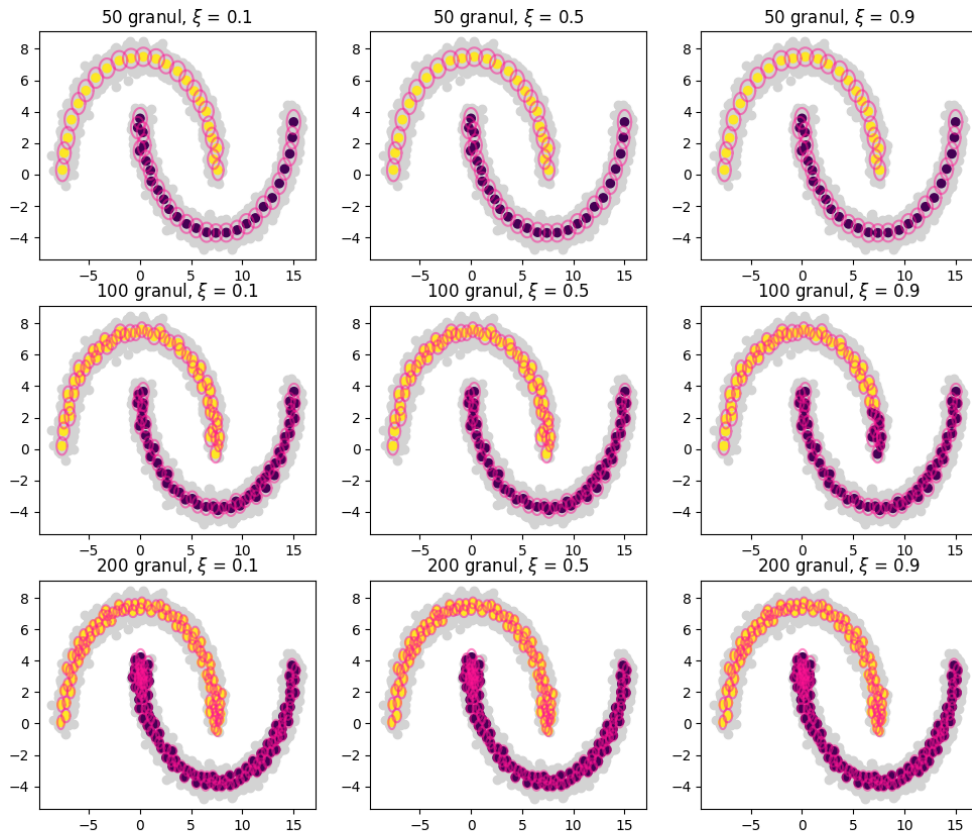
Rysunek 3.10: Efekty wykonania grupowania GRAGGLO dla danych ‘blobs’ o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.



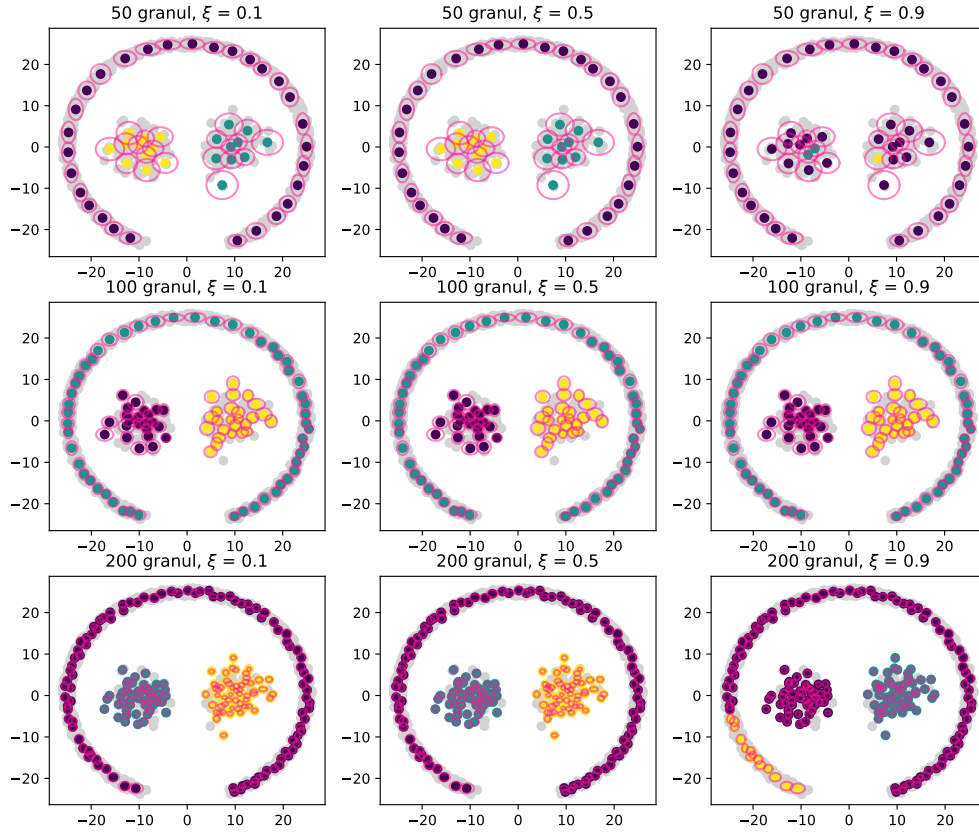
Rysunek 3.11: Efekty wykonania grupowania GRAGGLO dla danych ‘circles’ o rozmiarze 50 000 i euklidesowej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.



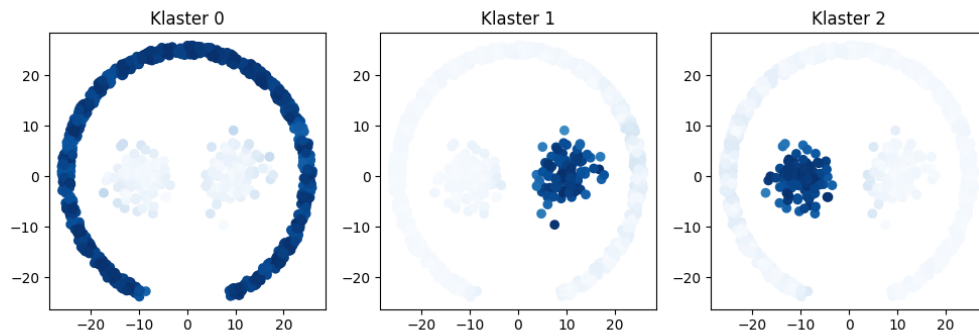
Rysunek 3.12: Efekty wykonania grupowania GRAGGLO dla danych ‘corners’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.



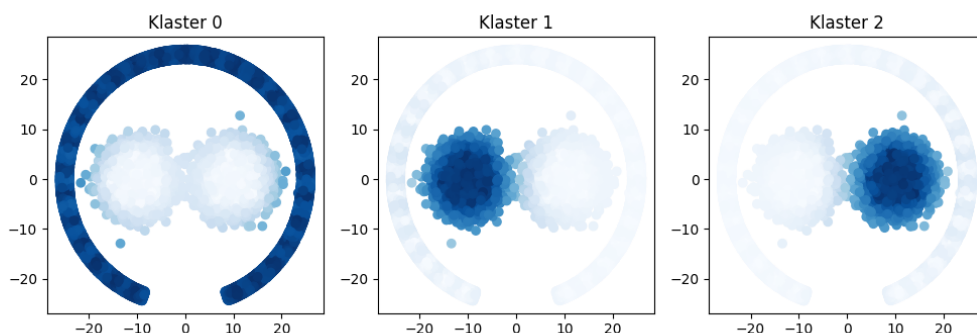
Rysunek 3.13: Efekty wykonania grupowania GRAGGLO dla danych ‘crescents’ o rozmiarze 5000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.



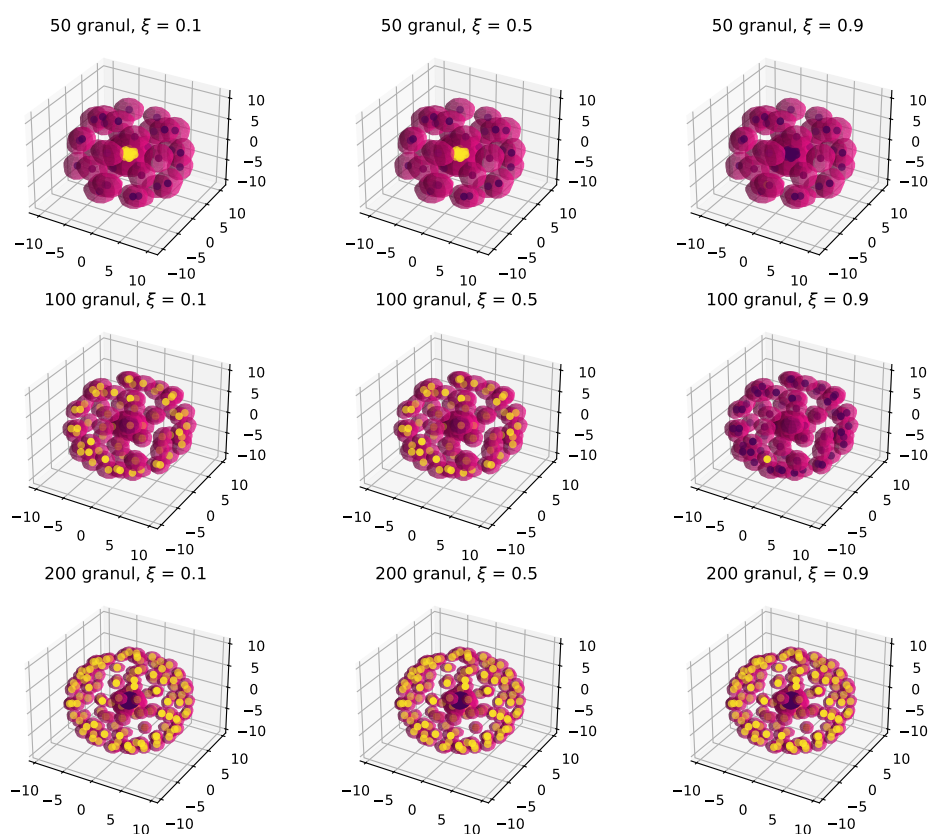
Rysunek 3.14: Efekty wykonania grupowania GRAGGLO dla danych ‘laguna’ o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.



Rysunek 3.15: Przynależność punktów do klastrów dla danych ‘laguna’ o rozmiarze 1000.



Rysunek 3.16: Przynależność punktów do klastrów dla danych ‘laguna’ o rozmiarze 40 000.



Rysunek 3.17: Efekty wykonania grupowania hierarchicznego granul dla danych ‘spheres’ o rozmiarze 5000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.

CompleteLinkage Najlepsze wyniki dla łączności COMPLETELINKAGE uzyskano dla danych ‘blobs’. Inaczej jest w przypadku najgorszych wyników: dane ‘laguna’ mają najgorszą dokładność i precyzję, a dane ‘spheres’ – najgorszą trafność. Wszystkie wyniki są znacznie gorsze od uzyskanych dla SINGLELINKAGE, za wyjątkiem trafności dla danych ‘blobs’. Jak widać na wykresach danych ‘blobs’ (rys. 3.18), podobnie jak dla łączności SINGLELINKAGE, jeżeli granule zostaną wyznaczone przez algorytm rozmytych c -średnich we wszystkich trzech docelowych klastrach, to grupowanie FUZZYAGGLO przebiega poprawnie. Z kolei jeżeli granule utworzone zostaną tylko w największym z klastrów, to wyznaczone zostają w nim trzy klastry o podobnych liczebnościach. W takich przypadkach dokładność, trafność i precyzja grupowania będą niższe, niż dla SINGLELINKAGE, gdzie prawie wszystkie granule przypisywane są do tego samego klastra.

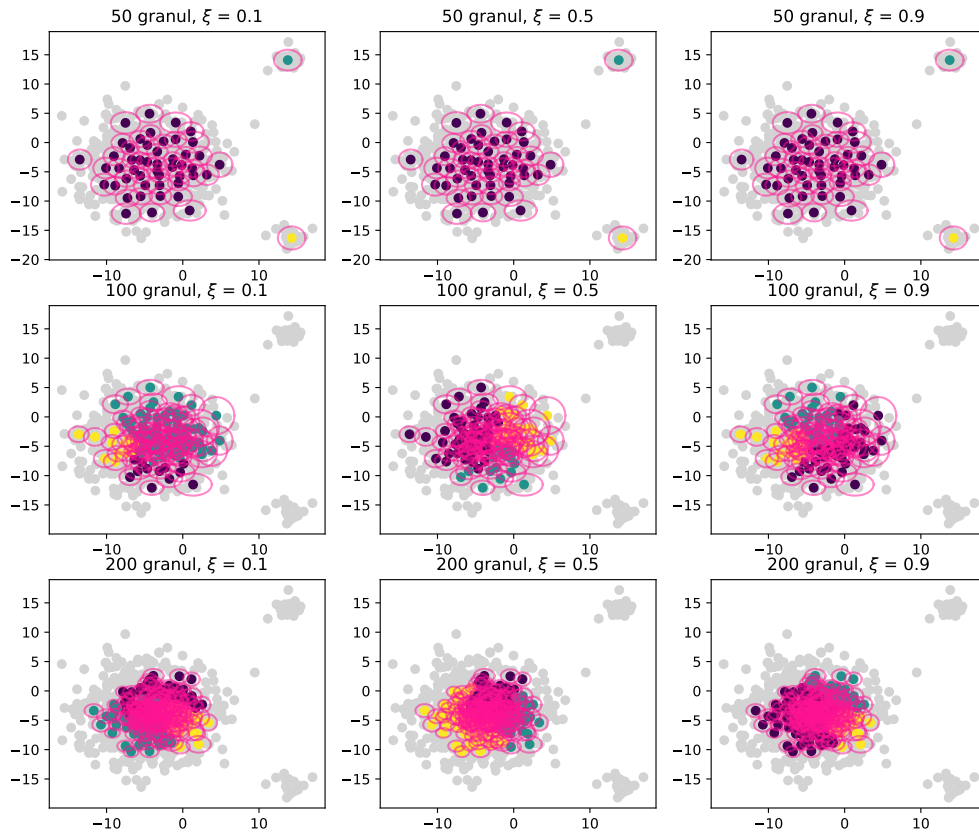
Mimo różnic w wartościach miar jakości, wnioski dla grupowania danych ‘circles’ (rys. 3.19), ‘corners’ (rys. 3.20), ‘crescents’ (rys. 3.21), ‘laguna’ (rys. 3.22) i ‘spheres’ (rys. 3.23) są podobne. Algorytm COMPLETELINKAGE, jak opisano w podsekcji 2.1.2, dobrze znajduje gęste, kuliste klastry. Wymienione dane mają inne kształty, widać, że znalezione przez algorytm klastry są w przybliżeniu owalne lub elipsoidalne. Kształt danych wydaje się nie być brany pod uwagę przez algorytm.

Wyniki te należy też analizować w porównaniu z wynikami algorytmu ostrego (tabele 3.6, 3.7 i 3.8). Dla kształtów danych ‘blobs’ oraz ‘spheres’ nastąpiła znacząca poprawa dokładności, trafności i precyzji. Dla pozostałych danych wyniki są zbliżone do wyników algorytmu ostrego.

Przynależność określona na podstawie relacji *-podobieństwa Wyniki dla pomiarów, gdzie przynależność punktów do granul określono na podstawie relacji *-podobieństwa przedstawiono w tabelach 3.14, 3.15 i 3.16. Dodatkowo, gdy szum nie jest uwzględniany, dla każdego pliku wejściowego istnieje taka kombinacja parametrów, dla której wartości metryk wynoszą 100%. Dla relacji trójkątnej, tylko pewien podzbiór punktów najbliższych środków granul będzie uwzględniony, jest więc mniejsza szansa na niezgodne z oczekiwaniami przypisanie do granuli. Jak pokazuje rys. 3.6, wiąże się to jednak z utratą dużego odsetka informacji.

Gdy szum nie jest uwzględniany w obliczaniu metryk, ich wartości są zbliżone do metryk dla przynależności określonej na podstawie macierzy μ (tabele 3.9, 3.10 i 3.11), natomiast gdy jest on uwzględniony, wyniki są niższe o nawet kilkanaście punktów procentowych.

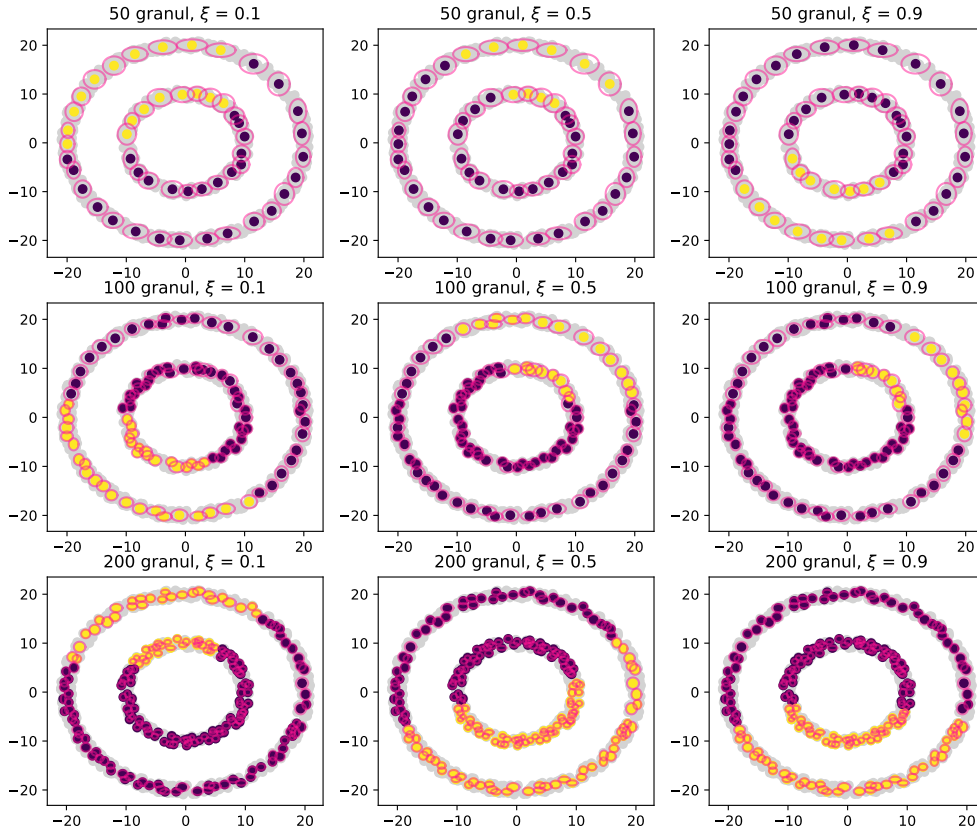
Podsumowanie Podobnie jak w przypadku algorytmu ostrego, GRAGGLO osiąga znacznie lepsze wyniki jakości grupowania wykorzystując łączność SINGLELINKAGE niż COMPLETELINKAGE. W większości przypadków, algorytm rozmyty jest w stanie osiągnąć te same, a nawet lepsze wyniki niż algorytm ostry.



Rysunek 3.18: Efekty wykonania grupowania GRAGGLO dla danych ‘blobs’ o rozmiarze 1000 i trójkątnej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Tabela 3.14: Dokładność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa

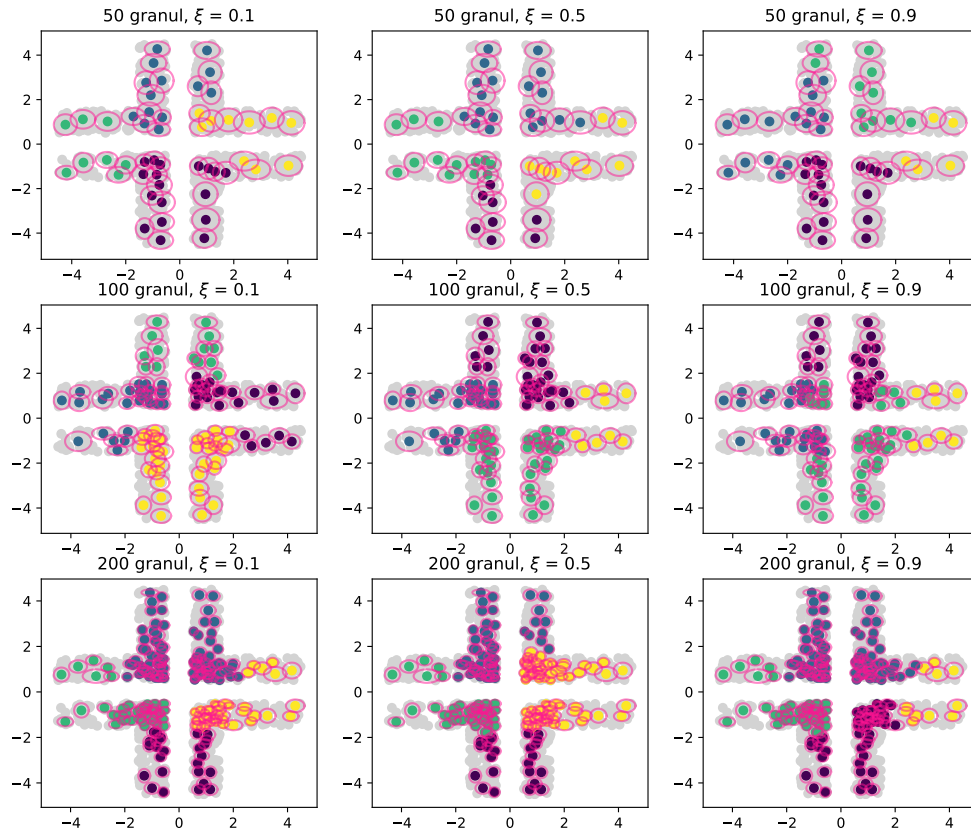
kształt	dokładność			
	bez szumu		z szumem	
	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
‘blobs’	0,9900	0,8755	0,8639	0,7609
‘circles’	0,9982	0,5596	0,8136	0,4623
‘corners’	0,9488	0,5248	0,7700	0,4285
‘crescents’	0,9943	0,7869	0,8188	0,6522
‘laguna’	0,9870	0,4748	0,7995	0,3827
‘spheres’	0,9871	0,6665	0,7806	0,5485



Rysunek 3.19: Efekty wykonania grupowania GRAGGLO dla danych ‘circles’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Tabela 3.15: Trafność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa

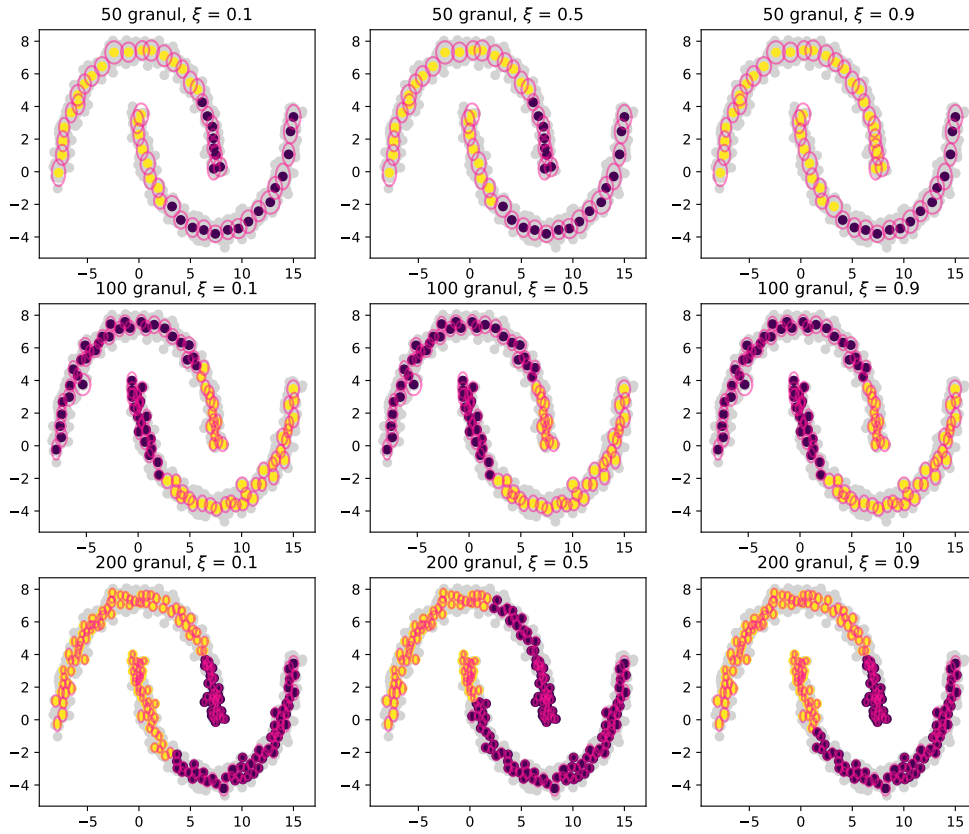
kształt	trafność			
	bez szumu		z szumem	
	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
‘blobs’	0,8585	0,8508	0,7474	0,7435
‘circles’	0,9971	0,4976	0,8125	0,4056
‘corners’	0,9485	0,5229	0,7699	0,4277
‘crescents’	0,9943	0,7868	0,8188	0,6521
‘laguna’	0,9629	0,5566	0,7769	0,4513
‘spheres’	0,9512	0,4083	0,7450	0,3251



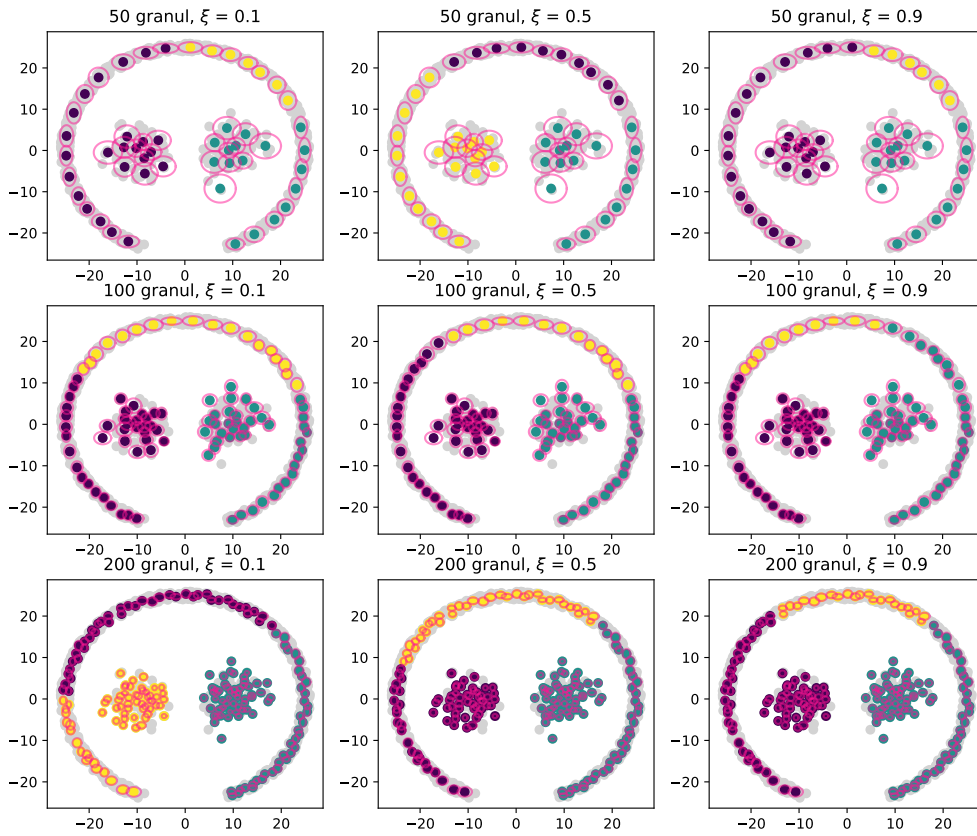
Rysunek 3.20: Efekty wykonania grupowania GRAGGLO dla danych ‘corners’ o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.

Tabela 3.16: Precyzja grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa

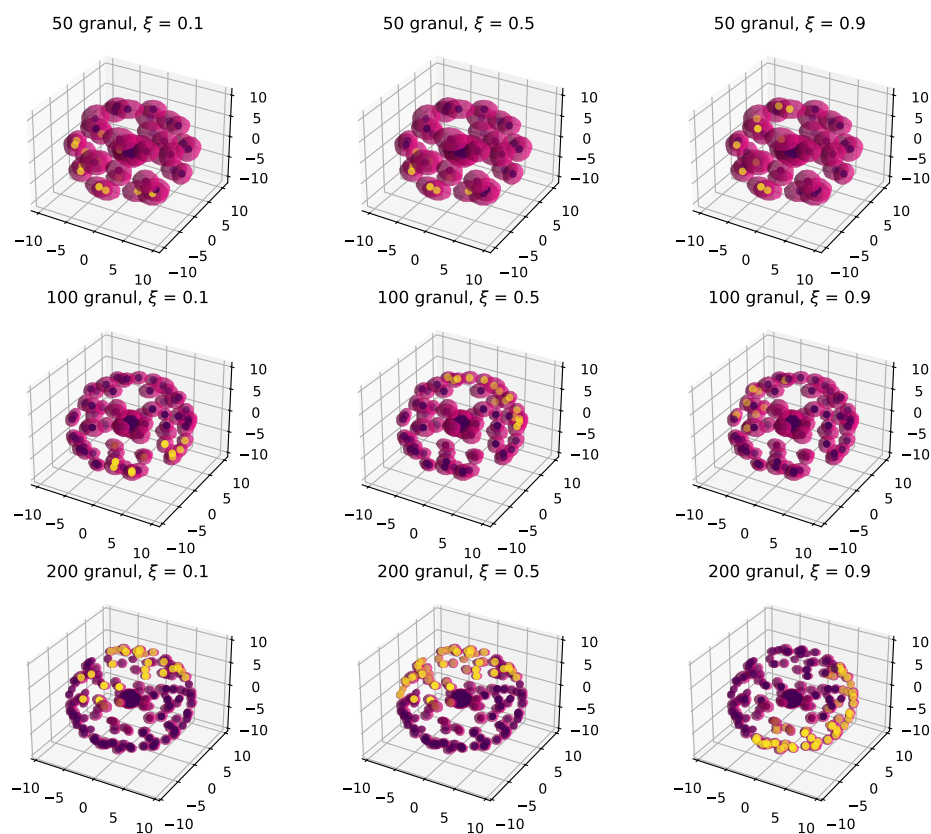
kształt	precyzja			
	bez szumu		z szumem	
	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
‘blobs’	0,8525	0,8278	0,7412	0,7179
‘circles’	0,9984	0,4949	0,8138	0,4034
‘corners’	0,9524	0,4894	0,7740	0,4006
‘crescents’	0,9962	0,8105	0,8207	0,6718
‘laguna’	0,9768	0,4077	0,7920	0,3277
‘spheres’	0,9477	0,4151	0,7418	0,3411



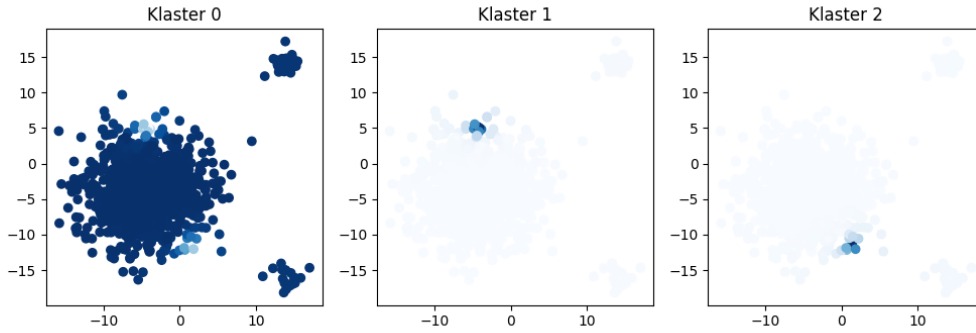
Rysunek 3.21: Efekty wykonania grupowania GRAGGLO dla danych ‘crescents’ o rozmiarze 1000 i trójkątnej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.



Rysunek 3.22: Efekty wykonania grupowania GRAGGLO dla danych ‘laguna’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.



Rysunek 3.23: Efekty wykonania grupowania hierarchicznego granul dla danych ‘spheres’ o rozmiarze 5000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.



Rysunek 3.24: Przynależność punktów do klastrów dla danych ‘blobs’ o rozmiarze 1000.

Poprawa w szczególności zachodzi w sytuacji, gdy oczekiwane klastry zaczynają się pokrywać i istnieją punkty bliskie różnym grupom, na przykład dla danych ‘laguna’ o dużych liczebnościach. Dla algorytmu ostrego, punkt taki może spowodować złączenie różnych klastrów, co pokazano na rys. 2.4. W przypadku algorytmu rozmytego granule są uogólnieniem grupy danych, co eliminuje takie pojedyncze punkty. Ponadto algorytm rozmytych c -średnich wyznacza granule bliżej środków klastrów, co dodatkowo pomaga w poprawnym grupowaniu.

Z drugiej strony, algorytm FCM może wprowadzić błędy w fazie przygotowania danych, jak pokazał przypadek danych ‘blobs’. Algorytm może błędnie wyznaczyć granule, gdy dane są bardzo nie zrównoważone pod względem liczebności klastrów.

Liczba ziaren Średnią dokładność, trafność i precyzję grupowania danych w zależności od liczby granul przedstawiono odpowiednio w tabelach 3.17, 3.18 i 3.19. W przypadku łączności SINGLELINKAGE najlepsze wyniki osiągnęte są dla 100 granul, a dla COMPLETELINKAGE – dla 50.

Dla danych takich, jak ‘circles’ (rys. 3.11) czy ‘crescents’ (rys. 3.13), gdzie klastry mają podobną liczebność i gęstość, zwiększenie liczby granul powoduje ich zagęszczenie, przy równomiernym rozłożeniu.

Inaczej jest w przypadku danych ‘blobs’ (rys. 3.10). Dla większych liczebności granul, możliwe jest, że algorytm rozmytych c -średnich nie wyznaczy ani jednego środka granuli w dwóch mniejszych klastrach, przez co niemożliwe jest poprawne grupowanie. Tak samo dzieje się, gdy pogrupowane zostaną same środki granul, bez uwzględnienia rozmycia. Jak można odczytać z rys. 3.24, w takim przypadku większość punktów zostanie zaklasyfikowana do tego samego klastra.

Podsumowanie Choć wydawać by się mogło, że zwiększenie liczby granul, a tym samym liczby grupowanych danych, powinno poprawić jakość grupowania, wyniki klasteryzacji algorytmem GRAGGLO są lepsze, gdy liczba granul jest mniejsza – 50 lub 100.

Tabela 3.17: Dokładność grupowania hierarchicznego danych w zależności od liczby granul i łączności

liczba granul	dokładność	
	SINGLELINKAGE	COMPLETELINKAGE
50	0,9792	0,6768
100	0,9872	0,6606
200	0,9853	0,6317

Tabela 3.18: Trafność grupowania hierarchicznego danych w zależności od liczby granul i łączności

liczba granul	trafność	
	SINGLELINKAGE	COMPLETELINKAGE
50	0,9639	0,6270
100	0,9700	0,6161
200	0,9352	0,5937

Tabela 3.19: Precyzja grupowania hierarchicznego danych w zależności od liczby granul i łączności

liczba granul	precyzja	
	SINGLELINKAGE	COMPLETELINKAGE
50	0,9658	0,6062
100	0,9698	0,5859
200	0,9389	0,5460

Tabela 3.20: Dokładność grupowania hierarchicznego danych w zależności od typu relacji i łączności

typ relacji	dokładność	
	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0,9952	0,6513
wykładnicza	0,9780	0,6570
gaussowska	0,9785	0,6608

Tabela 3.21: Trafność grupowania hierarchicznego danych w zależności od typu relacji i łączności

typ relacji	trafność	
	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0,9776	0,6123
wykładnicza	0,9476	0,6117
gaussowska	0,9438	0,6130

Gdy klastry mają kulisty kształt ('blobs', 'laguna'), granule wyznaczone są w ich środku, a im jest ich więcej, tym bardziej zagęszczone są w środku klastrów, co może powodować błędy obliczania przynależności danych bardziej oddalonych od tego centrum. Dodatkowo, w przypadku badanych danych, dla większej liczby granul bardziej prawdopodobne było błędne wyznaczenie granul przez algorytm FCM.

Typu liczb rozmytych Średnią dokładność grupowania danych w zależności od typu relacji między granulami dla różnych łączności przedstawiono w tabeli 3.20, trafność – w tabeli 3.21, a precyzję – w tabeli 3.22. Wyniki w zależności od typu relacji są zbliżone, jednak najlepszą dokładność, trafność i precyzję odnotowano dla relacji trójkątnej i łączności SINGLELINKAGE. Wiąże się to ze skończonym nośnikiem liczb rozmytych. Jako że większość kształtów danych zawiera dysproporcje w liczebności, średnie wartości trafności i precyzji są niższe od odpowiadającym im wartościom dokładności.

Wyniki dla pomiarów, gdzie przynależność punktów do granul określono na podstawie

Tabela 3.22: Precyzja grupowania hierarchicznego danych w zależności od typu relacji i łączności

typ relacji	precyzja	
	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0,9762	0,5784
wykładnicza	0,9517	0,5790
gaussowska	0,9466	0,5807

Tabela 3.23: Dokładność grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa

typ relacji	dokładność			
	bez szumu		z szumem	
	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0,9965	0,6265	0,4669	0,2999
wykładnicza	0,9779	0,6570	0,9779	0,6570
gaussowska	0,9783	0,6607	0,9783	0,6607

Tabela 3.24: Trafność grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa

typ relacji	trafność			
	bez szumu		z szumem	
	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0,9671	0,5907	0,4461	0,2818
wykładnicza	0,9465	0,6092	0,9465	0,6092
gaussowska	0,9426	0,6116	0,9426	0,6116

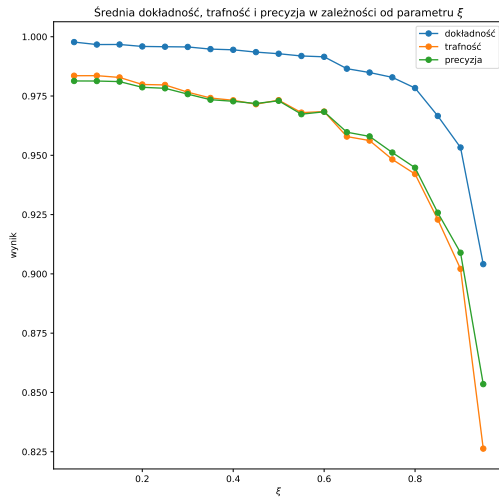
relacji *-podobieństwa przedstawiono w tabelach 3.23, 3.24 i 3.25. Dla relacji wykładniczej i gaussowskiej nie występuje szum, dlatego wyniki nie różnią się, gdy zostanie uwzględniony. Gdy szumu nie ma, lub nie jest uwzględniony, wartości metryk są bardzo zbliżone do metryk dla przynależności określonej na podstawie macierzy μ .

W przypadku relacji trójkątnej, szum stanowi przeciętnie około 47% danych – wyniki, gdy zostaje on uwzględniony, są niemal o połowę niższe.

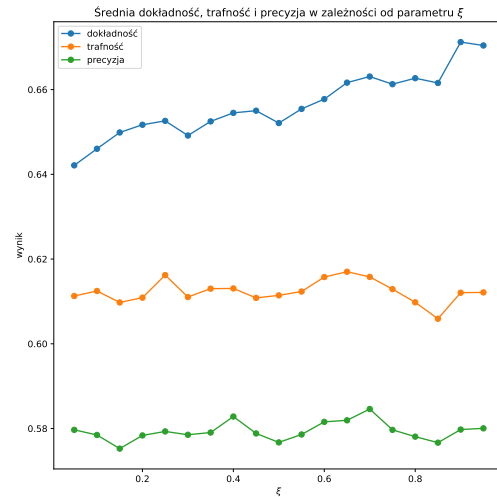
Podsumowanie Najwyższe wartość metryk jakości przy grupowaniu algorytmem SINGLELINKAGE uzyskuje się dla relacji trójkątnej, choć relacja wykładnicza i gaussowska również ma wysokie wyniki. Wybór typu relacji ma za to duże znaczenie, gdy na jej podstawie określa się przynależność punktów do granul – relacja trójkątna powoduje

Tabela 3.25: Precyzja grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa

typ relacji	precyzja			
	bez szumu		z szumem	
	SINGLELINKAGE	COMPLETELINKAGE	SINGLELINKAGE	COMPLETELINKAGE
trójkątna	0,9659	0,5635	0,4457	0,2721
wykładnicza	0,9507	0,5789	0,9507	0,5789
gaussowska	0,9454	0,5804	0,9454	0,5804



(a) SINGLELINKAGE



(b) COMPLETELINKAGE

Rysunek 3.25: Zależność dokładności, trafności i precyzji grupowania od wartości parametru ξ .

wtedy, że istotna część danych oryginalnych może nie zostać w ogóle przypisana do żadnej grupy. Dla danych, które zostały zaklasyfikowane, wartości metryk są za to wyższe, niż dla pozostałych relacji, dla których wszystkie dane są zgrupowane.

Parametr ξ Średnią dokładność, trafność i precyzję grupowania danych w zależności od parametru ξ przedstawiono na rys. 3.25.

Dla SINGLELINKAGE, najwyższą dokładność i precyzję – odpowiednio 99,7745% i 98,1348% – osiągnięto dla $\xi = 0,05$, najwyższą trafność – 98,3582% – dla $\xi = 0,1$. Wartości metryk spadają wraz ze wzrostem wartości parametru. Powyżej $\xi = 0,8$ następuje gwałtowne załamanie na wykresie wyników.

Zależność dokładności jest przeciwna dla COMPLETELINKAGE: najlepsze wyniki – 67,1215% – osiągnięto dla $\xi = 0,9$. Trafność i precyzja nie wpisują się w tę zależność – ich wartości wahają się odpowiednio około 60-61% i 58%. Najwyższa wartość trafności wynosi 61,7007% dla $\xi = 0,65$, a precyzji – 58,4605% dla $\xi = 0,7$.

Podsumowanie Eksperymenty sugerują, że dla łączności SINGLELINKAGE najlepiej używać małych wartości parametru $\xi \approx 0,1$, a dla COMPLETELINKAGE – dużych $\xi \approx 0,9$.

Kombinacja parametrów Wyniki pomiarów przeanalizowano też biorąc pod uwagę złożenie wpływu różnych parametrów – kształtu, liczby granul, typu relacji, parametru ξ – przy ich uśrednieniu dla wszystkich liczebności danych oryginalnych.

W tej sekcji zamieszczono wykresy złożonych zależności dla miary dokładności. W większości przypadków, trafność i precyzja posiadają na tyle podobne zależności, że nie

jest konieczne dodatkowe opisywanie ich – wnioski byłyby identyczne. Wszystkie wykresy zamieszczono w katalogu `img/complex`, pogrupowane w podkatalogach według kształtu danych ('blobs', 'circles', 'corners', 'crescents', 'laguna' i 'spheres'). Nazwa każdego pliku wskazuje na metrykę, łączność i sposób określenia przynależności danych do granul:

1. metryka: dokładność (`acc`), trafność (`rec`), precyzja (`prec`),
2. łączność: `SINGLELINKAGE (s1)`, `COMPLETELINKAGE (c1)`,
3. przynależność danych do granul: na podstawie macierzy przynależności μ (brak oznaczenia), na podstawie relacji *-podobieństwa (`shape_dep`).

'blobs' Jak pokazuje rys. 3.26, dla dokładności grupowania danych 'blobs' zdecydowanie największe znaczenie ma liczba granul. Najlepsze wyniki – około 100% niezależnie od innych parametrów – uzyskiwane są dla 50 granul. Dla takiej liczebności algorytm rozmytych c -średnich zazwyczaj tworzył granule we wszystkich trzech oczekiwanych klastrach.

Mniejsze znaczenie ma parametr ξ , jednak da się zauważyć spadek dokładności dla większych jego wartości dla `SINGLELINKAGE` i wzrost dokładności dla `COMPLETELINKAGE`, co jest zgodne z ogólną tendencją, przedstawioną na rys. 3.25. Typ relacji nie ma wyraźnego wpływu na jakość grupowania.

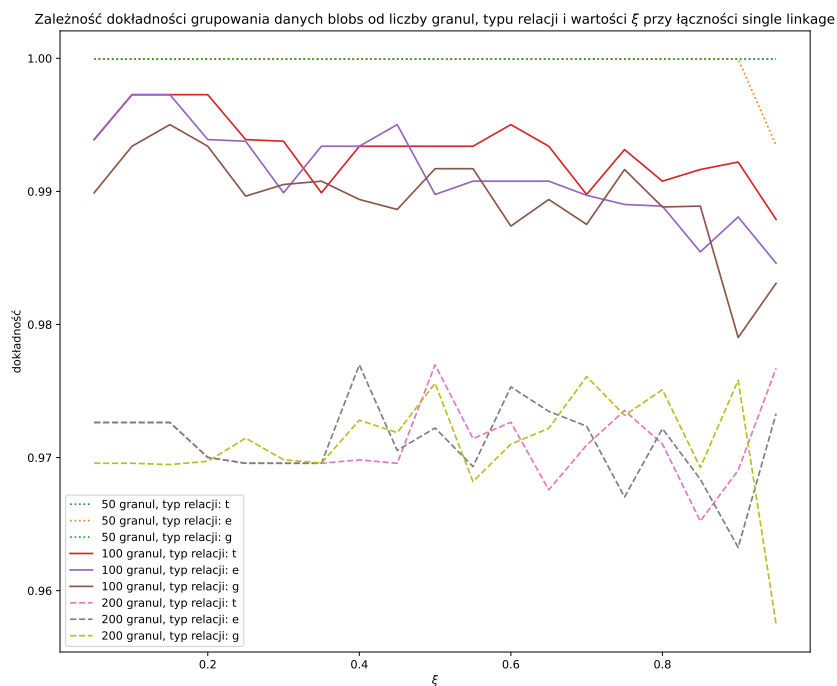
Jeżeli granule znalezione przez algorytm rozmytych c -średnich znajdują się we wszystkich trzech klastrach, to grupowanie `FUZZYAGGLO` przebiega poprawnie. W przeciwnym razie w centralnym klastrze, pojedyncze granule uznane zostaną za należące do innego klastra, jako że algorytm musi wyznaczyć trzy grupy (rys. 3.10). Parametr ξ i typ relacji mogą wtedy wpłynąć na to, ile punktów danych oryginalnych znajdzie się w tych nieprawidłowych klastrach, co prowadzi do fluktuacji dokładności.

'circles' Wykresy dla danych 'circles' (rys. 3.27) uwydatniają wpływ łączności na jakość grupowania.

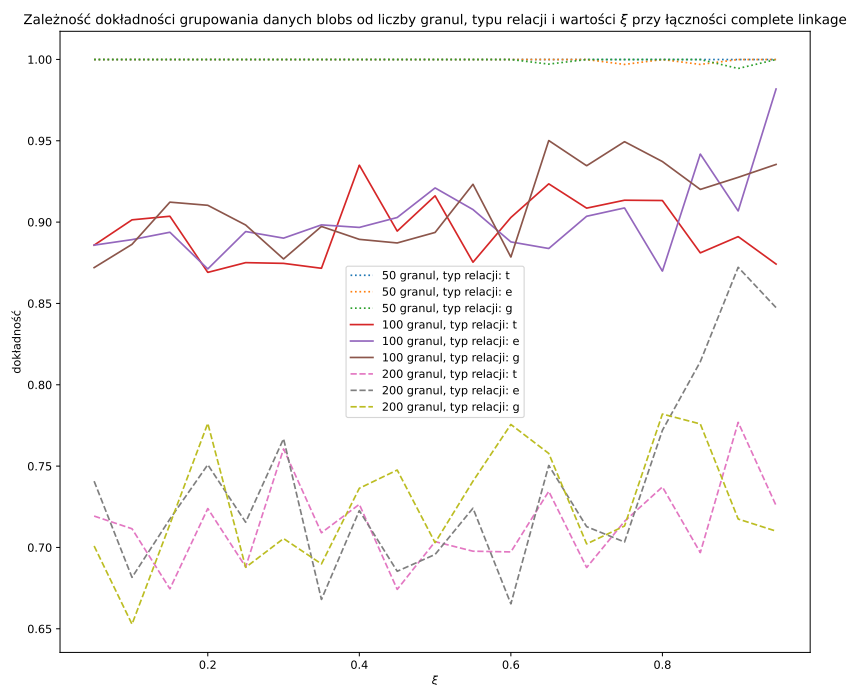
W przypadku łączności `SINGLELINKAGE`, dokładność jest niemal zawsze równa 100%. Spada ona dla wysokich ($\geq 0,9$) wartości parametru ξ . Najniższe wartości są osiąganiane dla relacji wykładniczej, wyższe dla gaussowskiej. Dla relacji trójkątnej, wyniki pozostają na poziomie 100% nawet dla $\xi = 0,95$.

Algorytm dla łączności `COMPLETELINKAGE` nie odnajduje poprawnie klastrów, co widać na rys. 3.19. Badane parametry nie mają znacznego wpływu na jakość.

'corners' Podobnie jak w przypadku 'circles', grupowanie danych 'corners' przy łączności `COMPLETELINKAGE` nie zwraca poprawnych wyników. Rys. 3.28 pokazuje, że przez to pozostałe parametry nie mają zauważalnego wpływu na jakość grupowania.

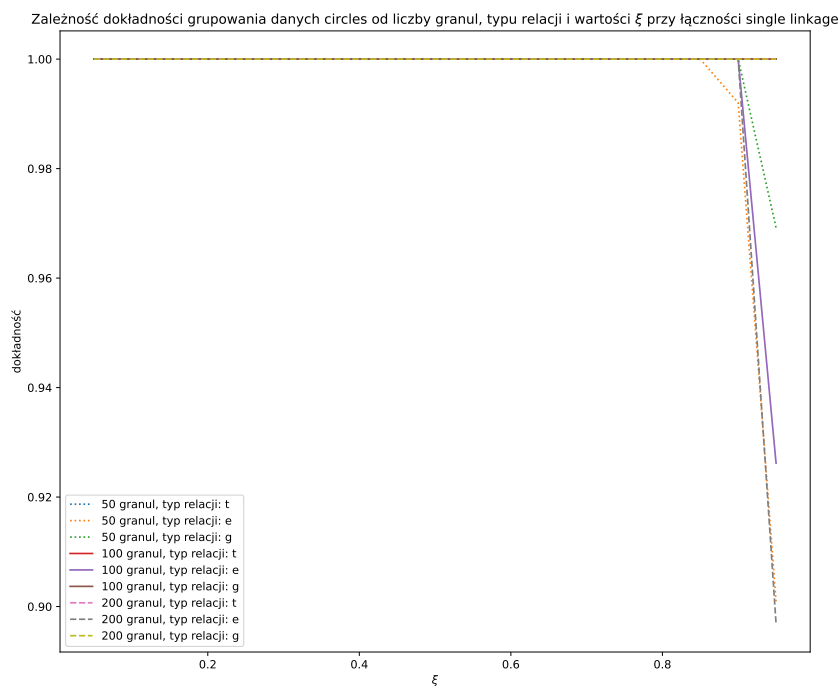


(a) SINGLELINKAGE

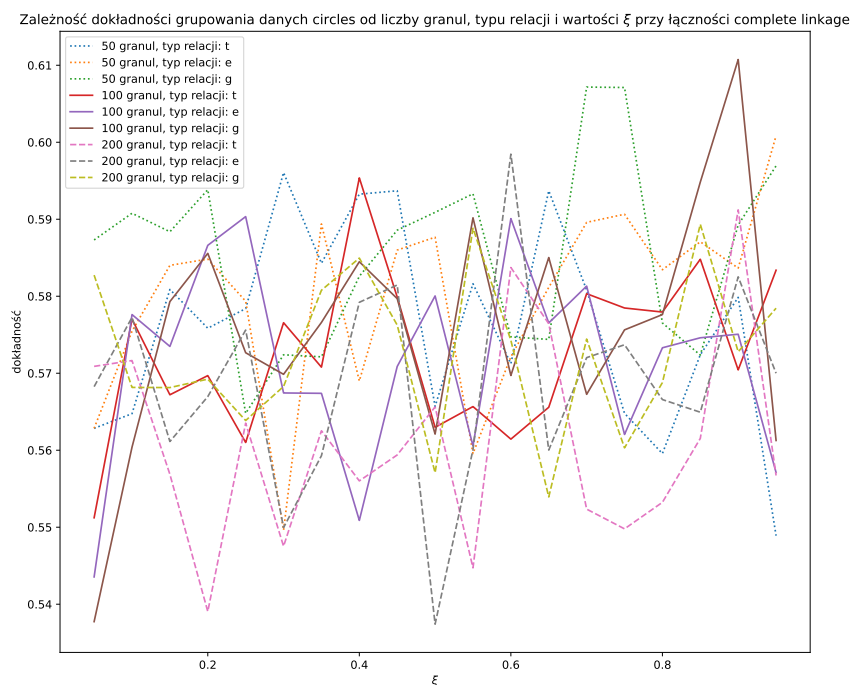


(b) COMPLETELINKAGE

Rysunek 3.26: Zależność dokładności grupowania danych ‘blobs’ od liczby granul, typu relacji i wartości parametru ξ .



(a) SINGLELINKAGE



(b) COMPLETELINKAGE

Rysunek 3.27: Zależność dokładności grupowania danych ‘circles’ od liczby granul, typu relacji i wartości parametru ξ .

Dla łączności SINGLELINKAGE największe znaczenie ma parametr ξ . Dokładność zmniejsza się wraz ze wzrostem wartości tego parametru. Ponadto znaczenie ma też liczba granul i typ relacji. Dokładność wcześniej zaczyna spadać dla mniejszej liczebności granul, najpierw dla relacji gaussowskiej, potem wykładniczej, na końcu trójkątnej. Dla relacji trójkątnej i większych liczebności granul – 100 i 200 – średnia dokładność zostaje na poziomie 100% nawet dla $\xi = 0,95$.

‘crescents’ Wnioski dla danych ‘crescents’ są bardzo podobne do wniosków dla danych ‘circles’. Jak pokazuje rys. 3.29, dla łączności SINGLELINKAGE średnia dokładność spada poniżej 100% dopiero dla wysokich ($\geq 0,9$) wartości ξ , najbardziej dla relacji wykładniczej. W przypadku COMPLETELINKAGE, również można zauważyć spadek dokładności dla tej relacji i wysokiego ξ , jednak nie ma jasnego wpływu badanych parametrów na jakość grupowania.

‘laguna’ Przy łączności SINGLELINKAGE dokładność spada wraz ze wzrostem wartości ξ – rys. 3.30. Najszybciej jakość zmniejsza się dla dużej liczebności granul – 200. Ma to związek z nieprawidłową przynależnością punktów do granul na zewnątrz dwóch wewnętrznych klastrów, jak widać na rys. 3.16. Tak jak w przypadku innych kształtów danych, najniższe wyniki są uzyskiwane dla relacji wykładniczej.

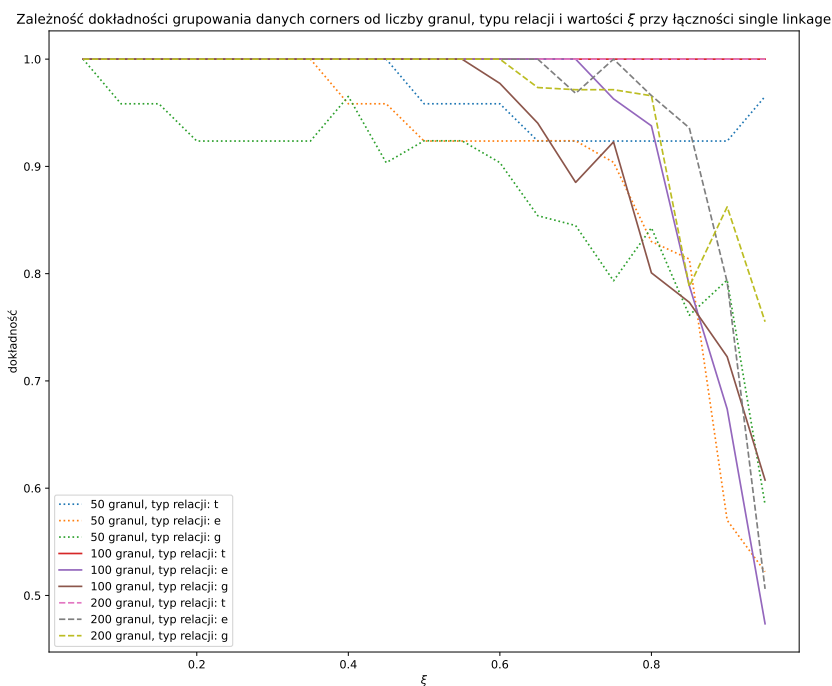
Obliczanie przynależności może też wpływać na wyniki dla COMPLETELINKAGE. Dla tej łączności znowu nie można jasno określić wpływu parametrów na dokładność grupowania, natomiast zauważyć można, że dla większych liczebności granul wyniki są bardziej zróżnicowane, natomiast dla 50 granul są one bliższe średniej.

‘spheres’ Wyniki dla danych ‘spheres’ (rys. 3.31) przedstawiają inne zależności od wcześniej opisanych danych.

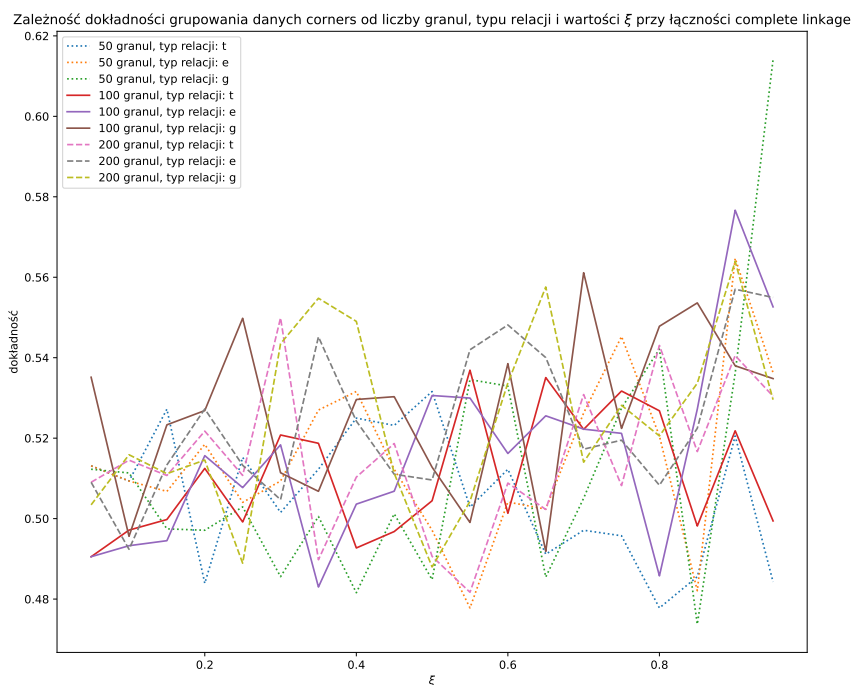
Przy łączności SINGLELINKAGE, dokładność zmniejsza się wraz ze wzrostem parametru ξ , jednak najgorsze wyniki otrzymywane są dla najmniejszej liczebności granul – 50. Gdy jest ich mniej, granule na zewnętrznej sferze mogą być znacznie oddalone od siebie nawzajem, przez co sfera wewnętrzna łączona jest z zewnętrzną, a pojedyncza granula ze sfery zewnętrznej zostaje uznana za osobny klaster (rys. 3.17).

Dla COMPLETELINKAGE można zauważyć wzrost średniej dokładności grupowania wraz ze wzrostem parametru ξ , szczególnie dla 50 granul.

Podsumowanie Łączność SINGLELINKAGE jest znacznie lepszym wyborem podczas grupowania badanych danych niż COMPLETELINKAGE. Dla tej łączności, niezależnie od kształtu danych, najlepsze wyniki uzyskuje się dla niskich wartości parametru progowego ξ . Najlepiej wykorzystywać też relację trójkątną – z nielicznymi wyjątkami przy danych

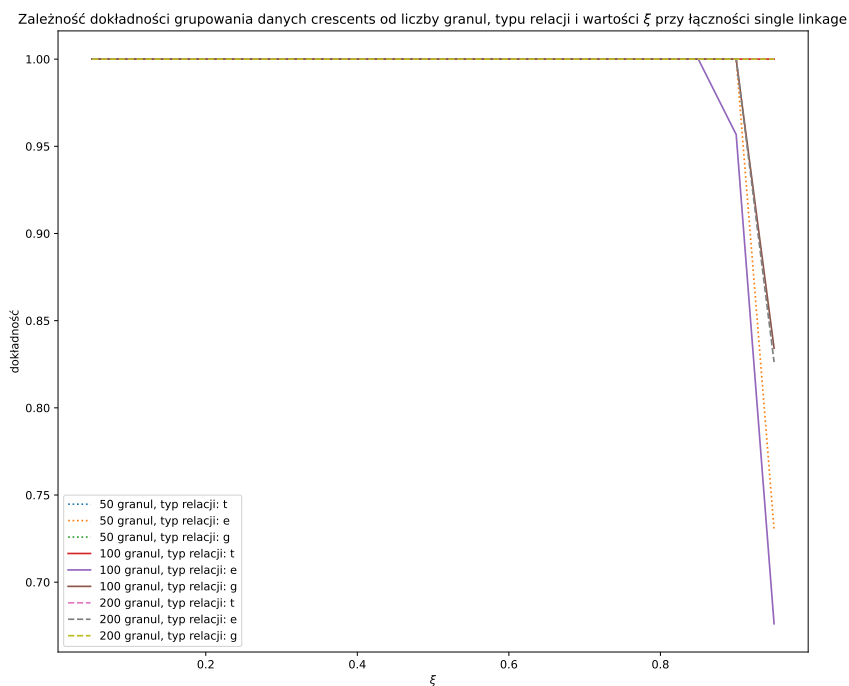


(a) SINGLELINKAGE

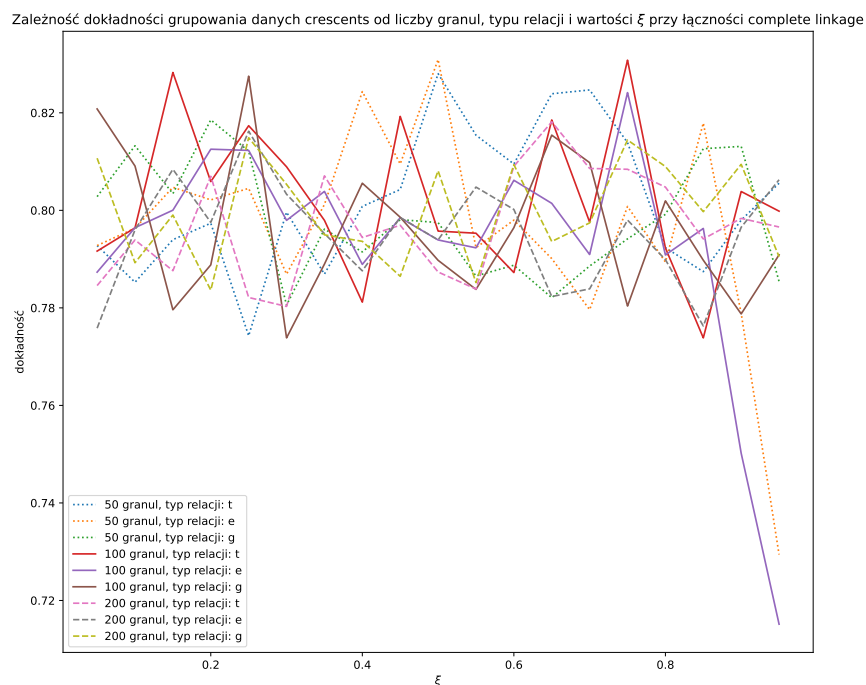


(b) COMPLETELINKAGE

Rysunek 3.28: Zależność dokładności grupowania danych ‘corners’ od liczby granul, typu relacji i wartości parametru ξ .

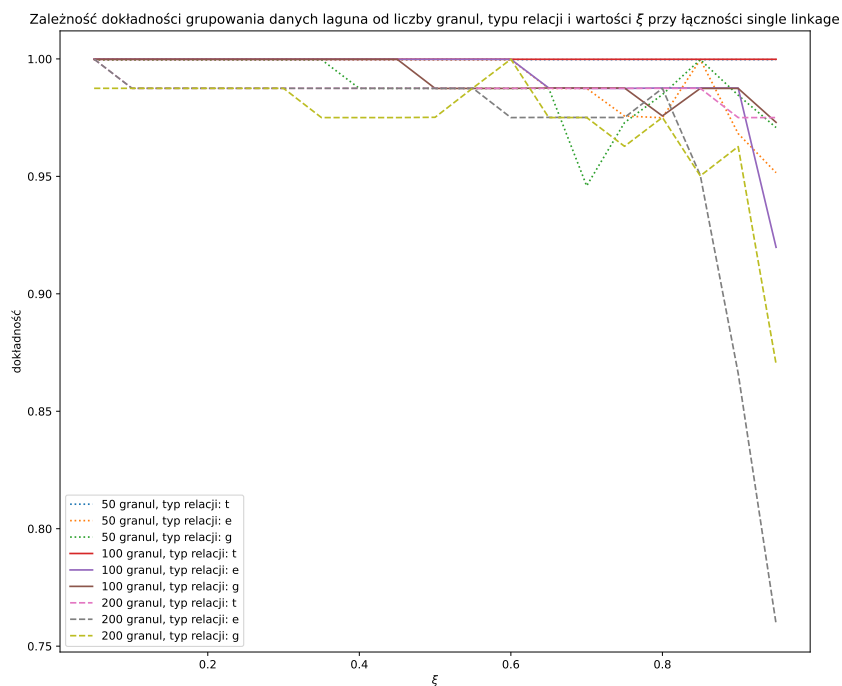


(a) SINGLELINKAGE

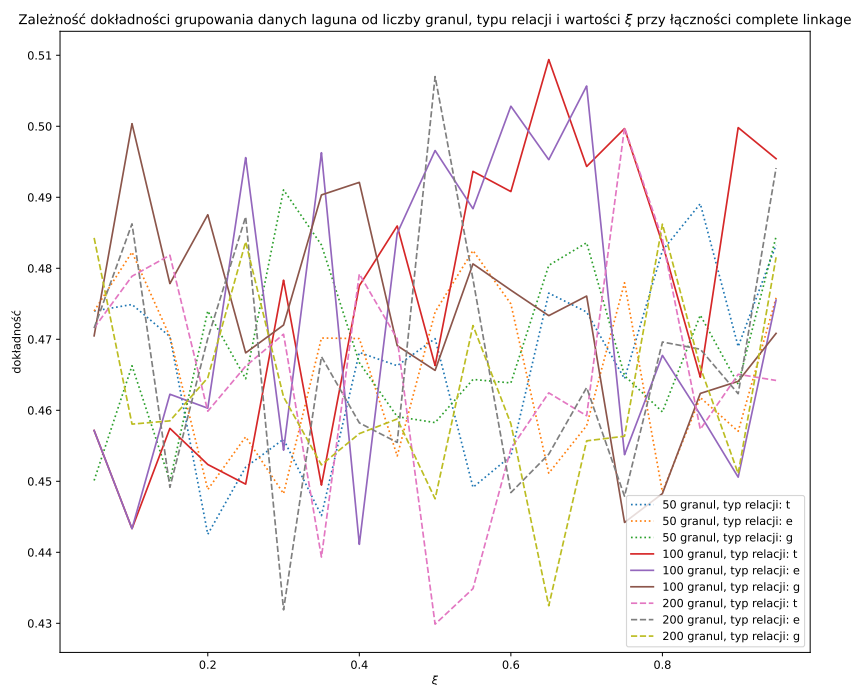


(b) COMPLETELINKAGE

Rysunek 3.29: Zależność dokładności grupowania danych ‘crescents’ od liczby granul, typu relacji i wartości parametru ξ .

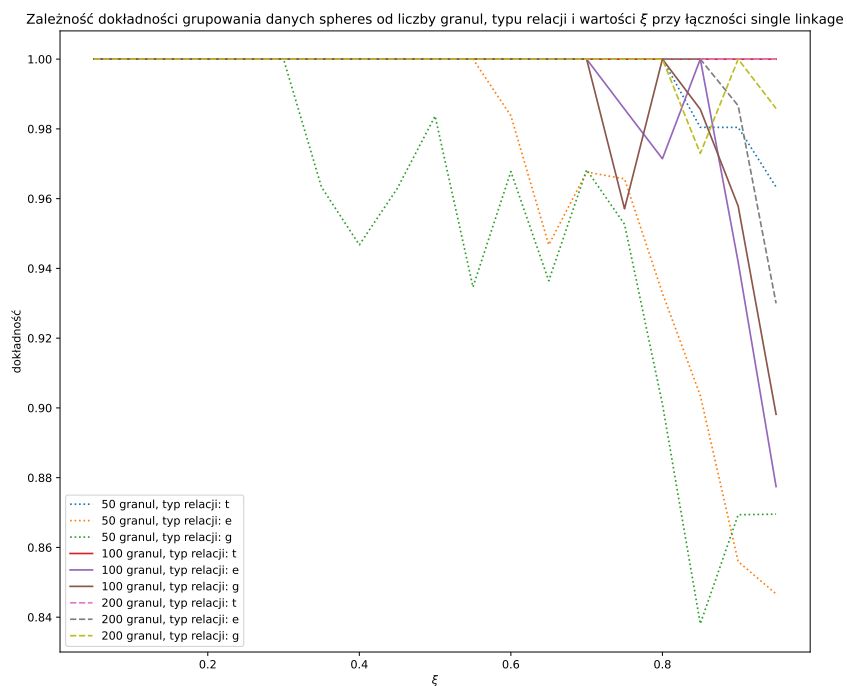


(a) SINGLELINKAGE

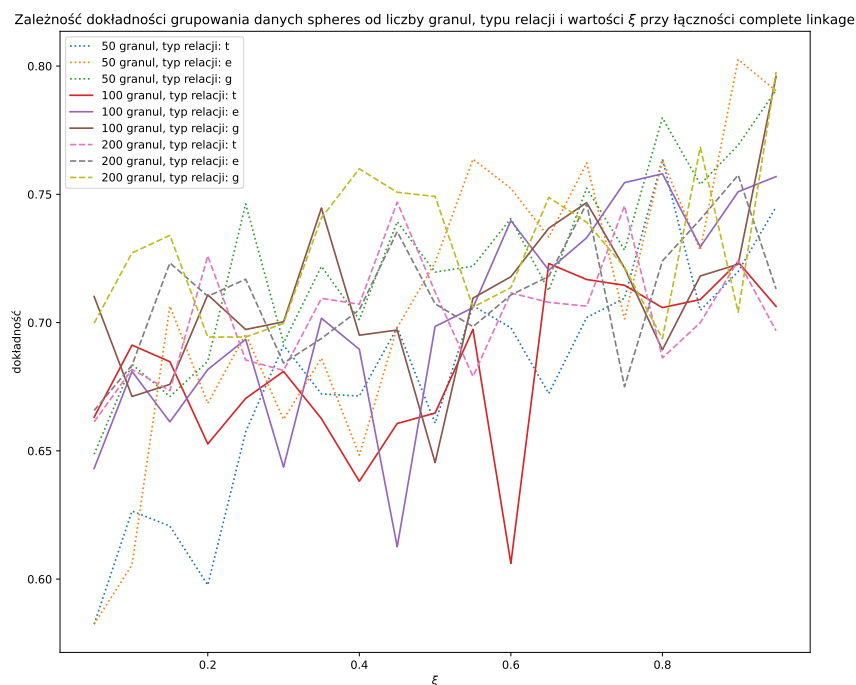


(b) COMPLETELINKAGE

Rysunek 3.30: Zależność dokładności grupowania danych ‘laguna’ od liczby granul, typu relacji i wartości parametru ξ .



(a) SINGLELINKAGE



(b) COMPLETELINKAGE

Rysunek 3.31: Zależność dokładności grupowania danych ‘spheres’ od liczby granul, typu relacji i wartości parametru ξ .

‘blobs’, wyniki są najwyższe dla tego kształtu granul, przy zwiększaniu parametru ξ jakość najpóźniej spada dla tej relacji.

Wnioski dotyczące optymalnej liczby granul zależą od kształtu danych. Gdy klastry danych układają się w kształt kulisty – ‘blobs’, ‘laguna’ – lepsze są małe liczby granul ($G = 50$), co poprawia jakość klasteryzacji algorytmem rozmytych c -średnich. Dla pozostałych badanych kształtów lepiej wybierać większe liczby, dzięki czemu granule rozkładają się bardziej równomiernie.

Przy grupowaniu danych algorytmem o łączności COMPLETELINKAGE, w większości przypadków nie widać jasnego wpływu parametrów na wyniki, za wyjątkiem grupowania danych ‘blobs’. Dla tych danych najlepsze wyniki uzyskiwane są dla małej liczby granul ($G = 50$), wysokiej wartości ξ oraz relacji wykładniczej.

Dane ‘brightkite’ i ‘gowalla’ Ze względu na brak etykiet danych ‘brightkite’ i ‘gowalla’, jakość grupowania można ocenić wizualnie, na podstawie wykresów – odpowiednio rys. 3.32 i 3.33.

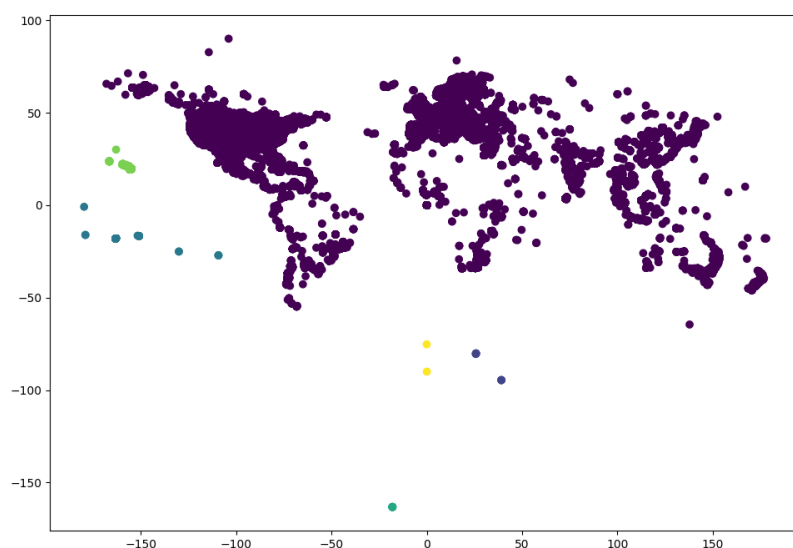
Podczas grupowania ostrego algorytmem `AgglomerativeClustering` wykorzystano losową próbkę 500 000 współrzędnych, jako że algorytm ten nie nadaje się do grupowania bardzo dużych ilości danych. Jako parametry grupowania GRAGGLO wybrano na podstawie wyników przedstawionych na stronie 37 i dalszych, relację trójkątną, 100 granul i $\xi = 0,1$. W obydwu przypadkach jako docelową liczbę klastrow wybrano 6 – jeden klaster to jeden kontynent.

Grupowanie ostre algorytmem `AgglomerativeClustering` nie odnajduje oczekiwanych klastrow. W przypadku danych ‘brightkite’, wszystkie kontynenty zostały połączone w jeden klaster, zaś elementy odstające – wyspy, ocean, błędne współrzędne – to pozostałe 5 klastrow. W przypadku danych ‘gowalla’ grupowanie jest lepsze – Ameryki stanowią osobny klaster niż reszta kontynentów – jednak wciąż 4 klastry odpowiadają wyłącznie danym odstającym.

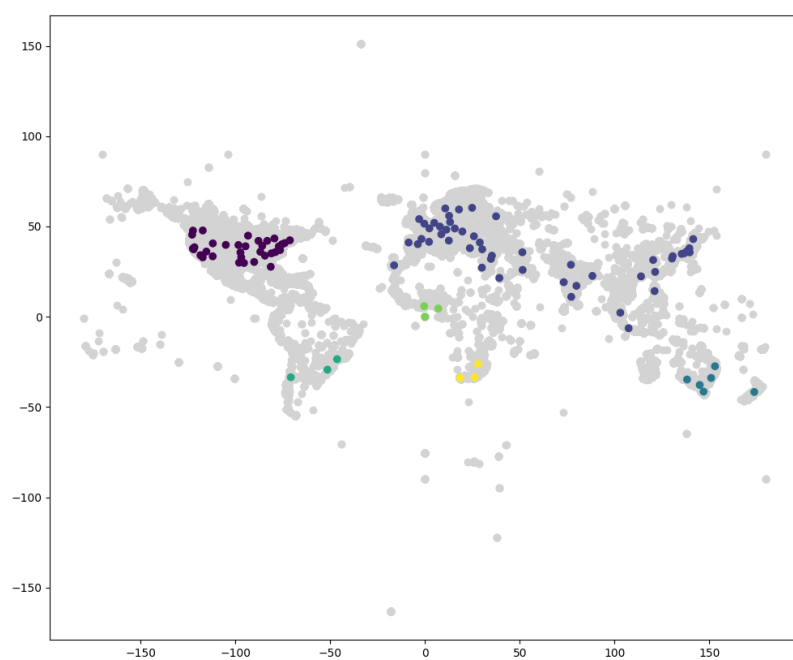
Wyniki grupowania algorytmem GRAGGLO są dużo bliższe oczekiwaniom. Wstępna granulacja FCM pozwoliła na eliminację szumów (w tym błędnych współrzędnych), na które wrażliwy jest algorytm grupowania hierarchicznego. Zauważyć można, że duży wpływ na rozmieszczenie granul ma zagęszczenie danych – najwięcej granul powstało we współrzędnych odpowiadających Europie i Ameryce Północnej. Zarówno w przypadku danych ‘brightkite’, jak i ‘gowalla’, Europa i Azja zostały uznane za jeden klaster.

3.6 Podsumowanie

Do analizy danych powszechnie wykorzystuje się algorytmy grupowania. Polegają one na wyznaczeniu takich grup danych, aby wszystkie dane przypisane do tej samej grupy były do siebie jak najbardziej podobne pod względem wybranej metryki, a dane przypisane

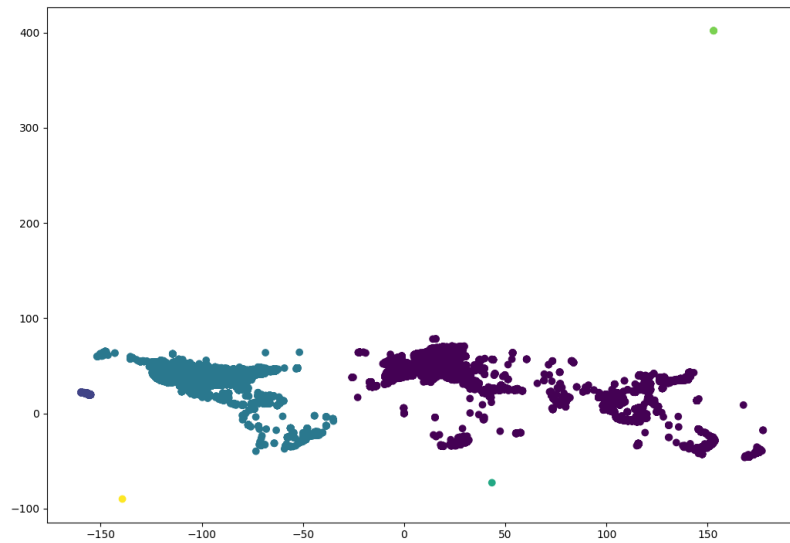


(a) Grupowanie ostre próbki losowej 500 000 punktów

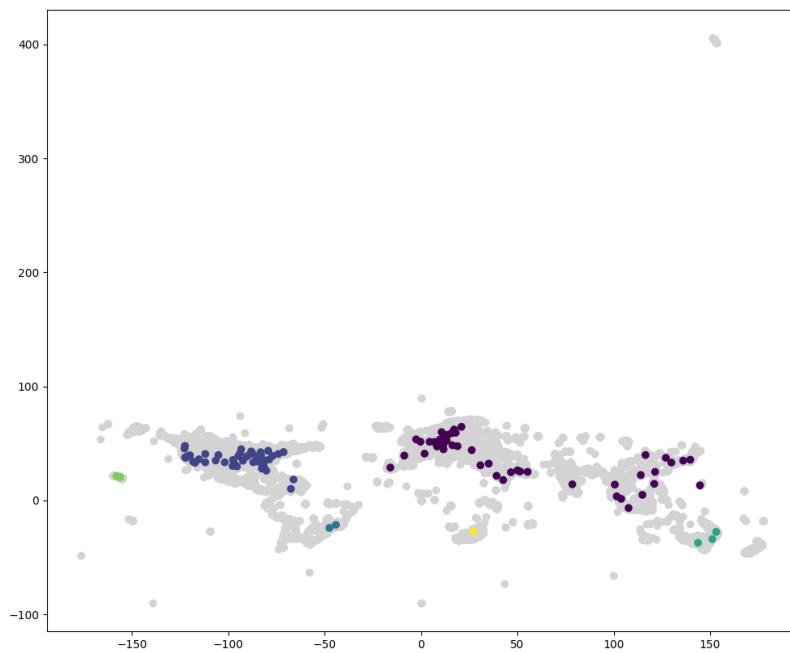


(b) Grupowanie algorytmem GRAGGLO całości danych

Rysunek 3.32: Wyniki grupowania hierarchicznego algorytmem ostrym **Agglomerative-Clustering** i algorytmem GRAGGLO danych ‘brightkite’



(a) Grupowanie ostre próbki losowej 500 000 punktów



(b) Grupowanie algorytmem GRAGGLO całości danych

Rysunek 3.33: Wyniki grupowania hierarchicznego algorytmem ostrym **Agglomerative-Clustering** i algorytmem GRAGGLO danych ‘gowalla’

sane do różnych grup – jak najbardziej niepodobne. Algorytmy takie mogą być na przykład oparte na minimalizacji pewnej funkcji kryterialnej bądź wykorzystywać zagęszczenie danych w pewnym obszarze do wyznaczenia klastrów. W pracy skupiono się na aglomeracyjnym algorytmie hierarchicznym, który polega na tworzeniu sekwencji zagnieżdżonych podziałów danych poprzez łączenie mniejszych klastrów w większe. Wadą tego algorytmu jest jego duża złożoność czasowa: sześcienna dla implementacji ogólnej. Choć istnieją specjalizowane implementacje o niższych złożonościach (kwadratowych), to pozostają one na tyle wysokie, że algorytm jest problematyczny do wykorzystania dla dużych liczebności danych.

W pracy do przyspieszenia działania aglomeracyjnego algorytmu hierarchicznego wykorzystano obliczenia granularne. Jest to dziedzina, która również stosowana jest do analizy danych. Polega na analizowaniu ich na sposób podobny do ludzkiego postrzegania rzeczywistości. Związany z nią pojęciem jest granula informacyjna, będąca encją o pewnej semantyce. W pracy granule realizowane są przy pomocy liczb rozmytych, czyli takich liczb, które modelują pewne ostre wartości liczbowe w jakimś przybliżeniu.

W pracy zaproponowany został autorski algorytm GRAGGLO, który wykonuje grupowanie hierarchiczne na granulach informacyjnych, zamiast na oryginalnych liczbach ostrych. Krokiem przygotowawczym algorytmu jest zgrupowanie danych wejściowych (współrzędnych punktów) w granule. Wykorzystano do tego algorytm rozmytych c -średnich FCM, który ma mniejszą – liniową – złożoność czasową niż algorytm hierarchiczny. Algorytm ten grupuje dane w klastry rozmyte, czyli takie, do których każdy punkt należy w pewnym stopniu, określonym jako liczba z przedziału $[0, 1]$. Następnie klastry te przekształcane są w granule: wielowymiarowe rozszerzalne liczby rozmyte, gdzie dla każdego wymiaru wyróżnić można środek i rozmycie granuli (ziaren informacyjnych). Wykorzystano trzy kształty ziaren: trójkątne, wykładnicze i gaussowskie, definiowane przez odpowiednie rozmyte relacje $*$ -podobieństwa. Właściwe aglomeracyjne grupowanie hierarchiczne algorytmu GRAGGLO wykorzystuje ogólną implementację o złożoności sześcienniej dostosowaną do operowania na granulach. Modyfikacje algorytmu w stosunku do standardowej, ostrej wersji, dotyczą operacji arytmetycznych i logicznych. Odległości euklidesowe między granulami również mają postać liczb rozmytych, które są następnie porównywane. Jako że wynikiem porównań liczb rozmytych jest wartość z przedziału $[0, 1]$, wprowadzony został dodatkowy parametr progowy ξ , określający, dla jakiej wartości operacja logiczna będzie ewaluowana jako prawda. Zaimplementowano dwie łączności grupowania hierarchicznego: SINGLELINKAGE oraz COMPLETELINKAGE.

W pracy przebadano własności zaproponowanego algorytmu – jego złożoność czasową oraz jakość grupowania. Porównano czas wykonania algorytmu GRAGGLO z czasem wykonania ostrego algorytmu hierarchicznego o analogicznej implementacji dla różnych liczebności danych. Odnotowano znaczne zmniejszenie czasu wykonania – przykładowo, dla 5000 danych algorytm GRAGGLO wykonywał się 1-4 sekundy (w zależności od liczby

granul), a algorytm ostry – około 100 sekund. Możliwe było też wykonanie algorytmu na bardzo dużych danych – kilku milionach punktów – w czasie rzędu kilkudziesięciu minut. Ponadto algorytm jest też szybszy od optymalnego algorytmu bibliotecznego powyżej pewnej liczby danych. Jego złożoność czasowa jest liniowa w zależności od liczby danych wejściowych – choć właściwe grupowanie hierarchiczne nadal ma złożoność sześcienną, to wykorzystuje ono granule informacyjne, których jest znacznie mniej niż danych wejściowych. Udało się więc osiągnąć główny cel pracy – znacznie zmniejszono czas wykonywania algorytmu hierarchicznego dla dużych liczebności danych. W zamian za przyspieszenie algorytmu dopuszczono pogorszenie jakości grupowania. Pomiarów dokładności, trafności i precyzji algorytmu GRAGGLO pokazały jednak, że w większości przypadków jego wyniki nie odbiegają od wyników algorytmu ostrego, a nawet mogą być lepsze. Algorytm badano dla różnych kombinacji parametrów: liczby granul, kształtu granul i parametru progowego ξ , dla obydwu zaimplementowanych łączności. Dla wszystkich parametrów, średnie wartości metryk jakości są zbliżone do wyników tych samych metryk dla algorytmu ostrego. Ponadto w większości przypadków można znaleźć taką kombinację tych parametrów, dla których wyniki są lepsze niż dla algorytmu ostrego. Analiza grupowania GRAGGLO pokazuje również, że granulacja danych wejściowych algorytmem rozmytych c -średnich pozwala na wyeliminowanie z danych elementów odstających, na które bardzo wrażliwy jest algorytm hierarchiczny.

Implementację algorytmu GRAGGLO oraz skrypty wykorzystane do badań jego działania zamieszczono w repozytorium pod adresem <https://github.com/ZofiaLor/magister-granule>.

Algorytm GRAGGLO może w prosty sposób zostać rozszerzony o inne metryki odległości. Dzięki zastosowaniu ogólnej implementacji, łatwo można też dodać inne opcje łączności, takie jak łączność średnia lub Warda. Ponadto implementację granul rozszerzyć można o inne kształty ziaren poprzez zastosowanie innych rozmytych relacji $*$ -podobieństwa.

Bibliografia

- [1] Rakesh Agrawal, Johannes Gehrke, Dimitrios Gunopulos i Prabhakar Raghavan. „Automatic subspace clustering of high dimensional data for data mining applications”. W: *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*. 1998, 94—105.
- [2] Sepehr Akbarzadeh. *Fuzzy C-Means — scikit-fuzzy-c-means "0.1.0" documentation*. 2024. URL: https://scikit-fuzzy-c-means.readthedocs.io/en/latest/fuzzy_c_means.html (term. wiz. 28.03.2026).
- [3] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel i Jörg Sander. „OPTICS: ordering points to identify the clustering structure”. W: *Proceedings on 1999 ACM SIGMOD international conference on management of data*. 1999, s. 49–60.
- [4] Barnabas Bede. *Mathematics of Fuzzy Sets and Fuzzy Logic*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013. ISBN: 978-3-642-35221-8.
- [5] *Comparing different hierarchical linkage methods on toy datasets — scikit-learn 1.8.0 documentation*. 2026. URL: https://scikit-learn.org/stable/auto_examples/cluster/plot_linkage_comparison.html (term. wiz. 04.04.2026).
- [6] J. Leskovec E. Cho S. A. Myers. *SNAP: Network datasets: Brightkite*. 2011. URL: <https://snap.stanford.edu/data/loc-brightkite.html> (term. wiz. 24.02.2026).
- [7] J. Leskovec E. Cho S. A. Myers. *SNAP: Network datasets: Gowalla*. 2011. URL: <https://snap.stanford.edu/data/loc-gowalla.html> (term. wiz. 24.02.2026).
- [8] Martin Ester, Hans-Peter Kriegel, Jörg Sander i Xiaowei Xu. „A density-based algorithm for discovering clusters in large spatial databases with noise”. W: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. 1996, s. 226–231.
- [9] Soumi Ghosh i Sanjay Kumar. „Comparative Analysis of K-Means and Fuzzy C-Means Algorithms”. W: *International Journal of Advanced Computer Science and Applications* 4 (2013), s. 35–39.
- [10] Seyed Emadedin Hashemi, Fatemeh Gholian-Jouybari i Mostafa Hajiaghaei-Keshteli. „A fuzzy C-means algorithm for optimizing data clustering”. W: *Expert Systems with Applications* 227 (2023), s. 120377.

- [11] M. Jezewski, R. Czabanski i J. Leski. „Introduction to Fuzzy Sets”. W: *Theory and Applications of Ordered Fuzzy Numbers* (2017), s. 3–22.
- [12] Joe H. Ward Jr. „Hierarchical Grouping to Optimize an Objective Function”. W: *Journal of the American Statistical Association* 58.301 (1963), s. 236–244.
- [13] R. Krishnapuram i J.M. Keller. „A possibilistic approach to clustering”. W: *IEEE Transactions on Fuzzy Systems* 1.2 (1993), s. 98–110.
- [14] *Lasso Selector — Matplotlib 3.10.9 documentation*. 2026. URL: https://matplotlib.org/stable/gallery/widgets/lasso_selector_demo_sgskip.html (term. wiz. 29.05.2026).
- [15] Jacek M. Leski. „Fuzzy c-ordered-means clustering”. W: *Fuzzy Sets and Systems* 286 (2016), s. 114–133.
- [16] *linkage – SciPy v1.17.0 Manual*. 2026. URL: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.linkage.html> (term. wiz. 25.03.2026).
- [17] P. Macnaughton-Smith, W. T. Williams, M. B. Dale i Mockett L. G. „Dissimilarity Analysis: a new Technique of Hierarchical Sub-division”. W: *Nature* 202 (1964), s. 1034–1035.
- [18] Daniel Müllner. „Modern hierarchical, agglomerative clustering algorithms”. W: *ArXiv* (2011). arXiv:1109.2378. DOI: 10.48550/arXiv.1109.2378.
- [19] Fionn Murtagh i Pedro Contreras. „Algorithms for hierarchical clustering: an overview”. W: *WIREs Data Mining and Knowledge Discovery* 2.1 (2012), s. 86–97.
- [20] Witold Pedrycz, Giancarlo Succi, Alberto Sillitti i Joana Iljazi. „Data description: A general framework of information granules”. W: *Knowledge-Based Systems* 80 (2015), s. 98–108.
- [21] *Recommended Use — skfuzzy v0.4.2 docs*. 2026. URL: <https://scikit-fuzzy.github.io/scikit-fuzzy/api/index.html> (term. wiz. 28.03.2026).
- [22] R. Sibson. „SLINK: An optimally efficient algorithm for the single-link cluster method”. W: *The Computer Journal* 16.1 (1973), s. 30–34.
- [23] Md. Abu Siddique, Rezoana Bente Arif, Mohammad Mahmudur Rahman Khan i Zahidun Ashrafi. „Implementation of Fuzzy C-Means and Possibilistic C-Means Clustering Algorithms, Cluster Tendency Analysis and Cluster Validation”. W: *ArXiv* (2018). arXiv:1809.08417. DOI: 10.48550/arXiv.1809.08417.
- [24] Krzysztof Siminski. „Arithmetic with extensional fuzzy numbers”. W: *Fuzzy Sets and Systems* 544 (2026), s. 110067. ISSN: 0165-0114. DOI: 10.1016/j.fss.2026.110067.

-
- [25] Martin Stepnicka i Michal Holcapek. „Arithmetics of Extensional Fuzzy Numbers - Part I: Introduction”. W: *IEEE International Conference on Fuzzy Systems*. 2013, s. 1–8.
 - [26] Dan Wang, Peng Nie, Zhu Xiubin, Witold Pedrycz i Zhiwu Li. „Designing of higher order information granules through clustering heterogeneous granular data”. W: *Applied Soft Computing* 112 (2021), s. 107820.
 - [27] Dongkuan Xu i Yingjie Tian. „A Comprehensive Survey of Clustering Algorithms”. W: *Annals of Data Science* 2 (2015), s. 165–193.
 - [28] R. Xu i D.C. Wunsch. *Hierarchical Clustering*. John Wiley & Sons, Ltd, 2008. ISBN: 9780470382776.
 - [29] Xiaowei Xu, M. Ester, H.-P. Kriegel i J. Sander. „A distribution-based clustering algorithm for mining in large spatial databases”. W: *Proceedings 14th International Conference on Data Engineering*. 1998, s. 324–331.
 - [30] Tian Zhang, Raghu Ramakrishnan i Miron Livny. „BIRCH: an efficient data clustering method for very large databases”. W: *SIGMOD Rec.* 25.1 (1996), s. 103–114.

Dodatki

Dokumentacja techniczna

Implementacja

Całość projektu zaimplementowano w języku Python¹ w wersji 3.12. Jest to jeden z najczęściej wykorzystywanych języków programowania, szczególnie w projektach związanych z analizą danych. Język posiada wiele bibliotek, które implementują większość potrzebnych funkcjonalności.

Biblioteka `scikit-fuzzy-c-means` dostarcza implementację algorytmu rozmytych *c*-średnich. Środki klastrów inicjalizowane są metodą *K-Means++*. Inną biblioteką, która również dostarcza tej implementacji, jest `scikit-fuzzy`. Oprócz tego, posiada ona bardziej rozwinięte API, umożliwiające wykonywanie różnych operacji na zbiorach rozmytych. Działanie algorytmu rozmytych *c*-średnich jest identyczne dla obydwu bibliotek, ostatecznie wybrano prostszą bibliotekę `scikit-fuzzy-c-means`.

Pozostałymi wykorzystanymi w projekcie bibliotekami są: `matplotlib` (wykresy danych), `scikit-learn` (aglomeracyjny algorytm hierarchiczny), `scipy` (dendrogram), `numpy` (obliczenia macierzowe, wykorzystywana też przez `scikit-learn` i `scipy`) i `pandas` (ramki danych).

Implementację algorytmu GRAGGLO oraz skrypty wykorzystane do badań jego działania zamieszczono w repozytorium pod adresem <https://github.com/ZofiaLor/magister-granule>.

Uruchamianie

Do badań przedstawionych w rozdziale 3.5 wykorzystano skrypty pomocnicze.

Pomiar czasu

Pomiaru czasu można dokonać przy pomocy skryptu `measure_time.py`. Dla każdej liczebności danych wykonywana jest określona parametrem liczba pomiarów czasu wykonania algorytmu GRAGGLO oraz algorytmu ostrego, po czym obliczana jest ich średnia arytmetyczna. Wyniki mają postać ramki danych zawierającej pomiary w milisekundach oraz wykresu ilustrującego zależność czasu wykonania algorytmów od liczby danych.

¹<https://www.python.org/>, dostęp 26.04.2026

Wywołanie skryptu z linii poleceń ma następującą składnię:

```
python measure_time.py [-h] [-r REPEATS] [-l] [-t] [-p] [-f {blobs,circles,corners,crescents,laguna,spheres}],
```

gdzie:

- `-h` (alternatywnie: `--help`) – wyświetlenie pomocy w języku angielskim i zamknięcie programu,
- `-r REPEATS` (alternatywnie: `--repeats REPEATS`) – parametr (liczba całkowita) odpowiadający liczbie powtórzeń każdego z pomiarów czasu; domyślnie wartość 4,
- `-l` (alternatywnie: `--library_function`) – użycie funkcji `AgglomerativeClustering` biblioteki `scikit-learn` zamiast własnej implementacji ostrego algorytmu hierarchicznego, porównywanego z algorytmem GRAGGLO. Pomiary zostaną wtedy wykonane na większych liczebnościach danych,
- `-t` (alternatywnie: `--save_times`) – zapisanie wyników do pliku `.csv`, wywołanie bez przełącznika powoduje wypisanie pomiarów do konsoli,
- `-p` (alternatywnie: `--save_plot`) – zapisanie wykresu do pliku `.pdf`, wywołanie bez przełącznika powoduje wyświetlenie wykresu na ekranie,
- `-f {blobs,circles,corners,crescents,laguna,spheres}` (alternatywnie: `--filename {blobs,circles,corners,crescents,laguna,spheres}`) – wybór danych do pomiaru; domyślnie wartość `'corners'`.

Pomiar jakości

Pomiaru jakości grupowania można dokonać przy pomocy skryptu `measure_quality.py`. Dla wybranych lub wszystkich danych mierzona jest dokładność, trafność i precyzja grupowania. Pomiary wykonywane są dla różnych liczebności granul, typów relacji i wartości parametru ξ . Gdy mierzone są wszystkie dane, wyniki zapisywane są w pliku `.csv`, w przeciwnym razie wypisywane są w konsoli.

Wywołanie skryptu z linii poleceń ma następującą składnię:

```
python measure_quality.py [-h] [-f FILENAME] [-l {sl,single,cl,complete}] [-r] [--strict],
```

gdzie:

- `-h` (alternatywnie: `--help`) – wyświetlenie pomocy w języku angielskim i zamknięcie programu,
- `-f FILENAME` (alternatywnie: `--filename FILENAME`) – wybór konkretnego pliku z danymi do pomiaru (na przykład „blobs1000” – dane ‘blobs’ o rozmiarze 1000).

Jeżeli konkretny plik nie zostanie podany, wykonywane są pomiary na wszystkich plikach,

- `-l {sl,single,cl,complete}` (alternatywnie: `--linkage {sl,single,cl,complete}`) – łączność grupowania hierarchicznego, gdzie „sl” i „single” to łączność SINGLELINKAGE, a „cl” i „complete” – COMPLETELINKAGE; domyślnie wartość „single”,
- `-r` (alternatywnie: `--relation_dependent`) – obliczanie przynależności punktów do granul na podstawie relacji *-podobieństwa S , zamiast na podstawie macierzy przynależności μ ,
- `--strict` – dodatkowe pomiary metryk jakości dla grupowania ostrego algorytmem AgglomerativeClustering biblioteki scikit-learn.

Wykresy grupowania granul

Wykresy wyników grupowania GRAGGLO można uzyskać przy pomocy skryptu `plot_granules.py`. Każdy wykres składa się z 9 podwykresów, zawierających wyniki dla różnych liczebności granul i wartości parametru ξ . Dla każdego pliku tworzone są 3 takie wykresy, dla każdego typu relacji.

Wywołanie skryptu z linii poleceń ma następującą składnię:

```
python plot_granules.py [-h] [-f FILENAME] [-l {single,complete}] [-p],
```

gdzie:

- `-h` (alternatywnie: `--help`) – wyświetlenie pomocy w języku angielskim i zamknięcie programu,
- `-f FILENAME` (alternatywnie: `--filename FILENAME`) – wybór konkretnego pliku z danymi do zgrupowania i przedstawienia na wykresie (na przykład „blobs1000” – dane ‘blobs’ o rozmiarze 1000). Jeżeli konkretny plik nie zostanie podany, wykresy tworzone są dla wszystkich plików,
- `-l {single,complete}` (alternatywnie: `--linkage {single,complete}`) – łączność grupowania hierarchicznego, gdzie „single” to łączność SINGLELINKAGE, a „complete” – COMPLETELINKAGE; domyślnie wartość „single”,
- `-p` (alternatywnie: `--save_plot`) – zapisanie wykresów do pliku .pdf, wywołanie bez przełącznika powoduje wyświetlenie wykresów na ekranie, jeżeli są one tworzone tylko dla jednego pliku. Gdy wykresy generowane są dla wszystkich plików, są one zapisywane do .pdf niezależnie od obecności przełącznika.

Analiza wyników

Skrypt `accuracy_analysis.py` służy do analizy wyników pomiarów wykonanych przy pomocy skryptu `measure_quality.py`. Do jego działania wymagane jest więc uprzednie wykonanie pomiarów dla wszystkich plików, przy wybranej konfiguracji łączności i obliczania przynależności punktów do granul. Możliwa jest analiza wpływu poszczególnych parametrów grupowania na wyniki.

Wywołanie skryptu z linii poleceń ma następującą składnię:

```
python accuracy_analysis.py [-h] [-l {single,complete}] [-s] [-d] [-r] [-g] [-x] [-c] [--strict],
```

gdzie:

- `-h` (alternatywnie: `--help`) – wyświetlenie pomocy w języku angielskim i zamknięcie programu,
- `-l {single,complete}` (alternatywnie: `--linkage {single,complete}`) – analiza wyników grupowania przy danej łączności, gdzie „single” to łączność `SINGLELINKAGE`, a „complete” – `COMPLETELINKAGE`; domyślnie wartość „single”,
- `-s` (alternatywnie: `--shape_dependent`) – analiza wyników grupowania, dla których przynależność punktów do granul obliczona była na podstawie relacji $*$ -podobieństwa S , zamiast na podstawie macierzy przynależności μ (w wynikach występować może szum),
- `-d` (alternatywnie: `--data_shape`) – analiza wpływu kształtu danych (na przykład ‘blobs’, ‘circles’) na jakość grupowania,
- `-r` (alternatywnie: `--relation_type`) – analiza wpływu typu relacji (trójkątna, wykładnicza, gaussowska) na jakość grupowania,
- `-g` (alternatywnie: `--granule_number`) – analiza wpływu liczby granul generowanych przez algorytm FCM na jakość grupowania,
- `-x` (alternatywnie: `--xi`) – analiza wpływu parametru progowego ξ na jakość grupowania,
- `-c` (alternatywnie: `--complex`) – analiza wpływu kombinacji parametrów (kształtu danych, typu relacji, liczby granul oraz parametru ξ) na jakość grupowania,
- `--strict` – dodatkowa analiza wyników jakości dla grupowania ostrego algorytmem `AgglomerativeClustering` biblioteki `scikit-learn`.

Wykresy przynależności

Skrypt `membership_plots.py` pozwala na utworzenie wykresów przynależności punktów do granul, na przykład 3.24.

Wywołanie skryptu z linii poleceń ma następującą składnię:

```
python membership_plots.py [-h] [-n NUM_GRANULES] filename,
```

gdzie:

- `-h` (alternatywnie: `--help`) – wyświetlenie pomocy w języku angielskim i zamknięcie programu,
- `-n NUM_GRANULES` (alternatywnie: `--num_granules NUM_GRANULES`) – liczba granul generowanych przez algorytm FCM; domyślnie 100,
- `filename` – nazwa pliku z danymi do zgrupowania i przedstawienia na wykresie (na przykład „blobs1000” – dane ‘blobs’ o rozmiarze 1000).

Analiza dużych danych

Skrypt `large_data_analysis.py` przeznaczony jest do analizy dwuwymiarowych danych o dużych rozmiarach (w ramach pracy wykorzystano go dla danych ‘brightkite’ i ‘gowalla’). Pozwala on na pomiar czasu wykonania algorytmu GRAGGLO przy określonym kroku oraz na przedstawienie na wykresie wyników grupowania danych przez algorytm GRAGGLO oraz przez algorytm ostry `AgglomerativeClustering` biblioteki `scikit-learn`.

Wywołanie skryptu z linii poleceń ma następującą składnię:

```
python large_data_analysis.py [-h] [-c N_CLUSTERS] [-s SIZE] [-f] {time,plot}  
filename,
```

gdzie:

- `-h` (alternatywnie: `--help`) – wyświetlenie pomocy w języku angielskim i zamknięcie programu,
- `-n NUM_CLUSTERS` (alternatywnie: `--num_clusters NUM_CLUSTERS`) – liczba klastrów do wyznaczenia przez algorytm hierarchiczny; domyślnie 6,
- `-s SIZE` (alternatywnie: `--size SIZE` lub `--step SIZE`) – w zależności od wyboru trybu rozmiar kroku kolejnych pomiarów czasu lub rozmiar danych do zgrupowania i przedstawienia na wykresie dla algorytmu ostrego,
- `-f` (alternatywnie: `--flip_coordinates`) – zamiana kolejnością wymiarów danych na potrzeby przedstawienia ich na wykresie,
- `filename` – ścieżka do pliku z danymi,

- `{time,plot}` – wybór trybu działania skryptu („time”: pomiar czasu, „plot”: wykresy wyników grupowania).

Spis skrótów i symboli

a atrybut (wymiar) danych

A dokładność (ang. *accuracy*)

c klaster

d odległość między danymi

FCM algorytm rozmytych c -średnich (*fuzzy c-means*)

g granula, ziarno informacyjne

$\mathbf{L}$ macierz pomyłek

P precyzja (ang. *precision*)

p rozmycie rozszerzalnej liczby rozmytej

R trafność, czułość (ang. *recall*)

S rozmyta relacja *-podobieństwa

$\mathbf{V}$ macierz środków klastrów FCM

μ macierz stopni przyleżności do zbioru rozmytego

ξ parametr progowy rozmytego grupowania hierarchicznego

Spis rysunków

2.1	Dendrogram otrzymany na podstawie grupowania hierarchicznego o łączności SINGLELINKAGE przykładowego zbioru danych ‘iris’.	4
2.2	Ilustracja działania algorytmu SINGLELINKAGE. Odległość klastra B od klastrow A_1 i A_2 jest znana. Po złączeniu A_1 i A_2 w klastrow A , nowa odległość od klastra B jest równa minimalnej z poprzednich dwóch – w tym przypadku jest to odległość $d(A_2, B)$.	7
2.3	Ilustracja wpływu danych odstających na działanie algorytmu SINGLELINKAGE.	7
2.4	Ilustracja działania algorytmu SINGLELINKAGE, gdy klastry są zbyt zbliżone.	8
2.5	Ilustracja działania algorytmu COMPLETELINKAGE, dla zbliżonych, gęstych klastrow.	9
2.6	Ilustracja działania algorytmu COMPLETELINKAGE, dla klastrow w kształcie dwóch okręgów.	10
3.1	Wykres trapezowego zbioru rozmytego, ilustrujący jego rdzeń i nośnik.	14
3.2	Wykresy danych o różnych kształtach z folderu dane o rozmiarze 1000 punktów.	22
3.3	Wykresy danych ‘brightkite’ i ‘gowalla’.	22
3.4	Wykresy danych o różnych kształtach z folderu dane_labelled o rozmiarze 1000 punktów, o etykietach oznaczonych kolorami.	27
3.5	Wykres danych ‘laguna’ z folderu dane_labelled o rozmiarze 20 000 punktów, o etykietach oznaczonych kolorami.	28
3.6	Przynależność punktów do granul w zależności od kształtu liczb rozmytych. 10 środków granul oznaczono kolorem czerwonym, punkty należące do różnych granul oznaczone są różnymi kolorami. Kolorem szarym oznaczono punkty stanowiące szum.	31
3.7	Charakterystyka czasu wykonania własnych implementacji algorytmów ostrego i rozmytego dla różnej liczby granul od liczby danych wejściowych.	34
3.8	Charakterystyka czasu wykonania algorytmu ostrego AgglomerativeClustering biblioteki scikit-learn i własnej implementacji algorytmu rozmytego dla różnej liczby granul od liczby danych wejściowych.	35

3.9	Charakterystyka czasu wykonania algorytmu rozmytego dla różnej liczby granul od liczby danych wejściowych dla dużych danych.	36
3.10	Efekty wykonania grupowania GRAGGLO dla danych ‘blobs’ o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.	43
3.11	Efekty wykonania grupowania GRAGGLO dla danych ‘circles’ o rozmiarze 50 000 i euklidesowej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.	44
3.12	Efekty wykonania grupowania GRAGGLO dla danych ‘corners’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.	45
3.13	Efekty wykonania grupowania GRAGGLO dla danych ‘crescents’ o rozmiarze 5000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.	46
3.14	Efekty wykonania grupowania GRAGGLO dla danych ‘laguna’ o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.	47
3.15	Przynależność punktów do klastrów dla danych ‘laguna’ o rozmiarze 1000.	47
3.16	Przynależność punktów do klastrów dla danych ‘laguna’ o rozmiarze 40 000.	48
3.17	Efekty wykonania grupowania hierarchicznego granul dla danych ‘spheres’ o rozmiarze 5000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności SINGLELINKAGE.	48
3.18	Efekty wykonania grupowania GRAGGLO dla danych ‘blobs’ o rozmiarze 1000 i trójkątnej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.	50
3.19	Efekty wykonania grupowania GRAGGLO dla danych ‘circles’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.	51
3.20	Efekty wykonania grupowania GRAGGLO dla danych ‘corners’ o rozmiarze 1000 i wykładniczej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.	52
3.21	Efekty wykonania grupowania GRAGGLO dla danych ‘crescents’ o rozmiarze 1000 i trójkątnej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.	53
3.22	Efekty wykonania grupowania GRAGGLO dla danych ‘laguna’ o rozmiarze 1000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.	54

3.23	Efekty wykonania grupowania hierarchicznego granul dla danych ‘spheres’ o rozmiarze 5000 i gaussowskiej relacji między liczbami rozmytymi, przy łączności COMPLETELINKAGE.	55
3.24	Przynależność punktów do klastrów dla danych ‘blobs’ o rozmiarze 1000. .	56
3.25	Zależność dokładności, trafności i precyzji grupowania od wartości parametru ξ	60
3.26	Zależność dokładności grupowania danych ‘blobs’ od liczby granul, typu relacji i wartości parametru ξ	62
3.27	Zależność dokładności grupowania danych ‘circles’ od liczby granul, typu relacji i wartości parametru ξ	63
3.28	Zależność dokładności grupowania danych ‘corners’ od liczby granul, typu relacji i wartości parametru ξ	65
3.29	Zależność dokładności grupowania danych ‘crescents’ od liczby granul, typu relacji i wartości parametru ξ	66
3.30	Zależność dokładności grupowania danych ‘laguna’ od liczby granul, typu relacji i wartości parametru ξ	67
3.31	Zależność dokładności grupowania danych ‘spheres’ od liczby granul, typu relacji i wartości parametru ξ	68
3.32	Wyniki grupowania hierarchicznego algorytmem ostrym Agglomerative-Clustering i algorytmem GRAGGLO danych ‘brightkite’	70
3.33	Wyniki grupowania hierarchicznego algorytmem ostrym Agglomerative-Clustering i algorytmem GRAGGLO danych ‘gowalla’	71

Spis tabel

3.1	Warunki t-norm i s-norm	14
3.2	Charakterystyka zbiorów danych dołączonych do projektu.	21
3.3	Dokładność grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE	37
3.4	Trafność grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE	37
3.5	Precyzja grupowania hierarchicznego danych algorytmem ostrym o łączności SINGLELINKAGE	38
3.6	Dokładność grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE	38
3.7	Trafność grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE	38
3.8	Precyzja grupowania hierarchicznego danych algorytmem ostrym o łączności COMPLETELINKAGE	39
3.9	Dokładność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności	39
3.10	Trafność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności	40
3.11	Precyzja grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności	40
3.12	Maksymalne wartości metryk jakości dla danych ‘blobs’ przy łączności SINGLELINKAGE.	41
3.13	Maksymalne wartości metryk jakości dla danych ‘laguna’ przy łączności SINGLELINKAGE.	43
3.14	Dokładność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa	50
3.15	Trafność grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa	51

3.16	Precyzja grupowania hierarchicznego danych w zależności od ich kształtu i wykorzystanej łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa	52
3.17	Dokładność grupowania hierarchicznego danych w zależności od liczby granul i łączności	57
3.18	Trafność grupowania hierarchicznego danych w zależności od liczby granul i łączności	57
3.19	Precyzja grupowania hierarchicznego danych w zależności od liczby granul i łączności	57
3.20	Dokładność grupowania hierarchicznego danych w zależności od typu relacji i łączności	58
3.21	Trafność grupowania hierarchicznego danych w zależności od typu relacji i łączności	58
3.22	Precyzja grupowania hierarchicznego danych w zależności od typu relacji i łączności	58
3.23	Dokładność grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa	59
3.24	Trafność grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa	59
3.25	Precyzja grupowania hierarchicznego danych w zależności od typu relacji i łączności, gdy przynależność określana jest na podstawie relacji *-podobieństwa	59